INTERVIEW CHEAT SHEET

Simplest design first → find the number that breaks it → fix only that → repeat.

45 minutes

THE 45 MINUTES

0–5 requirements → 5–8 NFRs + numbers → 8–10 estimates → 10–13 API → 13–25 design → 25–40 deep dives → 40–45 wrap-up

Most of the grade comes from 13–40: show where the simple design breaks, with a number, and fix only that.

NFR CHECKLIST

- ✓ Scale: DAU, peak QPS, read:write ratio
- ✓ Latency: which path, which percentile (p99)
- ✓ Availability: 99.9% ≈ 43 min/month down, 99.99% ≈ 4 min
- ✓ Consistency per feature: money CP, feeds AP
- ✓ Durability: can anything be lost after ack?
- ✓ Data size and growth per year
- ✓ Security, privacy, cost, regions

Estimate: DAU × actions ÷ 86,400 = avg QPS; × 3 = peak. Storage = items/day × size × retention. Compare each number with one machine: if it fits with headroom, don't scale it yet.

PATTERN → FIX

- Too many reads → CDN, cache-aside, replicas
- Too many writes → queue, batch, shard, pre-aggregate
- Live updates → SSE / WebSocket + pub/sub
- Many users, one item → conditional UPDATE, lock + TTL
- Steps across services → saga, outbox, idempotency
- Big files → presigned multipart upload + CDN
- Slow work → queue + workers + job status

Need	Default tech
Source of truth	Postgres
Cache, counters, locks, geo	Redis
Event stream, CDC, buffering	Kafka
Huge key-based writes	Cassandra / DynamoDB
Full-text search	Elasticsearch (via CDC)
Stream aggregation	Flink
Leader election, fencing	ZooKeeper / etcd
Files and media	S3 + CDN

AVOID

- × Technologies without a reason
- × Kafka, sharding, microservices before the numbers
- × Check-then-act in app code
- × 2PC across microservices
- × A Redis lock as the only guard
- × "Eventually consistent" for money
- × Designing silently, then presenting

Say instead: the number, the break, the fix, the trade-off you accept.

Instead of	Say
"Just add servers" for a hot key	Split the key or cache it locally
"Distributed transaction"	Saga + compensations + idempotency
"NoSQL scales better"	This access pattern at this write rate fits Cassandra
"Cache everything"	Cache what can be stale; checkout reads the DB

NUMBERS

1M/day ≈ 12/s · 1B/day ≈ 12k/s · peak ≈ 3× avg

Postgres primary ≈ 10–50k TPS, multi-TB

Redis node ≈ 100k+ ops/s, sub-ms

Kafka ≈ 5–10k msg/s per partition

Cassandra node ≈ 10k+ writes/s

RAM 100ns · SSD 100µs · same-DC RTT 0.5ms

WebSocket server ≈ 100k–1M connections

SAY THIS

- ✓ "Let me pin down the NFRs with numbers first."
- ✓ "At 100k reads/s one Postgres falls over, so I add a cache."
- ✓ "Booking is CP, browsing is AP."
- ✓ "The timeout means unknown outcome, so we reconcile by idempotency key."
- ✓ "One primary handles this write rate, so no sharding yet."
- ✓ "Delete the cache key on write; it is safe under concurrent writers."
- ✓ "Every write takes an idempotency key, so retries are safe."
- ✓ "I'll monitor p99, error rate and queue depth."

QUICK-FIRE

How do you stop a cache stampede?
Single-flight lock per key (only one request recomputes), jittered TTLs, serve stale while revalidating, pre-warm hot keys.

Two users book the last seat. What exactly goes wrong and how do you fix it?
Check-then-act race (write skew).

Why not 2PC across microservices?
It blocks on coordinator failure and couples availability.

What is the outbox pattern?
Write the business row and an event row in one local transaction; a relay/CDC publishes the event.

At-least-once vs exactly-once?
Delivery is at-least-once in practice; make processing idempotent (dedupe by ID, upserts) to get an effectively-once result.

Kafka vs SQS?
Kafka: ordered partitions, replay, multiple consumer groups, stream processing.

How do you make an API retry-safe?
Client-generated Idempotency-Key stored with a unique constraint; return the stored result on duplicates.

Redis lock is enough for correctness?
No. GC pauses or clock issues can leave two holders.

Full playbook: delivery framework · phrases and traps · quick-fire practice

System Design, Step by Step · CC BY-SA 4.0

29 PROBLEMS · STAFF CALLS

For each problem: the number that decides the design, and the call a staff engineer makes.

Staff level

PROBLEMS 1–15

Problem	Deciding number	Staff call
URL shortener (Bitly)	~100k redirects/s peak vs ~12 writes/s; 1B links × 500 B ≈ 500 GB, one Postgres.	Cache-aside Redis plus replicas; counter + base62 with servers leasing ID ranges.
Rate limiter	1M checks/s ≈ 10–20 Redis shards; ~1 GB of bucket state.	Token bucket in a Redis Lua script.
Business discovery (Yelp)	10M businesses × 1 KB ≈ 10 GB; ~20k searches/s at peak.	One Postgres + PostGIS + replicas + page cache.
News feed (Facebook)	115k feed reads/s; ~1.2M feed inserts/s with fan-out on write; celebrities with 100M followers.	Hybrid: push to followers below a threshold (~100k), pull celebrity posts at read.
News aggregator (Google News)	~12k reads/s, 50k+/s on breaking news; ~1 new article/s.	Precompute per region and category, cache at the CDN for ~60 s.
Local delivery (Gopuff)	~10k browse/s at peak vs ~12 orders/s; the 10k-row warehouse list fits in memory.	Browse is AP (cache + replicas); checkout is CP (conditional decrements in one transaction).
Dating app (Tinder)	~100k swipes/s at peak; precomputed stacks ≈ 32 GB.	Precomputed stacks + Bloom filter of swiped profiles; one atomic operation per pair of users for matches.
Distributed cache	1 TB ÷ ~32 GB per node ≈ 32+ nodes; each node ≫ 100k ops/s, so memory sets the count.	Consistent hashing with virtual nodes, async replica per primary.
Top-K videos (YouTube)	70B views/day ≈ 800k/s; 4B videos × 8 B ≈ 32 GB of counters.	Kafka partitioned by video; in-memory counts and local heaps; merge.
Ad click aggregator	10k clicks/s at peak, 100M/day, ~10 GB/day raw.	Kafka → Flink (dedupe by signed impression ID) → OLAP store, plus a nightly batch recount.
Web crawler	~23k pages/s; ~46 GB/s of bandwidth; ~20 PB raw.	Separate fetch and parse stages with queues and raw pages in S3.
Metrics monitoring (Datadog)	5M points/s; ~600 GB/day compressed.	Kafka → time-series DB with compression and downsampling; alerts evaluated on the stream.
Fitness tracking (Strava)	10M activities/day; ~1 TB/day of tracks.	Record on device; idempotent, resumable upload; process asynchronously.
Post search (Facebook)	~12k posts/s and ~10× that in likes.	Inverted index built from the stream, sharded by term; like-based ranking updated in batches or at thresholds.
Price tracker (CamelCamelCamel)	500M products; ~6k pages/s to crawl each one daily.	Crawl by priority (trackers, volatility) + validated crowd-sourced prices + event-driven alerts.

PROBLEMS 16–29

Problem	Deciding number	Staff call
Ride sharing (Uber)	250k location writes/s; ~100 ride requests/s on average.	Redis GEO for locations; offer to one driver at a time with a driver lock and conditional assignment; durable...
Live comments (Facebook Live)	1M viewers × 1k comments/s ≈ 1B deliveries/s for one stream.	SSE + pub/sub per video, viewers of a video routed to the same servers; sample comments on mega-streams.
Collaborative editor (Google Docs)	~500 ops/s on a hot document; correctness, not scale.	One owner server per document running OT, with an op log and snapshots.
Stock trading (Robinhood)	Thousands of updates/s across ~10k symbols; 20M users, so push not poll.	Persist the order as pending before calling the exchange, reconcile on timeout; throttle prices per symbol.
Online auction	~1k bids/s on one auction in its final seconds.	Conditional update on the auction row; a Kafka partition per auction for the hottest ones.
LLM chat (ChatGPT)	~12k generations/s, each holding a GPU for seconds.	Priority queue with admission control in front of the GPU pool, streamed tokens.
Chat app (WhatsApp)	~600k messages/s; hundreds of millions of open connections.	WebSockets + user→server registry + per-user pub/sub; persist before ack; an inbox per device.
Ticketmaster	~100k seats vs 10M users: bookings are capped, reads are not.	Conditional-update holds; virtual waiting room; browse is AP, booking is CP.
Payment system (Stripe)	~10k TPS: moderate.	Pending record first, idempotency keys, outbox, double-entry ledger, daily reconciliation.
Online judge (LeetCode)	~100 submissions/s at peak; ~500 sandboxes running at once.	Queue + microVM sandboxes + autoscaling workers.
Job scheduler	864M executions/day.	Time-bucketed store + poller + delay queue + leases.
Video platform (YouTube)	~1 PB/day raw uploads.	Presigned multipart uploads, a segment-parallel transcoding DAG, CDN delivery.
File storage and sync (Dropbox)	A 50 GB file is ~10k chunks; a 1-byte edit changes one chunk.	Content-defined chunking with dedupe, a change log with cursors.
Photo sharing (Instagram)	~115k feed reads/s; ~200 TB/day of media.	Presigned uploads, async processing, CDN; hybrid fan-out.

Details per problem: the "Staff-level lens" section on each problem page

System Design, Step by Step · CC BY-SA 4.0

STAFF ENGINEER CHEAT SHEET

Frame the tension · say the number · decide and own the trade-off · raise failures first · run it · evolve it.

Staff level

THE STAFF LENS

Frame the tension → Quantify: the deciding number → Decide + trade-off → Anticipate failures → Operate: SLOs, metrics → Evolve: what at 10×

A senior answer gets a working design. A staff answer explains why this design, what it costs, how it fails, and how it will be run.

SENIOR VS STAFF

Area	Senior	Staff
Scope	Designs what was asked	Clarifies the goal, cuts scope on purpose
Numbers	Estimates QPS, storage	One decisive number rules options in or out
Trade-offs	Lists pros and cons	Chooses, says what is given up and when it changes
Failure	Handles it when asked	Raises retries, idempotency, recovery unprompted
Consistency	Picks a database	CP or AP per feature, with the cost
Operations	Mentions monitoring	SLOs, alerts, rollout and rollback
Cost	Rarely	Prices the expensive part, designs around it
Evolution	Stated scale only	What breaks at 10×; simple today on purpose
Conversation	Follows	Drives the agenda, checks in, invites steering

RAISE BEFORE YOU ARE ASKED

- ! Timeouts with unknown outcome → idempotency keys, reconcile
- ! Retries → at-least-once delivery, idempotent consumers
- ! Hot keys and celebrity users → split keys, hybrid fan-out
- ! Cache stampede → single-flight, jittered TTLs, serve stale
- ! Replica lag → read-your-writes where it matters
- ! Leader or node failure → failover time, data loss window, fencing
- ! Thundering herds → backoff with jitter, admission control, waiting rooms
- ! Abuse and bots → rate limits, signed tokens
- ! Deploys → draining connections, backward-compatible schemas

RUN IT

- ✓ One SLO per critical path (p99 latency, success rate)
- ✓ Golden signals: latency, traffic, errors, saturation
- ✓ Pipeline health: queue depth, consumer lag, freshness
- ✓ Roll out behind flags; shadow mode for risky logic
- ✓ Rollback plan for every schema or config change
- ✓ Reconciliation jobs where money or inventory is involved

SAY THIS

- ✓ "The number that decides this is..."
- ✓ "I'll take X and accept Y; I'd revisit it if Z."
- ✓ "Before we go on, the failure I worry about is..."
- ✓ "This path is CP, that one is AP, because..."
- ✓ "I'd watch A and B, and alert on C."
- ✓ "At 10× the first thing to break is..., so I'd..."

COST AND SCALE CHECKS

Fits one machine? Keep it there.

Most expensive resource: GPUs, storage, egress?

Peak = 3–5× average; plan for peak

Cross-region traffic and storage tiers cost money

Every new system adds on-call load

System Design, Step by Step · CC BY-SA 4.0

AD CLICK AGGREGATOR

Clicks are money, so the numbers must be fast for dashboards and exact for billing.

Hard | Scaling writes

TL;DR

1. Redirect immediately and publish the click to Kafka; never aggregate raw rows at query time.
2. A stream processor counts clicks per ad per minute and writes them to a column store for fast queries.
3. Deduplicate by a signed impression ID, and recount from raw logs in a nightly batch job for billing accuracy.

REQUIREMENTS

- ✓ Click on an ad redirects the user to the advertiser
- ✓ Advertisers query click metrics over time (minute granularity), with filters

Non-functional

- 10k clicks/s peak, 100M/day
- Never lose or double-count clicks (it is billing)
- Metrics queries < 1s
- Freshness ~1 minute

NUMBERS

100M clicks × 100B ≈ 10GB/day raw.

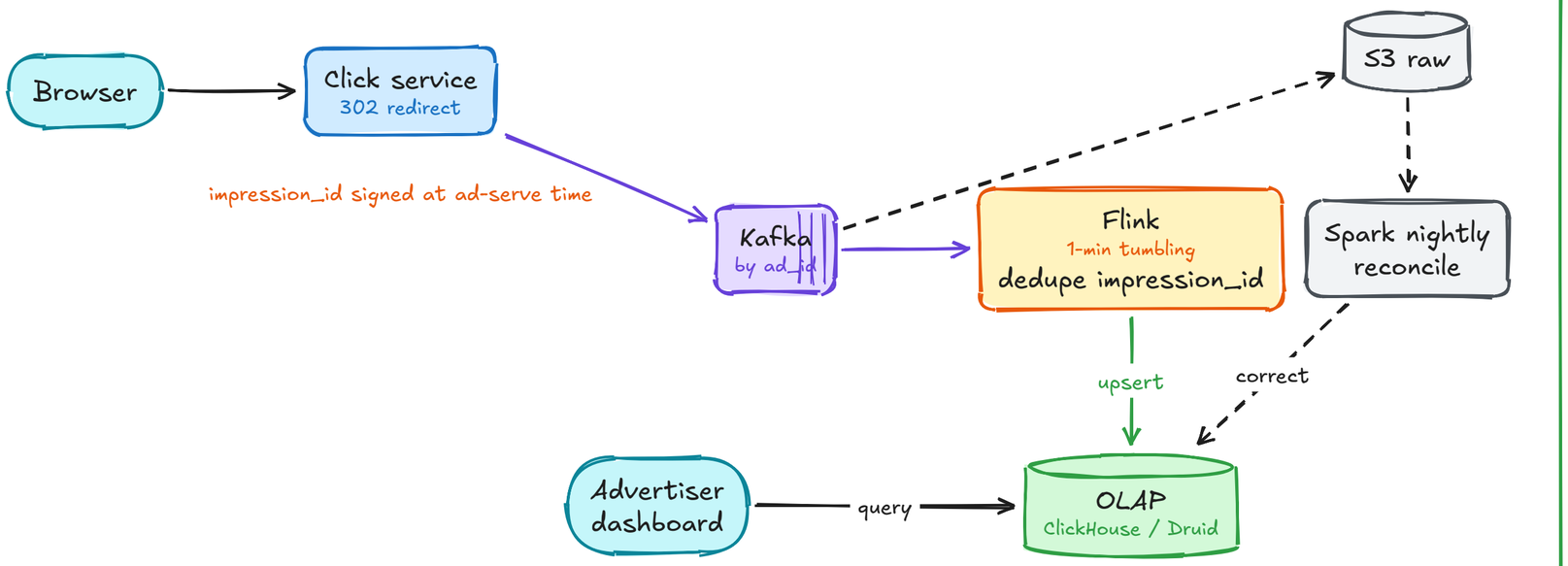
Aggregated rows: ads × minutes, which is small.

A 10k/s peak is easy for Kafka.

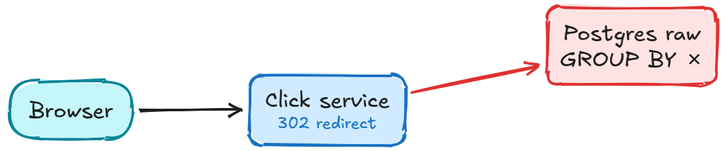
API + DATA

```
GET /click?ad_id&impression_id → 302 advertiser_url
GET /metrics?ad_id&from&to&granularity
click(impression_id, ad_id, ts, user) → agg(ad_id, minute, count)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: insert each click into Postgres and GROUP BY at query time. Slow queries, and writes compete with reads.

DEEP DIVES

Flink crashes?
Checkpoints + Kafka offsets give replay; the sink upserts (ad_id, minute), so replays overwrite instead of double-counting.

Hot ad (Super Bowl)?
Key on ad_id:random(0..N) to spread across partitions, then aggregate twice.

Why also batch?
Streams give fast numbers; batch from immutable raw data gives exact billing numbers.

Why OLAP?
Column store and pre-aggregates make time-range queries over billions of rows sub-second.

HOW IT EVOLVES

1. Click service redirects immediately and publishes to Kafka. Flink aggregates per (ad, minute) on event time and upserts into an OLAP column store.
2. Idempotency: every impression gets a signed unique ID when the ad is served; Flink dedupes on it, so double-clicks and retries count once and forged clicks are rejected.
3. Lambda-style safety net: raw clicks to S3; a nightly batch recount corrects aggregates, because billing must be exact.

AVOID

- × Aggregating raw rows at query time in the OLTP database.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"A stream processor counts clicks per ad per minute and writes them to a column store for fast queries."

DDIA: Ch 11 windows, exactly-once · Ch 12 Lambda · Ch 3 column stores · Similar: Top-K videos · Web crawler · Metrics monitoring · Fitness tracking

System Design, Step by Step · CC BY-SA 4.0

BUSINESS DISCOVERY (YELP)

The trap is over-engineering: the whole dataset fits on one database server.

Medium | Scaling reads

TL;DR

1. Ten million businesses at ~1 KB each is about 10 GB, small enough for one Postgres instance.
2. Two-dimensional "near me" queries need a geospatial index (PostGIS, geohash), not two B-tree indexes.
3. Keep the average rating correct under concurrent reviews by updating it atomically in the same transaction.

REQUIREMENTS

- ✓ Search businesses by location + radius, category, text
- ✓ View business details and reviews
- ✓ Leave one review per user per business; see average rating

Non-functional

- Search p99 < 500ms
- 100M DAU, read-heavy
- Ratings can lag slightly (eventual)
- Reviews durable

NUMBERS

10M businesses × 1KB = 10GB, which fits in memory.

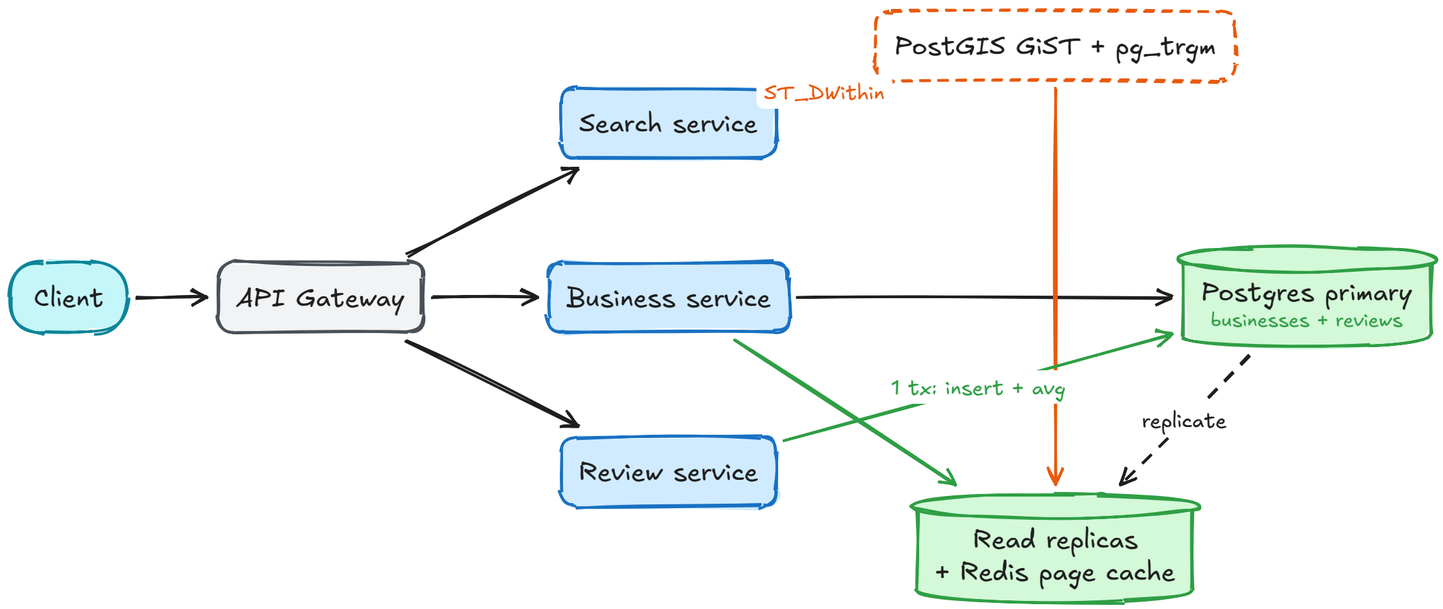
Search: 100M × 5 / 86,400 ≈ 6k/s, ~20k/s peak.

Reviews ~12/s.

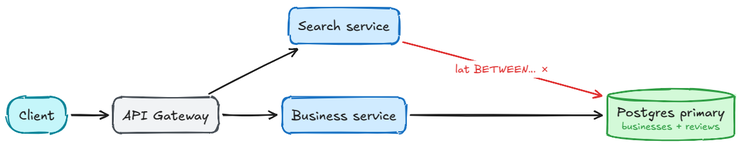
API + DATA

```
GET /businesses?lat&lng&radius&category&q&cursor
GET /businesses/{id}
POST /businesses/{id}/reviews {rating, text}
businesses(id, name, location, category, avg_rating, num_ratings)
reviews(id, business_id, user_id, rating, text, UNIQUE(user_id, business_id))
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: WHERE lat BETWEEN … AND lng BETWEEN …. Two 1-D B-tree indexes can't do a 2-D range efficiently.

DEEP DIVES

Which geo index?
Geohash: prefix = area, B-tree friendly; query the cell plus 8 neighbours to handle edges. Quadtree: adapts to density.

Average rating with concurrent reviews?
Store sum and count; UPDATE … SET sum=sum+:r, count=count+1 in the review's transaction. That is an atomic operation, so no lost update.

One review per user?
A DB unique constraint, not an application check (check-then-insert races).

When would you add Elasticsearch?
Only if search needs relevance ranking, fuzzy matching, and facets beyond pg_trgm, or QPS outgrows replicas.

HOW IT EVOLVES

1. Add a geospatial index (PostGIS GiST, ST_DWithin) and pg_trgm for name search. 10GB fits one Postgres, so no extra search cluster yet.
2. ~20k searches/sec: serve reads from replicas and cache hot business pages in Redis. The primary only takes writes.
3. Reviews: a unique (user, business) constraint, and the average updated in the same transaction as the insert, with no lost updates.

AVOID

- × A geo-sharded cluster for 10GB of data. One Postgres with PostGIS plus replicas is enough.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Two-dimensional "near me" queries need a geospatial index (PostGIS, geohash), not two B-tree indexes."

DDIA: Ch 3 geo indexes · Ch 7 atomic updates · Ch 5 read replicas · Similar: URL shortener · Rate limiter · News feed · News aggregator

System Design, Step by Step · CC BY-SA 4.0

CHAT APP (WHATSAPP)

Chat looks like CRUD until you notice the two users are on different servers and one of them is offline.

Medium | Real-time updates

TL;DR

1. Keep a persistent WebSocket per device; route messages between chat servers with a registry and per-user pub/sub.
2. Persist each message and add it to the recipient's inbox before acknowledging the sender, so nothing is lost.
3. Partition by conversation and number messages within it so every device shows the same order.

REQUIREMENTS

- ✓ 1:1 and group chats (up to ~100 members)
- ✓ Send and receive messages in real time
- ✓ Offline users get messages when they reconnect
- ✓ Delivery and read receipts; media

Non-functional

- Delivery < 500ms when online
- Messages never lost
- Ordered within a chat
- 1B DAU

NUMBERS

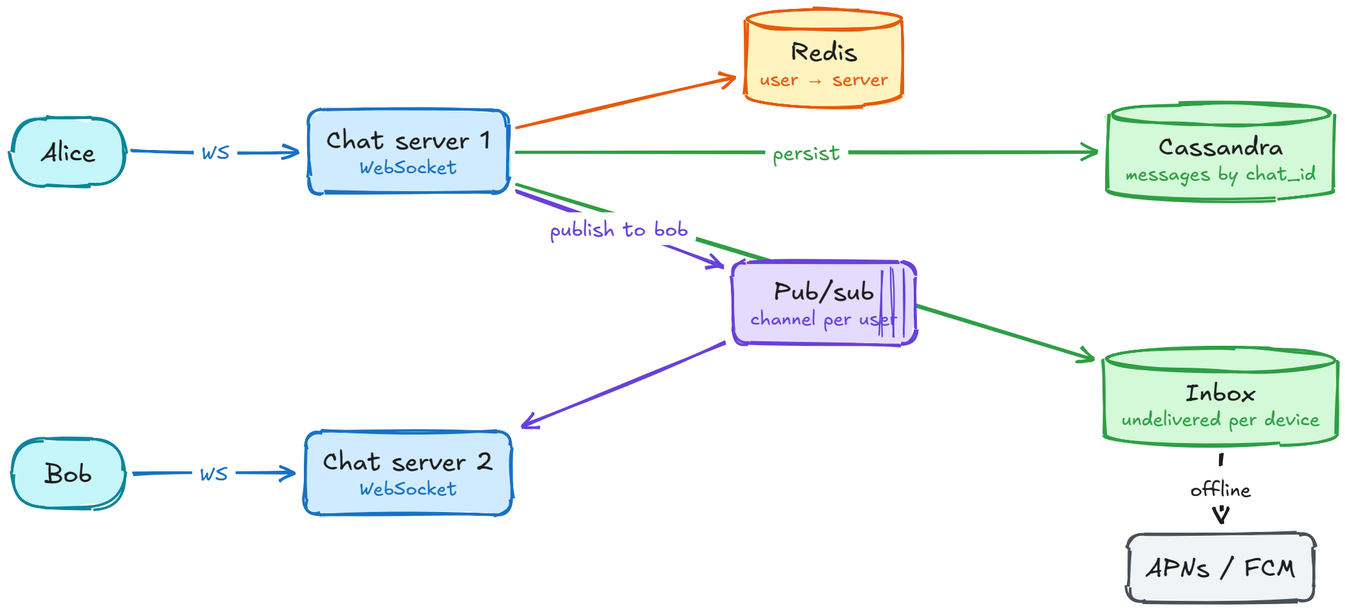
50B messages/day ≈ ~600k msgs/s.

Hundreds of millions of concurrent connections, at ~1M connections per tuned server → hundreds of chat servers.

API + DATA

```
WS connect → send {chat_id, client_msg_id, body}; receive {msg, seq}; ack {msg_id}
POST /media → presigned upload URL
messages(chat_id, seq, msg_id, sender, body, ts)
partitioned by chat_id
inbox(device_id, msg_id) until acked
chat_members
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Persistent WebSocket connections for both directions, so there is no polling.

Alice and Bob are usually connected to different servers, so you need routing between them.

DEEP DIVES

Never lose a message?
Persist before ack; at-least-once delivery from the inbox; client dedupes by msg_id.

Ordering?
Per-chat sequence numbers, with chat_id as the partition key so one writer orders them (DDIA Ch 5 consistent prefix).

Groups?
Fan-out per member at send time (≤ ~100 members) into inboxes and channels.

Media?
Upload to S3 via presigned URL; the message carries only the object key.

HOW IT EVOLVES

1. Alice and Bob are usually connected to different servers, so you need routing between them.
2. A registry (user → server) and a pub/sub channel per user: server 1 publishes to Bob's channel; server 2, which holds Bob's connection, delivers.
3. Durability first: persist to the messages table and the recipient's inbox before acknowledging Alice.

AVOID

- × Assuming both users share a server or are always online.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Persist each message and add it to the recipient's inbox before acknowledging the sender, so nothing is lost."

DDIA: Ch 5 consistent prefix · Ch 11 at-least-once + idempotency · Similar: Ride sharing · Live comments · Collaborative editor · Stock trading

System Design, Step by Step · CC BY-SA 4.0

COLLABORATIVE EDITOR (GOOGLE DOCS)

The challenge is correctness, not scale: everyone must end up with the same document.

Hard | Real-time updates

TL;DR

1. Send small operations, not whole documents; last-write-wins on documents loses people's work.
2. One server per document orders operations and transforms concurrent ones (OT); CRDTs are the serverless alternative.
3. Append every operation to a log before acknowledging it, and snapshot periodically for fast loading.

REQUIREMENTS

- ✓ Create and open documents
- ✓ Many users edit concurrently and see each other's changes live
- ✓ See collaborators' cursors and presence

Non-functional

- Remote edits visible < 100ms
- All replicas converge to the same document
- No lost edits (durable)
- ~100 concurrent editors per doc; millions of docs

NUMBERS

Each keystroke is an op (~100B).

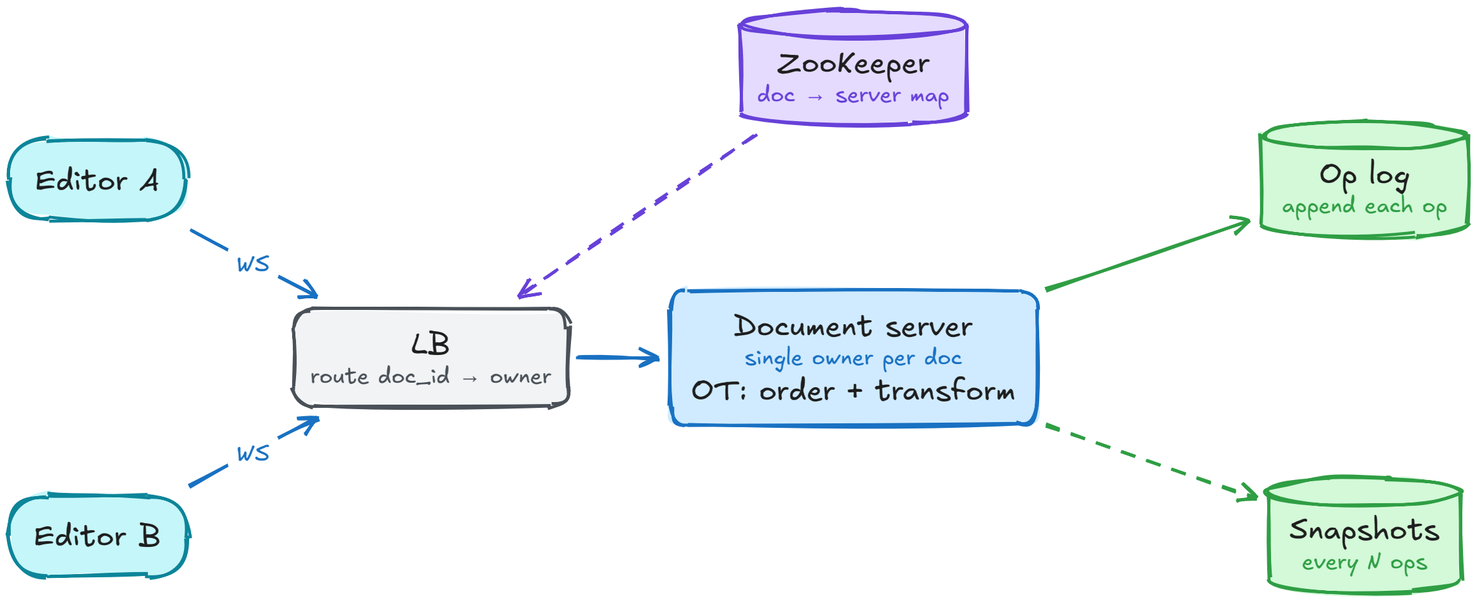
100 editors × 5 ops/s = 500 ops/s per hot doc, which one server handles easily.

The challenge is correctness, not scale.

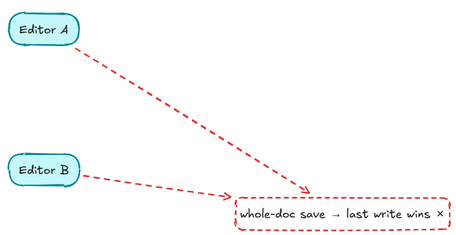
API + DATA

```
WS /docs/{id}: send op {type, pos, char,
base_version}; receive transformed ops
GET /docs/{id} (snapshot + version)
doc_ops(doc_id, version, op) append-only
doc_snapshots(doc_id, version, content)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: send the whole document on save. Last write wins, and someone's work disappears.

DEEP DIVES

OT or CRDT?
OT: a central server orders and transforms; compact; needs one coordinator per doc (Google Docs).

Why one server per doc?
OT needs a single total order of ops (DDIA Ch 9).

Offline edits?
Client buffers ops tagged with a base version; on reconnect the server transforms them against everything since that version.

Cursors?
Ephemeral presence over the same socket; not persisted.

HOW IT EVOLVES

1. Send operations (insert "x" at 5).
2. Every op is appended to the op log before it is acknowledged; periodic snapshots make loading fast (snapshot + tail).
3. All editors of a doc must reach the same owner server: consistent hashing on doc_id, with ownership coordinated through ZooKeeper.

AVOID

- × Locking the document or paragraph, which defeats real-time collaboration.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"One server per document orders operations and transforms concurrent ones (OT); CRDTs are the serverless alternative."

DDIA: Ch 5 conflict resolution · Ch 9 total order · Ch 11 event log · Similar: Ride sharing · Live comments · Stock trading · Online auction

System Design, Step by Step · CC BY-SA 4.0

DATING APP (TINDER)

Swipes are a firehose of writes, and two people swiping right at the same moment must never miss their match.

Medium | Scaling reads

TL;DR

1. Precompute each user's stack of nearby candidates and hide already-swiped profiles with a Bloom filter.
2. Swipes are write-heavy and append-only, a good fit for an LSM store like Cassandra.
3. Detect matches atomically on one key per pair of users, so simultaneous swipes can't both miss each other.

REQUIREMENTS

- ✓ Profile with preferences (age, distance)
- ✓ Get a stack of nearby candidates
- ✓ Swipe left/right; get notified on a mutual match

Non-functional

- Stack load p99 < 300ms
- No missed matches (strong consistency on swipes)
- Don't re-show swiped profiles
- 20M DAU × 100 swipes

NUMBERS

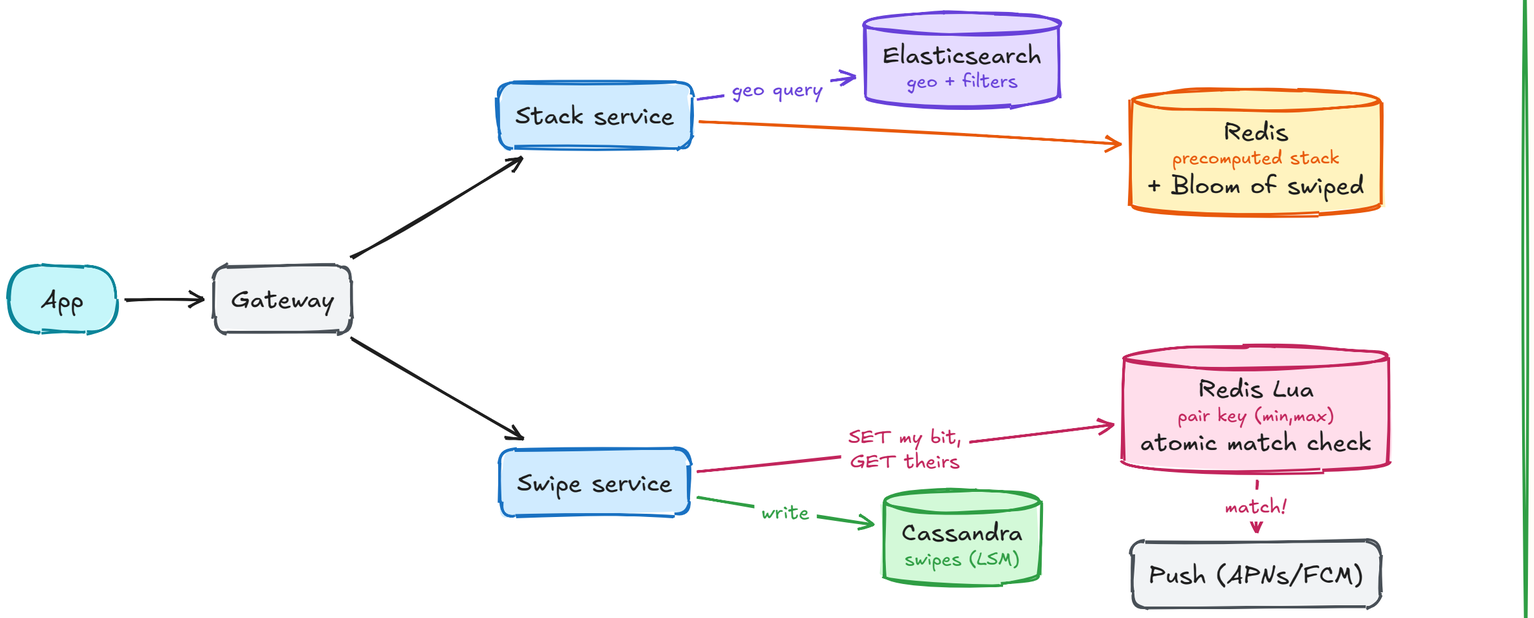
2B swipes/day ≈ 25k/s avg, ~100k/s peak, which is write-heavy.

Stacks precomputed: 20M × 200 IDs × 8B ≈ 32GB.

API + DATA

```
GET /feed?lat&lng → [profiles]
POST /swipes {target_id, decision}
push: match notification
profiles
swipes(swiper_id, swipee_id, decision) partitioned by swiper
matches(a, b, ts)
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Build each stack with a geo + preferences query in Elasticsearch. That works, but is slow on every open.

Precompute the next ~200 candidates per user in Redis and refresh in the background. A per-user Bloom filter hides profiles already swiped.

DEEP DIVES

Both swipe at once: why can a match be missed?
Two writes followed by two reads: each read runs before the other's write lands. That is write skew (Ch 7).

Hide already-swiped profiles?
A Bloom filter per user. A false positive skips an unseen profile, which is acceptable; no false negatives.

Location changes?
Invalidate and recompute the stack when the user moves more than X km.

HOW IT EVOLVES

1. Precompute the next ~200 candidates per user in Redis and refresh in the background. A per-user Bloom filter hides profiles already swiped.
2. Swipes are write-heavy (100k/s) and append-only, so an LSM store like Cassandra fits.
3. Match race: A and B swipe right at the same moment, and each check misses the other.

AVOID

- × Separate "write swipe" and "check other swipe" calls without atomicity, so matches are lost.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Swipes are write-heavy and append-only, a good fit for an LSM store like Cassandra."

DDIA: Ch 7 write skew · Ch 3 Bloom filters, LSM · Similar: URL shortener · Rate limiter · Business discovery · News feed

System Design, Step by Step · CC BY-SA 4.0

DISTRIBUTED CACHE

Design Redis itself: data structures inside one node, then distribution across many.

Medium | Scaling reads | Long-running tasks

TL;DR

1. One node is a hash map plus a doubly linked list for O(1) least-recently-used eviction.
2. Spread keys with consistent hashing and virtual nodes, so adding a node moves only a small share of keys.
3. Replicate asynchronously for failover, and spread hot keys over several nodes.

REQUIREMENTS

- ✓ get(key), set(key, value, ttl), delete(key)
- ✓ LRU eviction when full

Non-functional

- p99 ~1ms
- ~1TB data, 100k+ ops/s
- Survive node failure
- Eventual consistency acceptable

NUMBERS

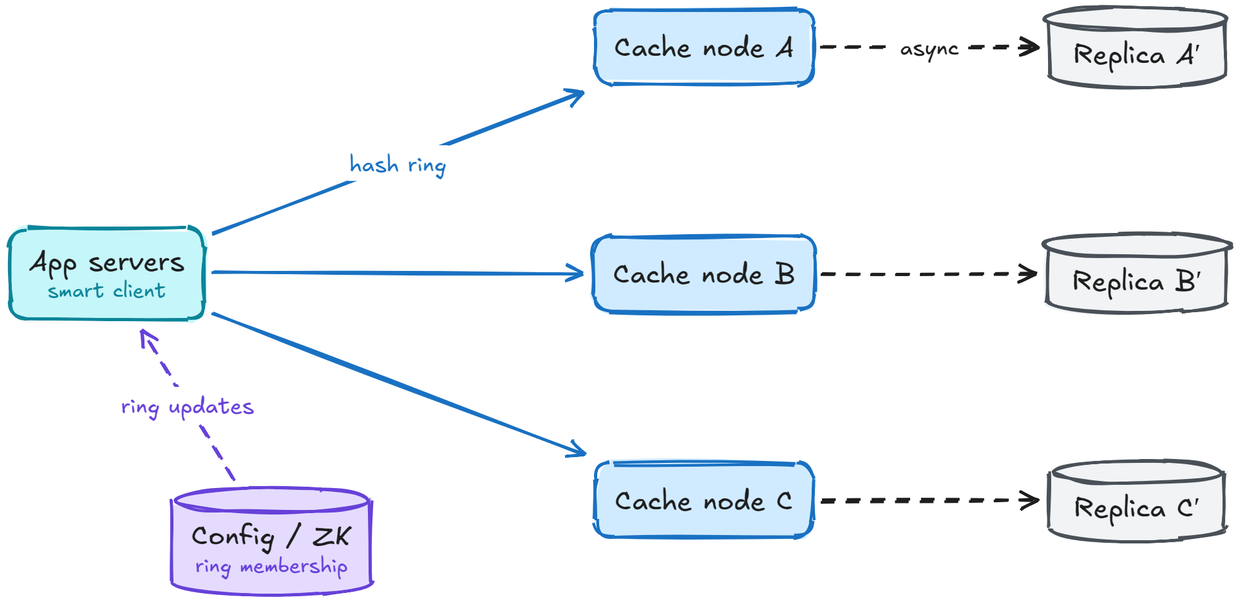
1TB ÷ ~32GB usable per node ≈ 32+ nodes (+ replicas).

Each node handles ≫ 100k ops/s, so memory, not CPU, sets the node count.

API + DATA

```
GET /cache/{key}
PUT /cache/{key} {value, ttl}
DELETE /cache/{key}
per node: HashMap key → node in a doubly linked list (LRU order) + expiry
```

FINAL DESIGN



V1, AND WHY IT BREAKS



One node: a HashMap for O(1) lookup plus a doubly linked list for LRU order (move to head on access, evict the tail). TTL by lazy check plus a background sweep.

DEEP DIVES

LRU in O(1)?
HashMap → linked-list node; get = move to head; evict = remove tail.

Add a node?
Consistent hashing moves ~1/N of keys; vnodes keep load even; warm gradually.

Hot key?
Copies key#0..k on several nodes with random reads, or a client-side L1 cache.

Why not hash % N?
Every scale event remaps ~all keys, and the cold cache stampedes the database.

HOW IT EVOLVES

1. Shard across nodes with consistent hashing (with virtual nodes). The client computes the owner directly, so there is no extra hop.
2. Each primary replicates asynchronously to a replica; on failure the replica is promoted.
3. Ring membership lives in a config service; clients watch it. Hot keys get replicated across several nodes or a short-TTL local cache.

AVOID

- × hash(key) % N sharding.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Spread keys with consistent hashing and virtual nodes, so adding a node moves only a small share of keys."

DDIA: Ch 6 consistent hashing · Ch 5 async replication · Similar: URL shortener · Rate limiter · Business discovery · News feed

System Design, Step by Step · CC BY-SA 4.0

FILE STORAGE AND SYNC (DROPBOX)

Syncing a 50 GB file after a one-byte edit should not mean uploading 50 GB again.

Easy | Handling large blobs

TL;DR

1. Split files into chunks fingerprinted by hash, and upload only the chunks the server doesn't already have.
2. Upload and download directly to object storage with presigned URLs.
3. Other devices follow a per-user change log with a cursor and fetch only what changed.

REQUIREMENTS

- ✓ Upload and download files (up to 50GB)
- ✓ Automatic sync across a user's devices
- ✓ Share files and folders

Non-functional

- Resumable uploads
- Changes visible on other devices within seconds
- Durable (S3 11 nines)
- Bandwidth-efficient sync

NUMBERS

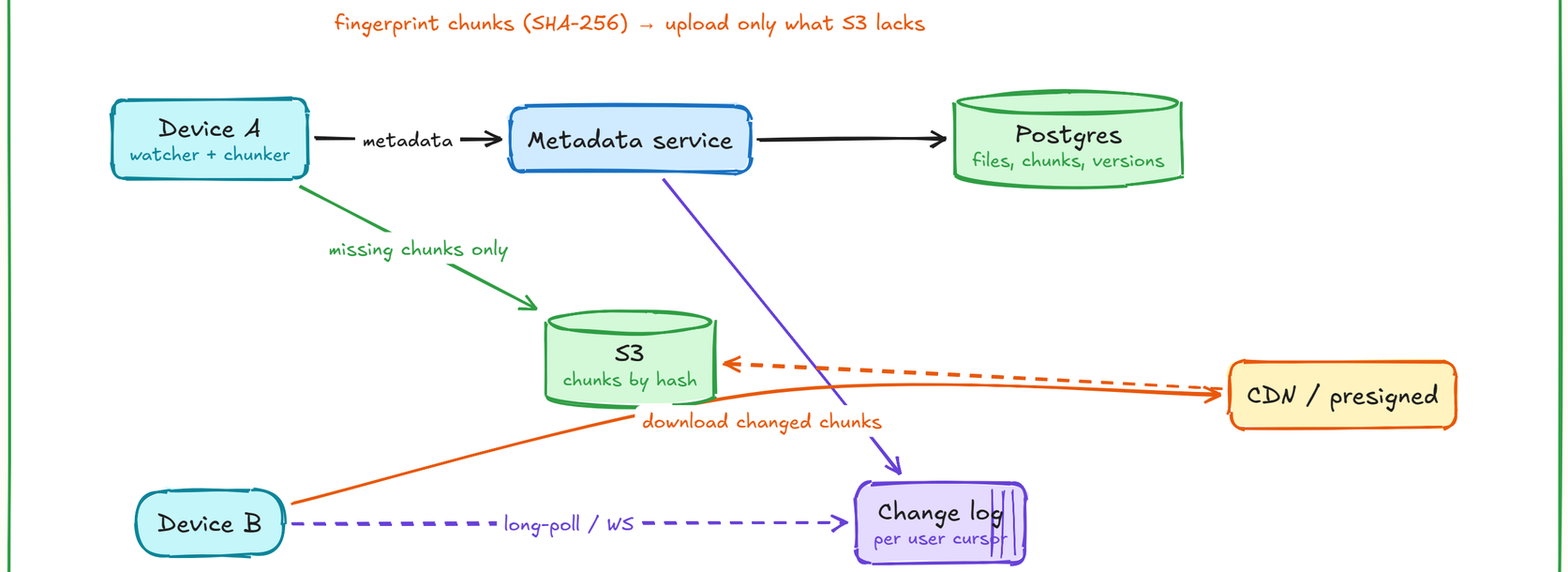
Large files chunked at ~5MB: a 50GB file = 10k chunks.

Only changed chunks are re-uploaded, so a 1-byte edit sends 5MB, not 50GB.

API + DATA

```
POST /files {name, size, chunk_hashes[]} → missing chunks + presigned URLs
GET /changes?cursor → file metadata diffs
GET /files/{id} → chunk list + presigned download URLs
files(id, owner, path, version)
file_chunks(file_id, idx, sha256)
chunks(sha256, s3_key)
shares(file_id, user, perm)
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Naive: upload the whole file through the API. A 50GB upload dies at 99% and starts over.

Split into ~5MB chunks; upload directly to S3 via presigned URLs, which is resumable and parallel.

DEEP DIVES

Resume after failure?
The server knows which chunk hashes exist; the client re-sends only missing ones.

How do devices learn about changes?
Change log + cursor per device; long-poll or WebSocket notifications.

Edits on two offline devices?
Upload carries a base version; if stale, save a "conflicted copy" (DDIA Ch 5 multi-leader conflict).

Trust the client said "done"?
Verify via S3 event or ETags before marking the file version committed.

HOW IT EVOLVES

1. Split into ~5MB chunks; upload directly to S3 via presigned URLs, which is resumable and parallel.
2. Fingerprint each chunk, using content-defined chunking so an insert near the start doesn't shift every boundary.
3. Other devices follow a per-user change log with a cursor (long-poll or WebSocket), fetch the metadata diff, and download only the changed chunks.

AVOID

- × Proxying file bytes through API servers, and re-uploading whole files on small edits.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Upload and download directly to object storage with presigned URLs."

DDIA: Ch 5 conflicts · Ch 11 change log / cursor · Similar: Video platform · Photo sharing

System Design, Step by Step · CC BY-SA 4.0

FITNESS TRACKING (STRAVA)

The key insight is where the work happens: on the phone, not the server.

Medium | Scaling writes

TL;DR

1. Record GPS on the device and upload the whole activity when it ends; no need to stream every point.
2. Store the track in object storage and the metadata in a database, with an idempotent, resumable upload.
3. Compute stats and leaderboards asynchronously from a queue; leaderboards are Redis sorted sets.

REQUIREMENTS

- ✓ Start/pause/stop recording an activity (GPS track)
- ✓ View activity stats and maps
- ✓ Share with friends; see friends' activities; leaderboards

Non-functional

- Works offline; never lose an activity
- 10M DAU
- Feed eventually consistent
- Uploads resumable

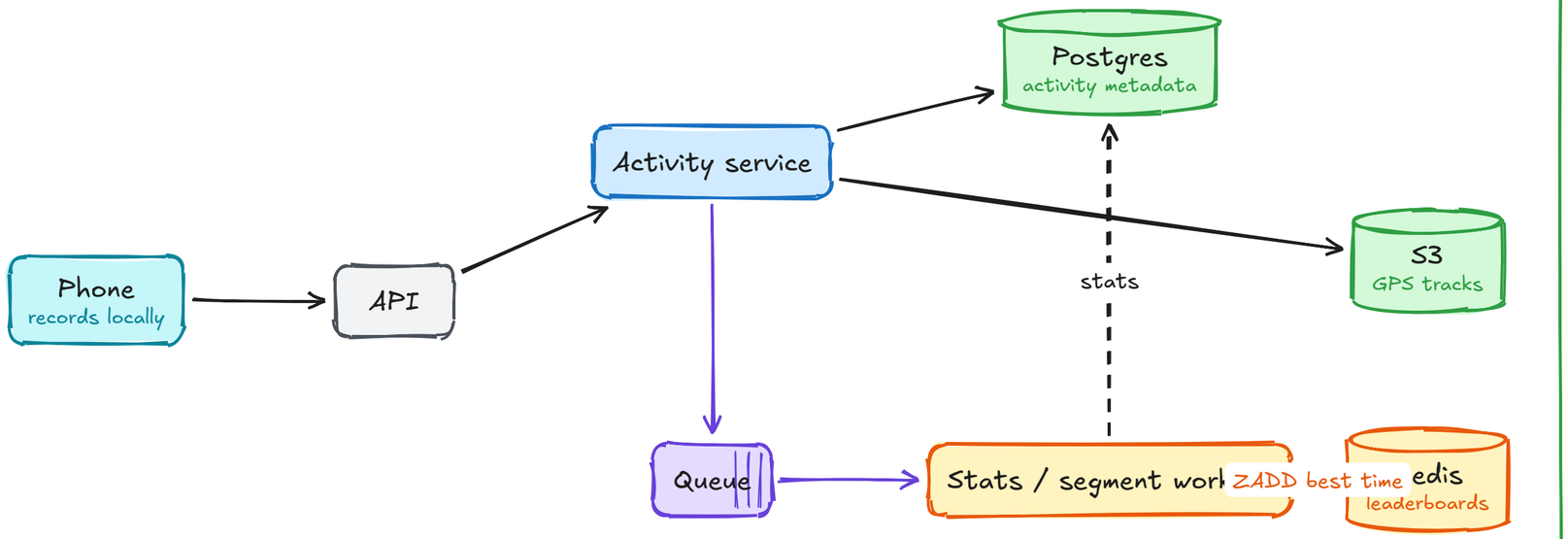
NUMBERS

10M activities/day; 1hr at 1 point/5s = 720 points ≈ 15–100KB per track → ~1TB/day of track blobs.

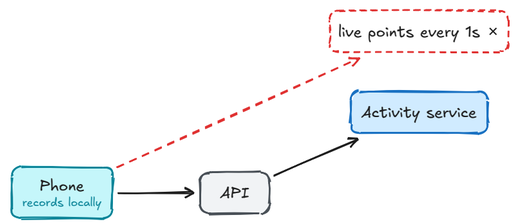
API + DATA

```
POST /activities (complete track, idempotency key)
GET /activities/{id}
GET /feed?cursor
activities(id, user_id, type, start, distance, duration,
track_s3_key)
follows
leaderboard:{segment} ZSET
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: stream every GPS point to the server in real time. 10M devices means millions of writes/sec, for no user value.

DEEP DIVES

Why device-first?
Signal is unreliable; users need it to work offline. Only optional live tracking sends periodic points (best effort).

Upload reliability?
Idempotency key = activity_id; resumable chunks; retry until acked.

Leaderboards?
ZADD segment:{id} with the time as score; ZRANGE for top *N*; snapshot to DB.

Time correctness?
Use device timestamps (event time), not upload time.

HOW IT EVOLVES

1. Record on the device (works in tunnels and mountains). Upload the whole activity when finished, with an idempotency key and resumable upload.
2. Track blob to S3, metadata to Postgres.
3. Heavy processing (stats, segment matching) runs asynchronously from a queue; leaderboards are Redis sorted sets per segment.

AVOID

- × Server-side real-time ingestion of every GPS point.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Store the track in object storage and the metadata in a database, with an idempotent, resumable upload."

DDIA: Ch 11 event time · Ch 12 idempotent upload · Similar: Top-K videos · Ad click aggregator · Web crawler · Metrics monitoring

System Design, Step by Step · CC BY-SA 4.0

JOB SCHEDULER

Ten thousand jobs per second, each within two seconds of its scheduled time.

Hard | Multi-step processes | Long-running tasks

TL;DR

1. Cron on one machine is a single point of failure that can't scale.
2. Store executions partitioned by time; a poller moves soon-due jobs into a delay queue that releases each on time.
3. Workers lease jobs with a visibility timeout, so crashed jobs are retried; jobs must be idempotent.

REQUIREMENTS

- ✓ Schedule jobs: one-off at time T, or cron
- ✓ Execute jobs with a retry policy
- ✓ View job status and history

Non-functional

- 10k job executions/s
- Start within 2s of scheduled time
- At-least-once; jobs idempotent
- Never lose a job; billions of jobs stored

NUMBERS

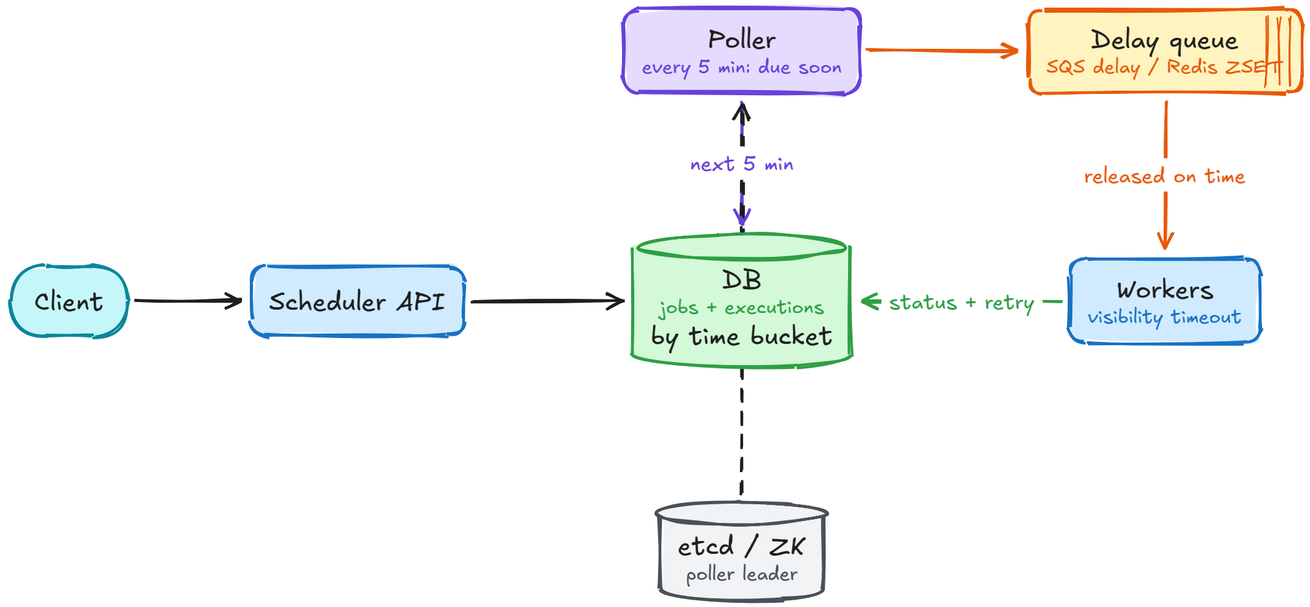
10k executions/s = 864M/day.

A DB poll every second for "due now" across billions of rows will not hold, so use a two-layer design.

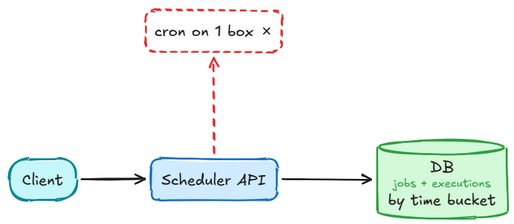
API + DATA

```
POST /jobs {schedule (cron | at), payload, retry_policy}
GET /jobs/{id}/executions
jobs(id, user, schedule, payload)
executions(time_bucket, execution_id, job_id,
scheduled_at, status, attempt) partitioned by time
bucket
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: cron on one server. A single point of failure that can't scale.

DEEP DIVES

How do you hit 2s precision?
DB scan every few minutes → delay queue (SQS DelaySeconds ≤ 15 min, or a Redis ZSET by timestamp) releases on time.

Worker dies mid-job?
Lease expires and the job is retried. Heartbeats extend leases for long jobs; fencing token on completion writes.

Duplicate enqueues?
Dedupe by execution_id at the queue or worker.

User history queries?
Secondary index on (user_id, scheduled_at).

HOW IT EVOLVES

1. Store executions in a DB partitioned by time bucket; a poller fetches executions due in the next few minutes.
2. Two layers: the poller hands near-due jobs to a delay queue that releases each one at its exact time, giving second-level precision without hammering the DB.
3. Workers take jobs with a visibility timeout (lease). A crash means the job reappears and is retried, so jobs must be idempotent.

AVOID

- × Polling the entire jobs table every second.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Store executions partitioned by time; a poller moves soon-due jobs into a delay queue that releases each on time."

DDIA: Ch 9 leader election · Ch 8 leases / fencing · Ch 6 time partitioning · Similar: Payment system · Online judge

System Design, Step by Step · CC BY-SA 4.0

LIVE COMMENTS (FACEBOOK LIVE)

One popular stream can mean a million viewers each expecting new comments within a fraction of a second.

Medium | Real-time updates

TL;DR

1. Polling at this scale is a self-inflicted denial of service; push with Server-Sent Events.
2. Write each comment once and publish it once; pub/sub fans it out to every server holding viewers of that video.
3. Route viewers of the same video to the same servers so each server subscribes to few channels.

REQUIREMENTS

- ✓ Post a comment on a live video
- ✓ Viewers see new comments in near real time
- ✓ Late joiners can scroll back through earlier comments

Non-functional

- Delivery p99 < 200ms
- Millions of concurrent viewers; mega-streams with 1M+ viewers
- Eventual consistency and ordering are acceptable

NUMBERS

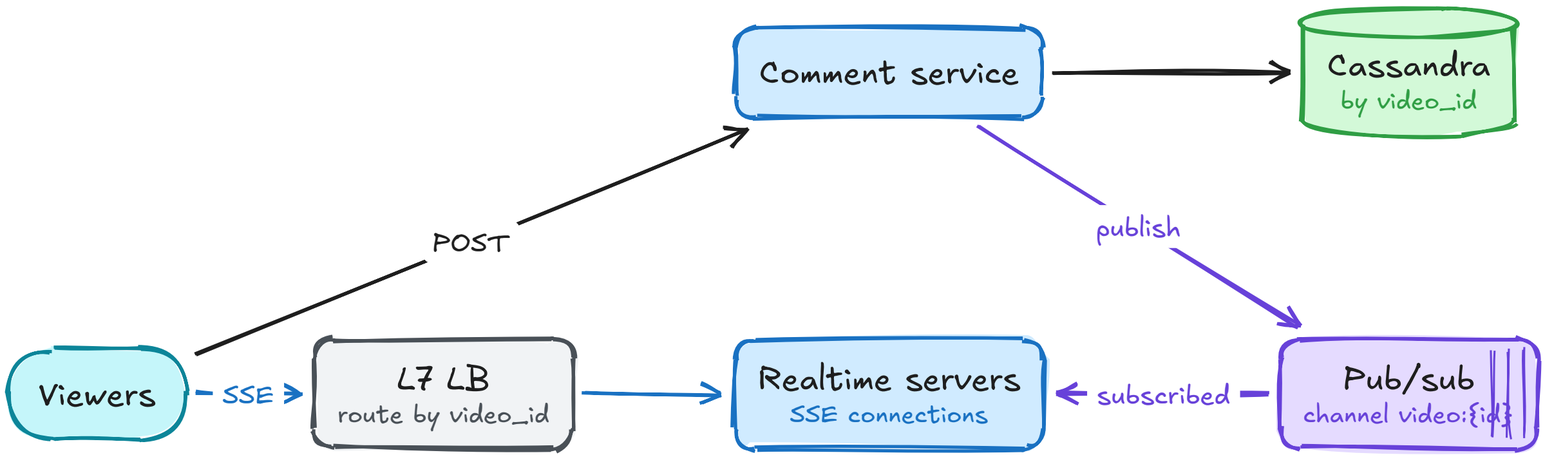
Hot stream: 1M viewers × 1k comments/s = 1B deliveries/s for one video.

Polling every 2s = 500k req/s from that one stream.

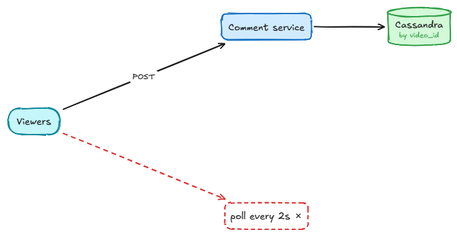
API + DATA

```
POST /videos/{id}/comments
GET /videos/{id}/comments?cursor (history)
GET /videos/{id}/comments/stream (SSE)
comments(video_id, comment_id, user_id, text, ts)
partitioned by video_id, clustered by ts
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: viewers poll for new comments. Latency = poll interval, and 1M viewers create a self-inflicted DDoS.

DEEP DIVES

SSE or WebSocket?
Traffic is server→client. SSE is plain HTTP with auto-reconnect; WebSocket is only needed for heavy bidirectional traffic.

How does one comment reach 1,000 servers?
Pub/sub fan-out: publish once; each server is subscribed to the channels of videos its viewers watch.

Mega-streams?
Sample or rate-limit the comment stream per viewer (nobody can read 1k/s); prioritize friends' comments.

Late join?
Page the last *N* from Cassandra by cursor, then attach to the stream.

HOW IT EVOLVES

1. Server push over SSE: one long-lived HTTP connection per viewer. Posting a comment stays a normal POST.
2. A comment is written once to Cassandra and published once to a pub/sub channel for that video.
3. An L7 load balancer routes viewers of the same video to the same servers (consistent hashing on video_id), so each server subscribes to few channels.

AVOID

- × Polling, which gives high latency and huge wasted load.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Write each comment once and publish it once; pub/sub fans it out to every server holding viewers of that video."

DDIA: Ch 11 pub/sub · Ch 6 partition by video · Similar: Ride sharing · Collaborative editor · Stock trading · Online auction

System Design, Step by Step · CC BY-SA 4.0

LLM CHAT (CHATGPT)

GPUs are the scarce resource, so every queueing decision is really a cost decision.

Hard | Real-time updates

TL;DR

1. Stream tokens to the client as they are generated so the first words appear in under a second.
2. Put a queue with priorities in front of the GPU pool for backpressure, fairness and batching.
3. Buffer tokens per message so a client that reconnects can resume where it left off.

REQUIREMENTS

- ✓ Send a message in a conversation
- ✓ Stream the model's answer token by token
- ✓ View conversation history

Non-functional

- Time to first token < 1s
- GPU capacity is the scarce resource
- Conversations durable
- 100M DAU; bursty

NUMBERS

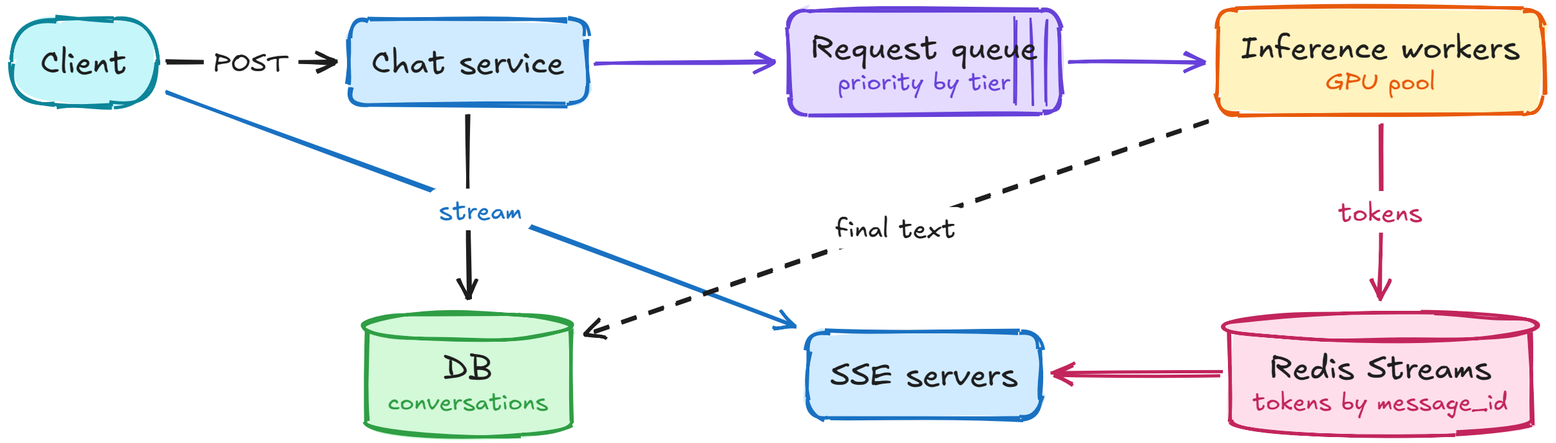
100M DAU × 10 msgs = 1B/day ≈ 12k/s generations.

Each holds a GPU for seconds, so GPUs are the bottleneck and every queueing decision is about them.

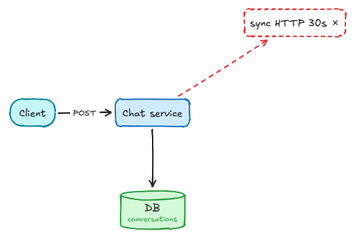
API + DATA

```
POST /conversations/{id}/messages → message_id
GET /messages/{id}/stream (SSE, Last-Event-ID resume)
GET /conversations/{id}
conversations(id, user_id)
messages(conversation_id, id, role, content, status)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: one synchronous request waits ~30s for the full answer. Timeouts, no progress shown, no backpressure.

DEEP DIVES

Why a queue before GPUs?
Scarce and expensive capacity: queueing gives backpressure, priority, fairness, and batching.

Client disconnects?
Generation continues; tokens are buffered in the stream; resume with Last-Event-ID.

Long context?
Load recent messages; summarize or truncate to fit the window; cache hot conversations.

Cost control?
Per-user rate limits and token quotas at the gateway.

HOW IT EVOLVES

1. Stream tokens to the client over SSE as they are generated, so time-to-first-token stays low.
2. Put a queue in front of the GPU pool: absorbs bursts, enables priority for paid tiers, batching, and admission control (429 when full).
3. Workers write tokens into a Redis Stream keyed by message_id; SSE servers read from it.

AVOID

- × Holding a synchronous connection through the load balancer straight to a GPU.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Put a queue with priorities in front of the GPU pool for backpressure, fairness and batching."

DDIA: Ch 11 logs and streams · Ch 1 tail latency · Similar: Ride sharing · Live comments · Collaborative editor · Stock trading

System Design, Step by Step · CC BY-SA 4.0

LOCAL DELIVERY (GOPUFF)

It looks like an ordering system, but most of the load is people checking what is available near them.

Easy | Scaling reads

TL;DR

1. Browsing availability is far more frequent than ordering, so cache it and serve it from replicas.
2. Find nearby warehouses with an in-memory list and a cheap distance filter before calling any travel-time API.
3. Prevent overselling with conditional stock decrements for every item in one transaction.

REQUIREMENTS

- ✓ Check availability of items deliverable to a location (DCs within 1 hour)
- ✓ Order multiple items; never sell stock you don't have

Non-functional

- Availability p99 < 100ms
- Orders strongly consistent (no overselling)
- 10k distribution centres (DCs), ~100k SKUs
- Browse load ≫ order load

NUMBERS

Browse ~10M × 10 / 86,400 ≈ 1.2k/s, ~10k/s peak.

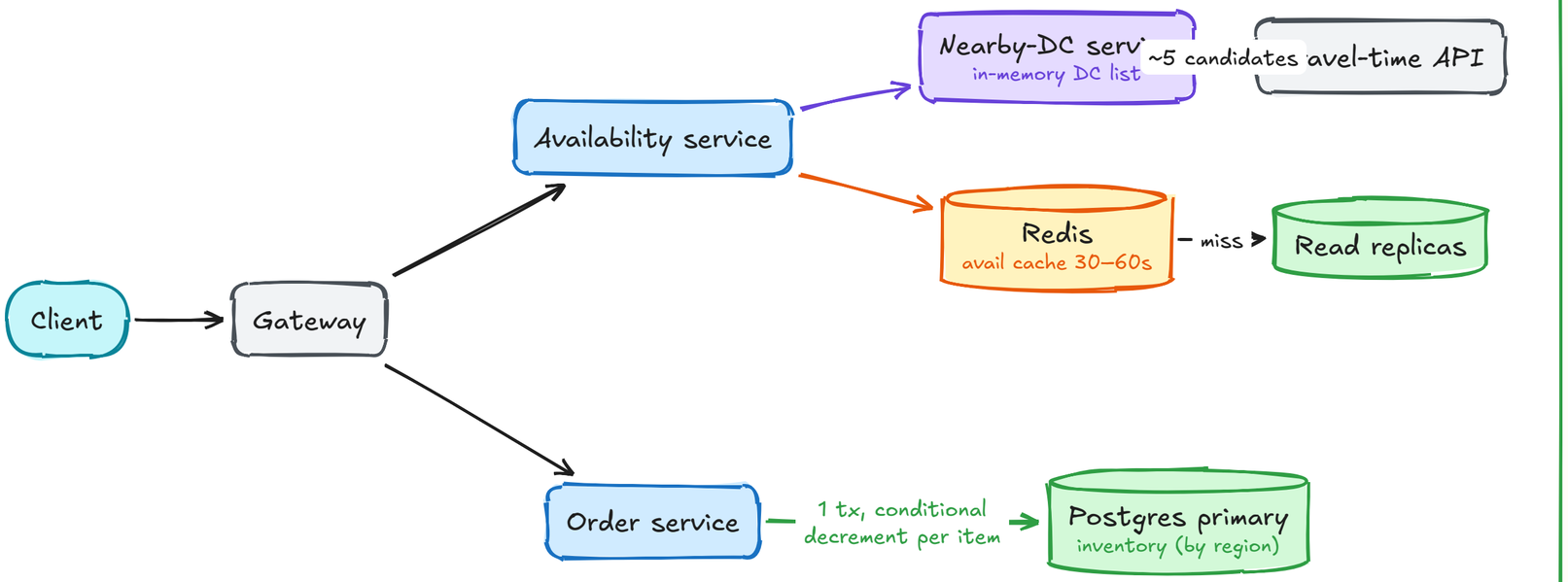
Orders ~1M/day ≈ 12/s.

The DC list (10k rows) fits in memory.

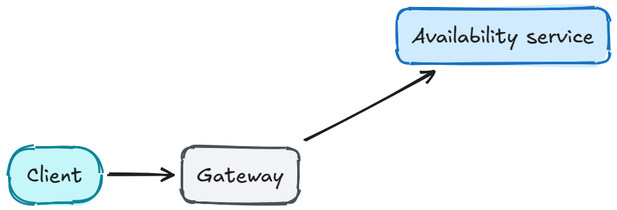
API + DATA

```
GET /availability?lat&lng&item_ids
POST /orders {items:[{id, qty}], lat, lng}
dcs(id, lat, lng, region)
inventory(dc_id, item_id, qty) PK(dc_id, item_id)
orders, order_items
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: call a travel-time API for every DC. 10k calls per page view is far too slow.

DEEP DIVES

How do you prevent overselling?
A conditional decrement per item inside one transaction; row locks serialize concurrent buyers.

Stale availability vs orders?
Different consistency per path: browse is cached and eventually consistent; checkout reads and writes the primary.

Scale orders?
Partition inventory by region; an order only touches nearby DCs, so there are no cross-shard transactions.

Hot item (flash sale)?
Row contention on one inventory row: queue orders per item, or split stock into sub-buckets.

HOW IT EVOLVES

1. Keep the small DC list in memory, prefilter by straight-line distance, then ask travel time for only a handful of candidates.
2. Browse reads come from a short-TTL cache and read replicas. Slightly stale is fine here (AP).
3. Ordering hits the primary: one transaction with UPDATE … SET qty = qty − n WHERE qty ≥ n per item.

AVOID

- × Redis as the inventory source of truth, with no multi-item transaction or durability.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Find nearby warehouses with an in-memory list and a cheap distance filter before calling any travel-time API."

DDIA: Ch 7 conditional updates · Ch 5 replica lag · Similar: URL shortener · Rate limiter · Business discovery · News feed

System Design, Step by Step · CC BY-SA 4.0

METRICS MONITORING (DATADOG)

Millions of data points per second arrive, and alerts must fire within a minute.

Hard | Scaling writes

TL;DR

1. Buffer the write firehose through Kafka into a time-series database built for appends and compression.
2. Downsample old data and move it to cheap storage; dashboards read rollups.
3. Evaluate alert rules on the stream itself instead of polling the database.

REQUIREMENTS

- ✓ Ingest metrics from 500k servers
- ✓ Query dashboards over time ranges
- ✓ Alert rules with notifications

Non-functional

- 5M datapoints/s ingest
- Recent-data queries < 1s
- Alerts within ~1 minute
- Late/out-of-order tolerant, durable

NUMBERS

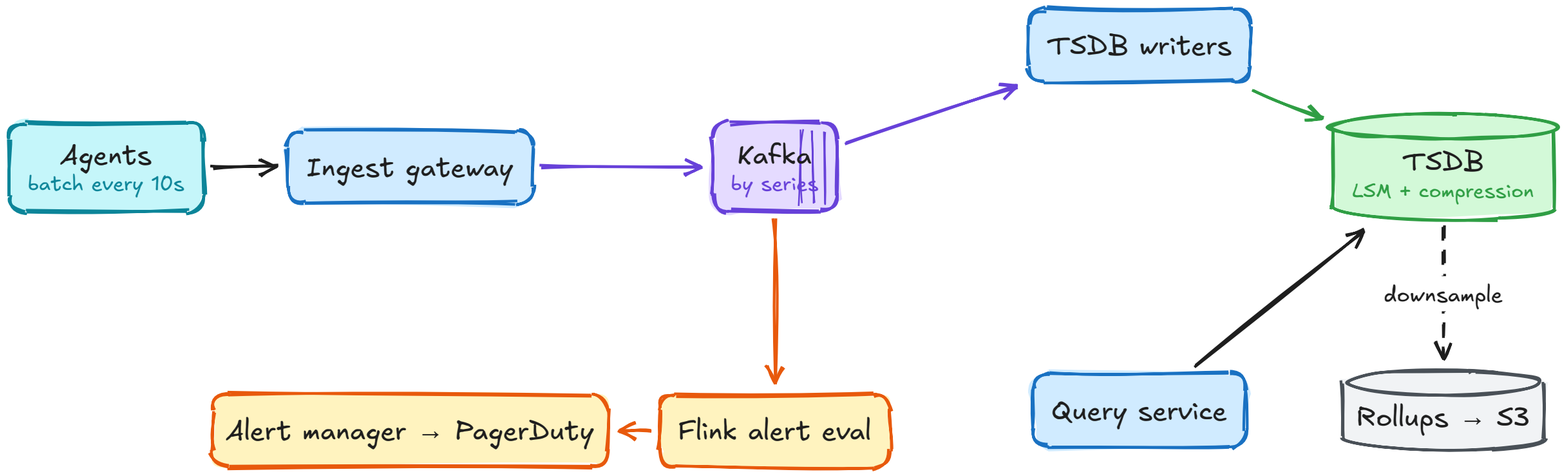
500k hosts × 100 metrics ÷ 10s = 5M points/s.

Gorilla compression ≈ 1.4B/point → ~600GB/day; downsample after 7 days.

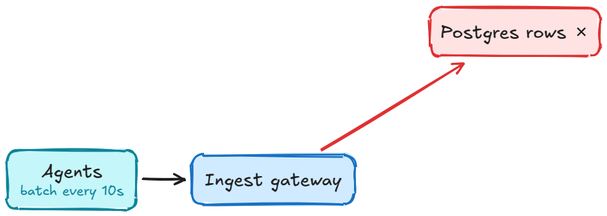
API + DATA

```
POST /ingest (batched points from agents)
GET /query?metric&tags&from&to&step
POST /alerts {rule, threshold, channel}
series(metric, tags) → (ts, value) blocks
rollups 1m / 1h
alert_rules
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: one Postgres row per datapoint with a B-tree index. It falls over around ~50k inserts/sec.

DEEP DIVES

Why a TSDB?
Time-ordered appends suit LSM; compression gives ~10×; queries are range scans per series.

High cardinality?
A user_id tag explodes series count. Cap or reject high-cardinality labels.

Late data?
Event-time windows with watermarks; allowed lateness for corrections.

Alert flapping?
Require N consecutive windows; dedupe and group in the alert manager.

HOW IT EVOLVES

1. Buffer through Kafka into a time-series DB: append-only LSM writes, series-by-time blocks, delta-of-delta + XOR compression.
2. Queries hit recent blocks; older data is downsampled (1s → 1m → 1h) and tiered to S3. Dashboards use rollups.
3. Alerts are evaluated on the stream (Flink windows over Kafka), not by polling the TSDB with thousands of queries.

AVOID

- × Row-per-point in Postgres, and alerting by polling queries.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Downsample old data and move it to cheap storage; dashboards read rollups."

DDIA: Ch 3 LSM, compression · Ch 11 stream alerting · Similar: Top-K videos · Ad click aggregator · Web crawler · Fitness tracking

System Design, Step by Step · CC BY-SA 4.0

NEWS AGGREGATOR (GOOGLE NEWS)

A feed problem where the feed is the same for everyone in a region, which changes everything.

Easy | Scaling reads

TL;DR

1. Writes are tiny (new articles); reads spike hard when big news breaks.
2. Precompute one list per region and category at ingest time instead of querying on every request.
3. Because the feed is not per-user, a CDN can cache it for a minute and absorb almost all traffic.

REQUIREMENTS

- ✓ Feed of recent articles from thousands of publishers, by region and category
- ✓ Click through to the publisher; infinite scroll

Non-functional

- Feed p99 < 200ms
- 100M DAU with breaking-news spikes
- New articles visible within ~30 min
- Availability over consistency

NUMBERS

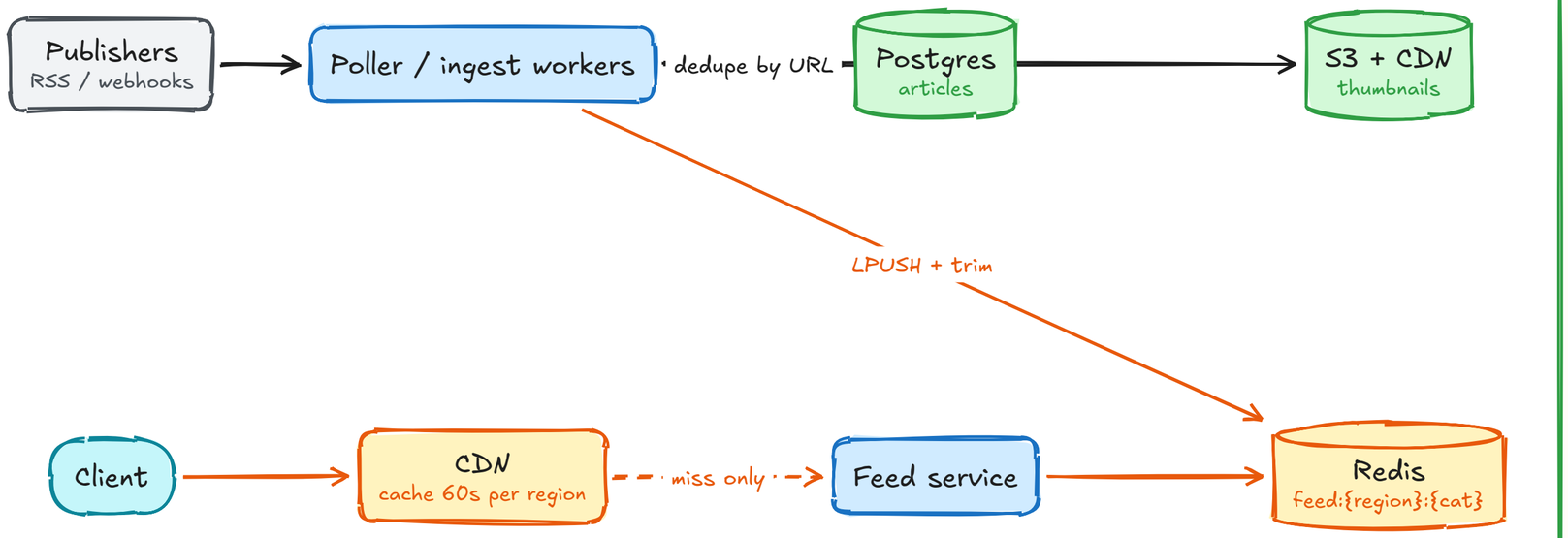
100M × 10 = 1B feed reads/day ≈ 12k/s, 50k+/s spikes.

Writes: ~100k articles/day ≈ 1/s.

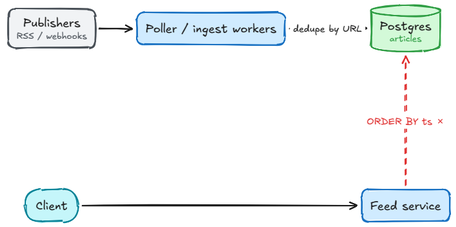
API + DATA

```
GET /feed?region&category&cursor → [{title, summary, thumbnail, url, publisher}]
articles(id, canonical_url UNIQUE, title, summary, thumbnail_url, publisher_id, region, category, published_at)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Pull articles from publishers on a schedule and query Postgres for each feed. Fine at low traffic.

DEEP DIVES

Why can a CDN cache the feed?
Everyone in a region gets the same response. 60s of staleness is fine for news (AP).

Polling cost vs freshness?
Adaptive polling: busy publishers every few minutes, quiet ones hourly. Use webhooks where offered.

Duplicate stories?
Unique canonical URL; near-duplicates clustered with SimHash/MinHash on title and body.

Pagination?
Cursor on (published_at, id). Offsets break when new articles arrive.

HOW IT EVOLVES

1. Precompute one list per (region, category) in Redis at ingest time. Reads become a single list read.
2. The feed is not personalized, so a CDN can cache it for 60s per region. A 50k/s spike becomes a few origin requests.
3. Thumbnails are copied to S3 and served by CDN; publishers that support webhooks push instead of being polled.

AVOID

- × Per-user fan-out like a social feed when the feed is per region and category: build it once per region and let the CDN cache it.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Precompute one list per region and category at ingest time instead of querying on every request."

DDIA: Ch 1 load parameters · Ch 10 ingest pipelines · Similar: URL shortener · Rate limiter · Business discovery · News feed

System Design, Step by Step · CC BY-SA 4.0

NEWS FEED (FACEBOOK)

The canonical fan-out problem: build a personalised feed for billions of users in a few hundred milliseconds.

Medium | Scaling reads

TL;DR

1. Computing the feed on every read means hundreds of queries per page view, so precompute it when people post.
2. Push new post IDs into each follower's feed list in Redis (fan-out on write); a feed read becomes one lookup.
3. Accounts with millions of followers break fan-out on write, so pull their posts at read time instead (hybrid).

REQUIREMENTS

- ✓ Create posts
- ✓ Follow users
- ✓ View feed of followed users' posts, newest first, paginated

Non-functional

- Feed p99 < 500ms
- 2B users; eventual consistency (post appears within ~1 min)
- Handle accounts with millions of followers
- Highly available

NUMBERS

1B DAU × 10 feed loads = ~115k feed reads/s.

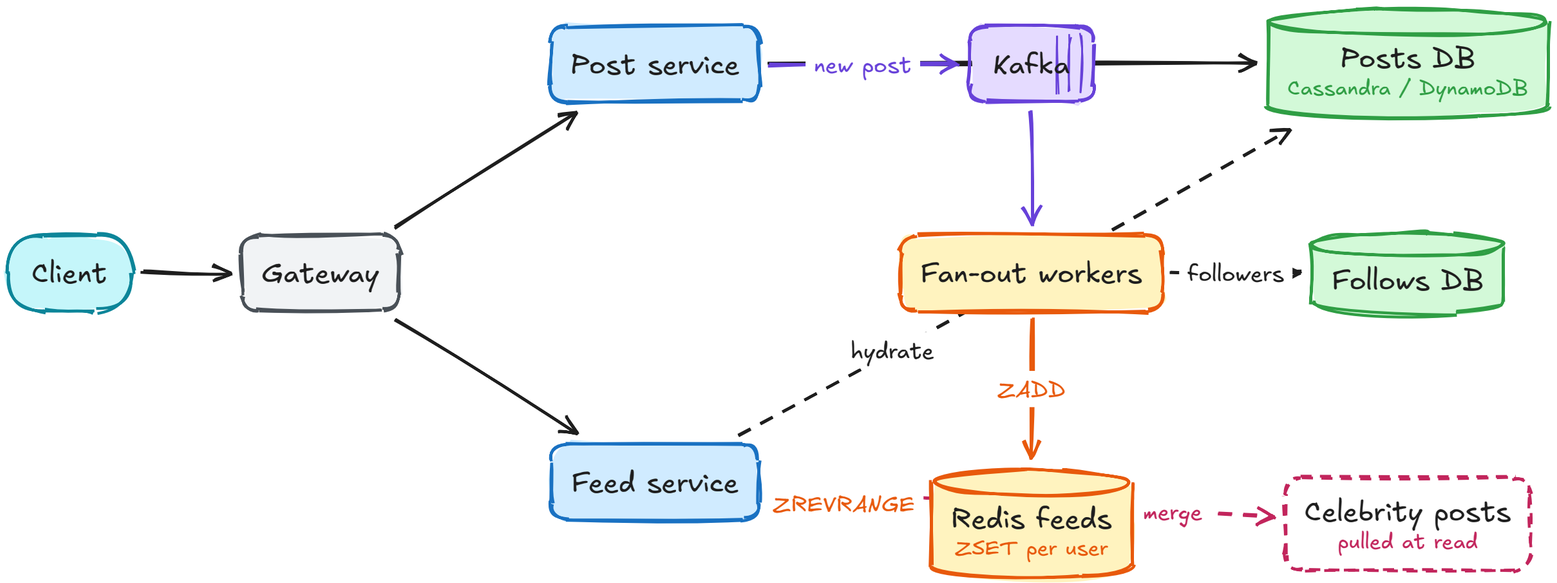
~500M posts/day ≈ 6k/s.

With ~200 followers each, fan-out on write ≈ 1.2M feed inserts/s.

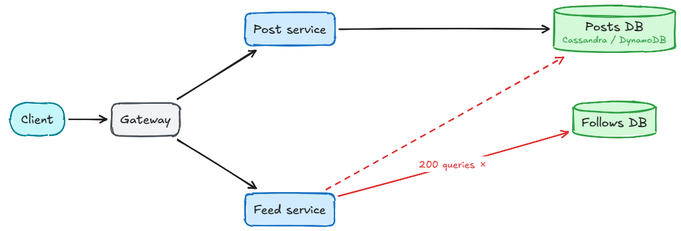
API + DATA

```
POST /posts {content}
PUT /users/{id}/followers
GET /feed?cursor=(ts,post_id)
posts(id, author_id, content, ts) partitioned by author
follows(follower_id, followee_id) + reverse index
feed:{user} = Redis ZSET(ts, post_id), capped at 500
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Fan-out on read: look up 200 followees, then their posts, on every feed load. At 115k feeds/sec that is millions of queries/sec.

DEEP DIVES

Celebrity with 100M followers?
Hybrid. Don't fan out; followers pull celebrity posts at read time and merge. Threshold ~100k–1M followers.

How much memory for feeds?
Only active users, only IDs, capped at 500: 300M × 500 × ~8B ≈ 1.2TB across a Redis cluster. Inactive users get rebuilt on demand.

Pagination?
Cursor = (ts, post_id) of the last item. Offsets shift as new posts arrive.

User follows someone new?
An async job backfills the followee's recent posts into the follower's feed.

HOW IT EVOLVES

1. Fan-out on write: a new post goes to Kafka; workers push its ID into every follower's Redis sorted set. A feed load becomes one ZREVRANGE.
2. Celebrity problem: one post to 100M followers means 100M writes. Hybrid: skip fan-out above ~100k followers and merge their recent posts at read time.
3. Hydrate post IDs into content from the posts store, with a post cache. Cursor pagination on (ts, post_id).

AVOID

- × Fan-out on write for everyone: one celebrity post means 100M writes and workers hours behind.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Push new post IDs into each follower's feed list in Redis (fan-out on write); a feed read becomes one lookup."

DDIA: Ch 1 Twitter load · Ch 11 materialized views · Ch 6 hot keys · Similar: URL shortener · Rate limiter · Business discovery · News aggregator

System Design, Step by Step · CC BY-SA 4.0

ONLINE AUCTION

Many people bid on one row at the same time, and exactly one of them must win each race.

Medium | Real-time updates | Dealing with contention

TL;DR

1. Comparing bids in application code loses updates under concurrency.
2. Accept a bid with one conditional update: set the new maximum only if it beats the current one and the auction is open.
3. Push accepted bids to watchers in real time; for very hot auctions, serialize bids through a partitioned log.

REQUIREMENTS

- ✓ Create an auction (item, starting price, end time)
- ✓ Place bids that must beat the current highest
- ✓ Watch the highest bid update live; winner decided at the end

Non-functional

- Bids strongly consistent (never accept a stale "highest")
- Live updates < 1s
- No lost bids
- Hot auctions spike in the final seconds

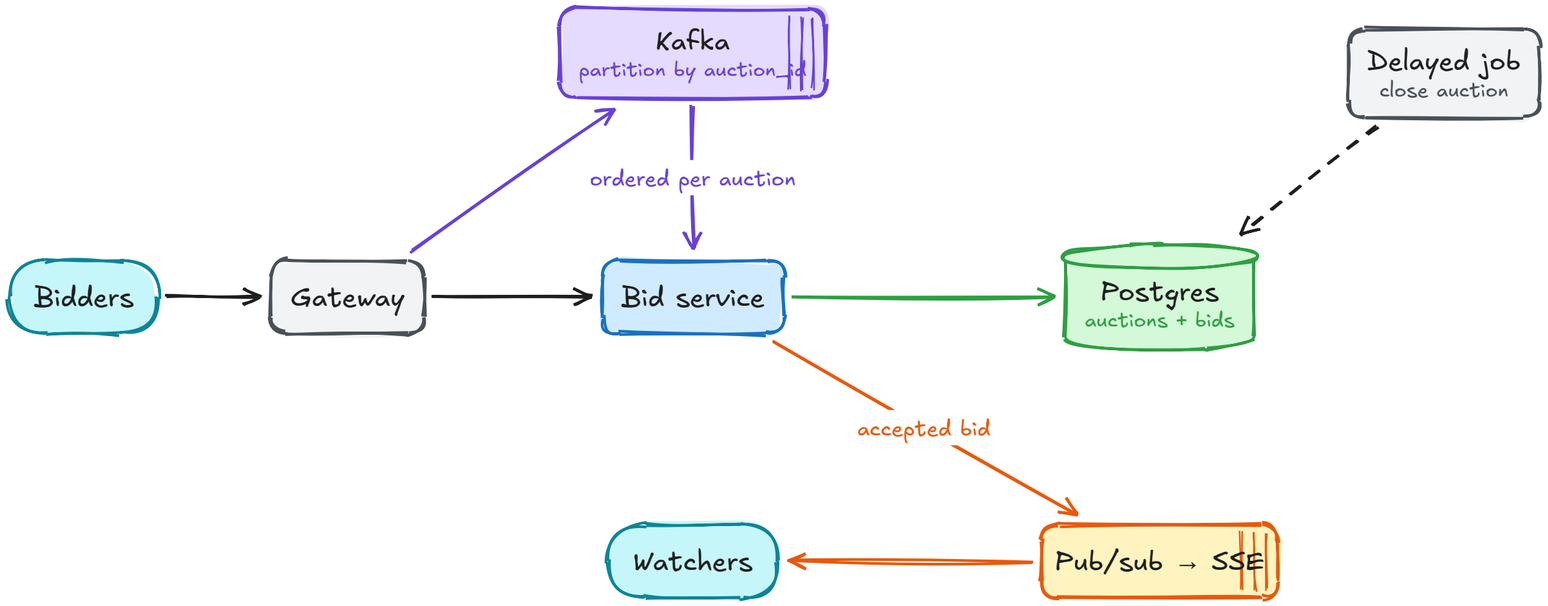
NUMBERS

10M auctions, ~100 bids each over a week ≈ low average QPS, but a hot auction can see ~1k bids/s on one row near the end.

API + DATA

```
POST /auctions
POST /auctions/{id}/bids {amount} + Idempotency-Key
GET /auctions/{id}/stream (SSE)
auctions(id, max_bid, max_bidder, end_time, status)
bids(auction_id, bid_id, user_id, amount, ts) append-only
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: read max_bid, compare in application code, insert. Two concurrent bids both "win", a lost update.

HOW IT EVOLVES

1. Make it atomic in the DB: UPDATE auctions SET max_bid=:amt, max_bidder=:u WHERE id=:a AND max_bid now(). 1 row = accepted, 0 = outbid.
2. Accepted bids are published to pub/sub and pushed over SSE to everyone watching.
3. For very hot auctions, a Kafka partition per auction_id serializes bids before the DB (durable and ordered).

DEEP DIVES

Two bids at the same instant?
Conditional update (or SELECT FOR UPDATE) on the auction row. That is the lost-update fix from DDIA Ch 7.

Clock fairness at the deadline?
Use the DB server time inside the same conditional update, never client time.

Durability?
Acknowledge only after commit (or Kafka acks=all).

Anti-sniping?
Extend end_time by N seconds on late bids, inside the same update.

AVOID

- × Comparing bids in a Redis cache without atomicity.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Accept a bid with one conditional update: set the new maximum only if it beats the current one and the auction is open."

DDIA: Ch 7 lost update · Ch 11 partitioned log · Ch 8 clocks · Similar: Ride sharing · Live comments · Collaborative editor · Stock trading

System Design, Step by Step · CC BY-SA 4.0

ONLINE JUDGE (LEETCODE)

You are running strangers' code on your servers, so isolation comes first.

Medium | Multi-step processes | Long-running tasks

TL;DR

1. Run every submission in a locked-down sandbox: no network, read-only filesystem, strict CPU, memory and time limits.
2. Queue submissions and return immediately; workers autoscale on queue depth and the client polls for results.
3. Keep contest leaderboards in a Redis sorted set.

REQUIREMENTS

- ✓ Browse problems
- ✓ Submit code in several languages; get pass/fail per test
- ✓ Contest leaderboard

Non-functional

- Untrusted code fully isolated
- Result within ~5s
- Contest spikes (100k users submitting)
- Scalable

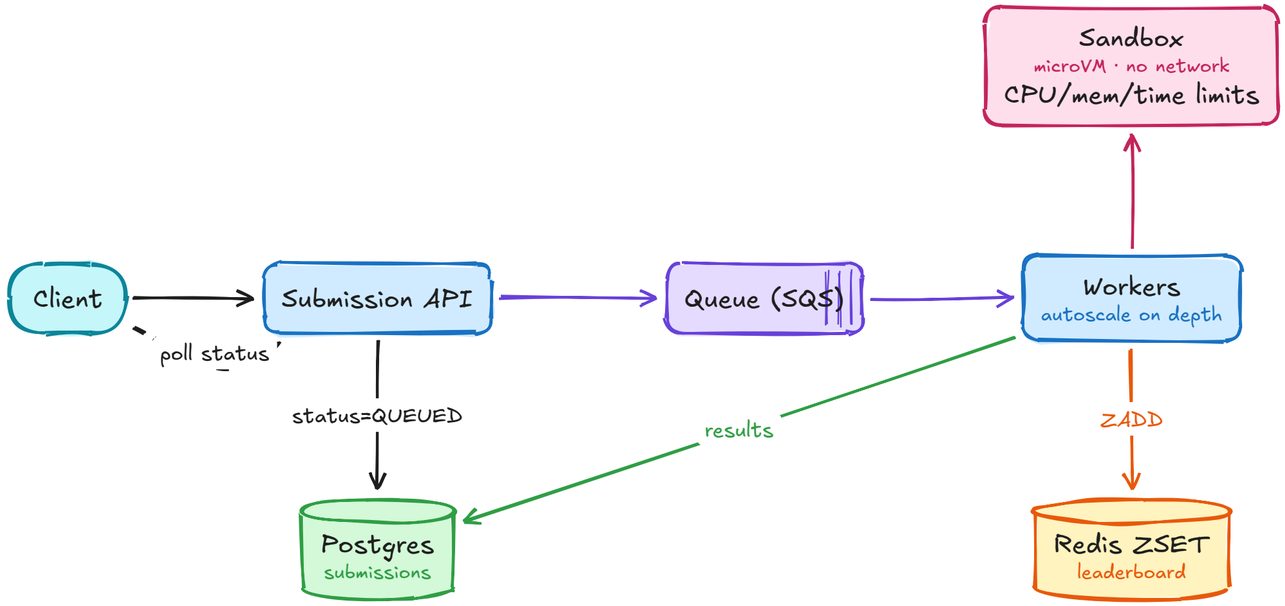
NUMBERS

Contest: 100k users × ~5 submissions in 90 min ≈ ~100 submissions/s peak; each run takes 1–5s of sandbox CPU, so ~500 concurrent sandboxes.

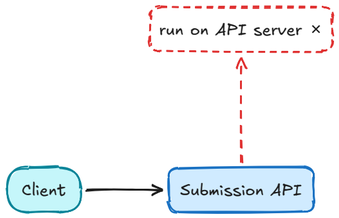
API + DATA

```
POST /problems/{id}/submissions {lang, code} → submission_id
GET /submissions/{id} (poll status)
GET /contests/{id}/leaderboard
problems
test_cases (S3)
submissions(id, user, problem, status, results)
leaderboard:{contest} ZSET
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: run user code on the API servers. It is a security hole, and slow code blocks everything.

DEEP DIVES

Security layers?
MicroVM or gVisor, no network, read-only FS, seccomp filters, resource limits, kill on timeout.

Contest spike?
The queue absorbs it; scale workers on queue depth; pre-warm containers per language.

Polling vs push?
Polling every 1s is simple and fine at this scale; SSE is optional.

Leaderboard ties?
score = points × 1e6 − penalty_seconds.

HOW IT EVOLVES

1. Run every submission in an isolated sandbox (Firecracker/gVisor): read-only filesystem, no network, seccomp, strict CPU/memory/time limits.
2. Decouple with a queue: the API records QUEUED and returns immediately; autoscaled workers run tests and write results; the client polls status every second (the long-running-task pattern).
3. Contest leaderboard in a Redis sorted set; the score encodes points and time penalty.

AVOID

- × Executing user code with network access or on shared API hosts.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Queue submissions and return immediately; workers autoscale on queue depth and the client polls for results."

DDIA: Ch 11 queues · Ch 1 bursty load · Similar: Payment system · Job scheduler

System Design, Step by Step · CC BY-SA 4.0

PAYMENT SYSTEM (STRIPE)

Throughput is moderate. The hard part is staying correct when a call to the bank has an unknown outcome.

Hard | Multi-step processes

TL;DR

1. Record the payment as pending before calling the processor, and pass your ID as the idempotency key.
2. On a timeout, query by that key or retry it; never create a second charge.
3. Publish status changes through an outbox, keep an append-only double-entry ledger, and reconcile daily.

REQUIREMENTS

- ✓ Merchants create payment intents; customers pay by card
- ✓ Merchants receive status via webhooks
- ✓ Refunds

Non-functional

- Never double-charge (exactly-once effect)
- Durable and auditable; strongly consistent ledger
- ~10k TPS
- PCI: raw card data isolated

NUMBERS

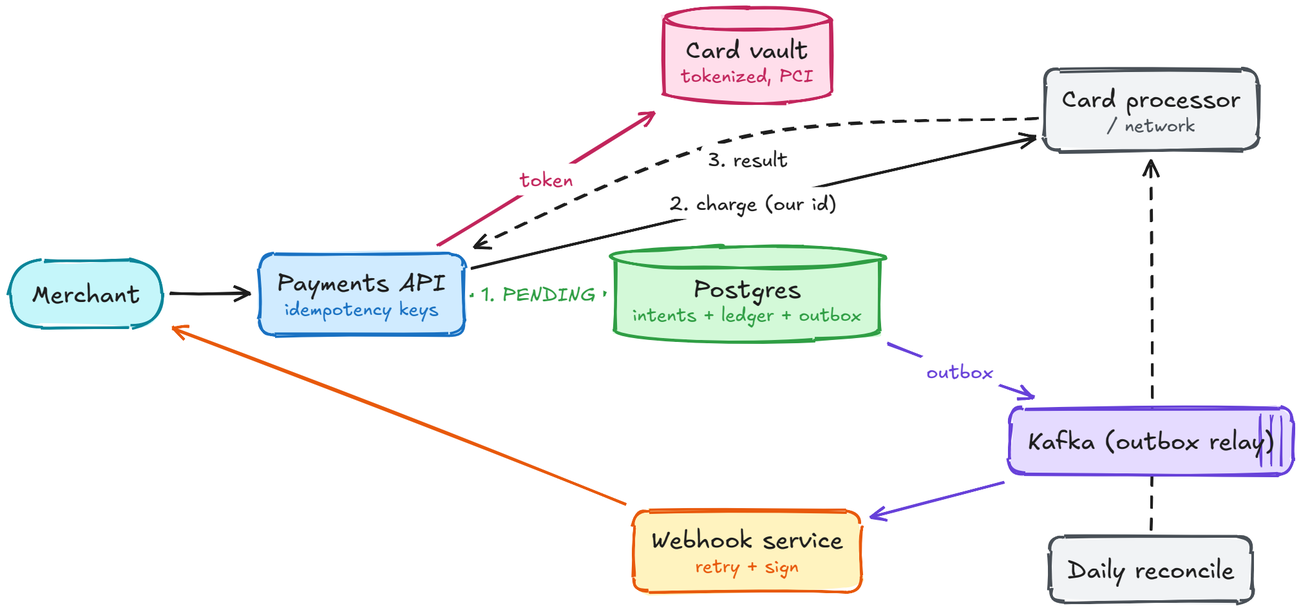
10k TPS is moderate for a well-indexed Postgres cluster.

The hard part is correctness under unknown outcomes, not throughput.

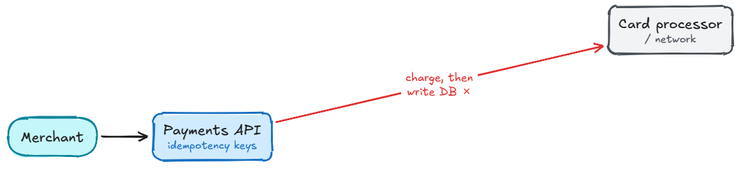
API + DATA

```
POST /payment_intents {amount, currency} + Idempotency-Key
POST /payment_intents/{id}/confirm {payment_method_token}
webhook → merchant: payment.succeeded / failed
payment_intents(id, merchant, amount, status state machine)
ledger_entries(id, txn_id, account, debit, credit) append-only
outbox
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: call the processor, then write the DB. A crash in between charges the card with no record of it.

DEEP DIVES

Processor times out?
Unknown outcome (DDIA Ch 8): keep PENDING, retry with the same idempotency key or query status by reference.

Webhook delivery?
Outbox in the same transaction; exponential backoff; merchants dedupe by event_id.

Why double-entry?
Every movement is a debit + credit summing to zero, so balances are derivable and auditable; never UPDATE amounts.

Distributed transaction with the bank?
Impossible. Design a state machine with compensation (refund) and reconciliation.

HOW IT EVOLVES

1. Write the intent as PENDING first, call the processor with our intent ID as the idempotency key, then record the result.
2. Merchants send an Idempotency-Key on every mutating call (stored with a unique index).
3. Status changes write an outbox row in the same transaction; a relay publishes and webhooks are delivered with retries and signatures.

AVOID

- × Using 2PC or assuming the processor call succeeds or fails cleanly.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"On a timeout, query by that key or retry it; never create a second charge."

DDIA: Ch 8 unknown outcomes · Ch 12 end-to-end idempotency, integrity · Ch 11 outbox · Similar: Online judge · Job scheduler

System Design, Step by Step · CC BY-SA 4.0

PHOTO SHARING (INSTAGRAM)

A news feed plus a media pipeline: two well-known problems combined.

Medium | Handling large blobs | Scaling reads

TL;DR

1. Upload media straight to object storage; process thumbnails and sizes asynchronously; serve them from a CDN.
2. Build feeds with fan-out on write into Redis, and pull celebrity posts at read time.
3. Show a post only once its media has finished processing.

REQUIREMENTS

- ✓ Post photos and videos with captions
- ✓ Follow users
- ✓ View a feed of followed users' posts

Non-functional

- Feed p99 < 500ms
- Media upload and delivery fast globally
- 500M DAU
- Eventual consistency for the feed

NUMBERS

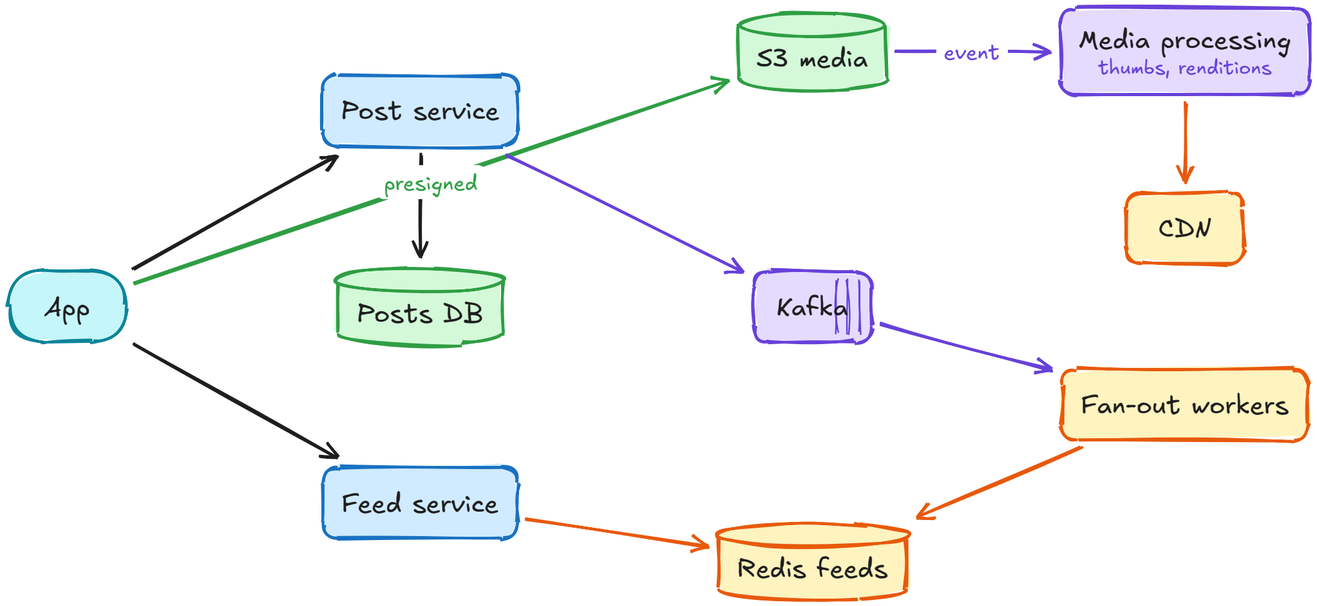
500M DAU × 20 feed loads ≈ ~115k feed reads/s.

100M posts/day; media ~2MB avg → 200TB/day to S3, served by CDN.

API + DATA

```
POST /posts {caption} → {post_id, presigned upload URL}
GET /feed?cursor
POST /users/{id}/follow
posts(id, author, caption, media_keys, status)
follows
feed:{user} ZSET
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Naive: media through the API and a fan-out-on-read feed. Both break at this scale.

Media: the client uploads directly to S3 with a presigned URL; an S3 event triggers thumbnails and renditions; the CDN serves them.

DEEP DIVES

Post visible before media is processed?
Keep status PROCESSING; show it once renditions are ready (or show to the author only).

Celebrities?
Hybrid fan-out as in News Feed.

Like counts?
Redis INCR or sharded counters, flushed asynchronously to the DB.

Global latency?
CDN for media; regional read replicas or caches for the feed.

HOW IT EVOLVES

1. Media: the client uploads directly to S3 with a presigned URL; an S3 event triggers thumbnails and renditions; the CDN serves them.
2. Feed: same as News Feed. New posts go to Kafka, workers push post IDs into followers' Redis feeds, and celebrities are pulled at read time (hybrid).

AVOID

- × Serving images from app servers or storing blobs in the database.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Build feeds with fan-out on write into Redis, and pull celebrity posts at read time."

DDIA: Ch 1 fan-out · Ch 11 async pipelines · Similar: Video platform · File storage and sync

System Design, Step by Step · CC BY-SA 4.0

POST SEARCH (FACEBOOK)

Search over billions of posts, where like counts change constantly.

Medium | Scaling writes | Scaling reads

TL;DR

1. Build an inverted index (term → post IDs) from the stream of new posts, sharded by term.
2. Keep two orderings per term, recency and likes, and update like-based ranking only at thresholds or in batches.
3. Keep hot postings in memory, move old ones to cheaper storage, and cache popular queries.

REQUIREMENTS

- ✓ Search posts by keyword
- ✓ Sort results by recency or by likes
- ✓ (Inputs: create posts, like posts)

Non-functional

- Search < 500ms
- New posts searchable within ~1 minute
- Huge write volume (posts and likes)
- Read volume ~10k searches/s

NUMBERS

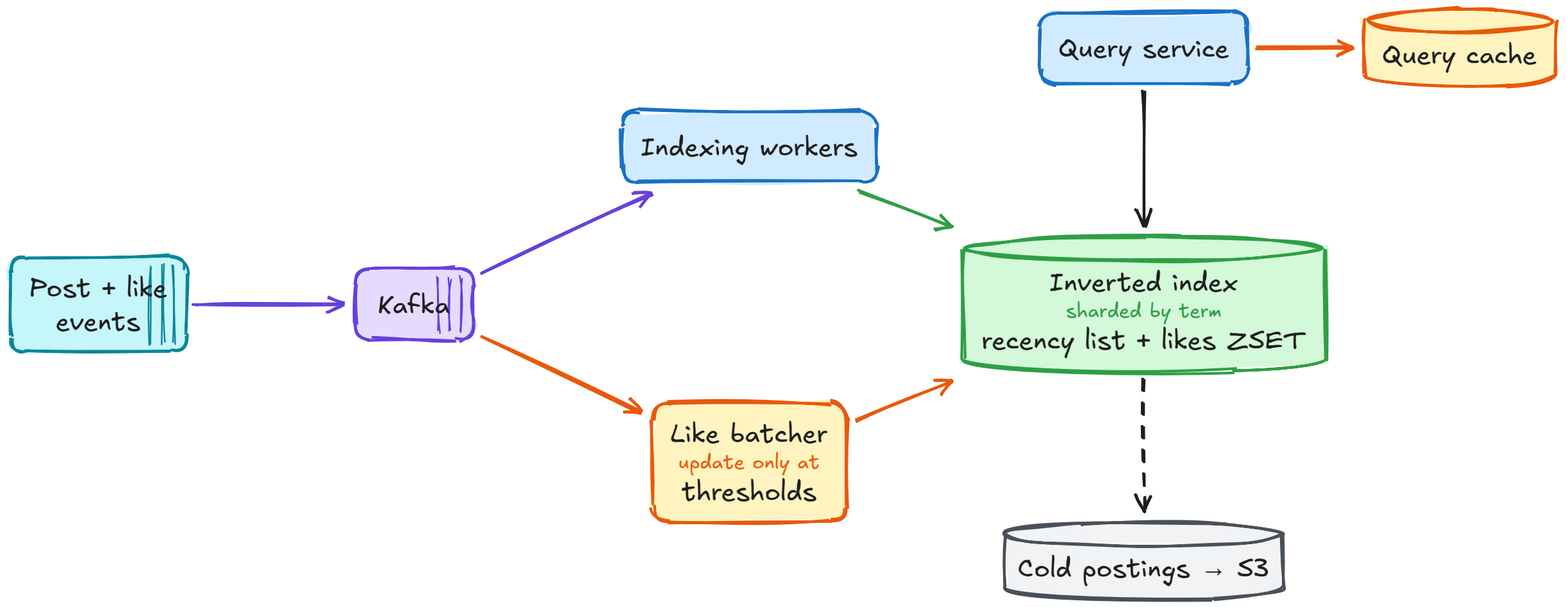
~1B posts/day ≈ 12k/s and ~10× that in likes.

Indexing every like individually is far too much write load, so batch it.

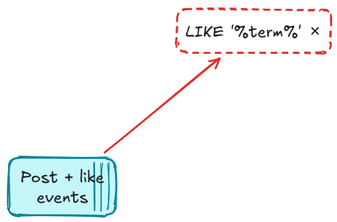
API + DATA

```
GET /search?q&sort=recency|likes&cursor
inverted index: term → post IDs (recency list) and
term → ZSET(post_id, like_bucket)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: LIKE '%term%' on the posts table, a full scan at any real scale.

HOW IT EVOLVES

1. Build an inverted index (term → post IDs) from the event stream, sharded by term. New posts are searchable within seconds.
2. Two orderings: a recency list and a likes-sorted set.
3. Keep hot postings in memory and move older ones to blob storage; cache popular queries; intersect posting lists for multi-word queries.

DEEP DIVES

Why not just Elasticsearch?
In a real design it is a fine answer; know its sharding (document-partitioned, scatter-gather) and refresh interval.

Likes change constantly?
Threshold or batched updates to the ranking score; exact counts live elsewhere.

Multi-keyword?
Intersect posting lists, smallest first; consider bigram indexes.

Hot term?
Term-partitioned index means one shard per term, so replicate hot terms.

AVOID

- × A Postgres LIKE scan, or re-indexing on every like.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Keep two orderings per term, recency and likes, and update like-based ranking only at thresholds or in batches."

DDIA: Ch 3 inverted index · Ch 6 term-partitioned index · Ch 11 stream indexing · Similar: Top-K videos · Ad click aggregator · Web crawler · Metrics monitoring

System Design, Step by Step · CC BY-SA 4.0

PRICE TRACKER (CAMELCAMELCAMEL)

Five hundred million products can't all be crawled often, so decide what is worth checking.

Medium | Scaling writes | Scaling reads

TL;DR

1. Prioritise crawling by how many people track a product and how often its price changes.
2. Add crowd-sourced prices from a browser extension, validated before they trigger alerts.
3. Make alerts event-driven: when a price drops, look up subscriptions for that product only.

REQUIREMENTS

- ✓ Track price history for products
- ✓ Users subscribe to alerts at a target price
- ✓ View price charts

Non-functional

- 500M products
- Alert within ~1 hour of a drop
- Respect site rate limits (crawl budget)
- Charts load fast

NUMBERS

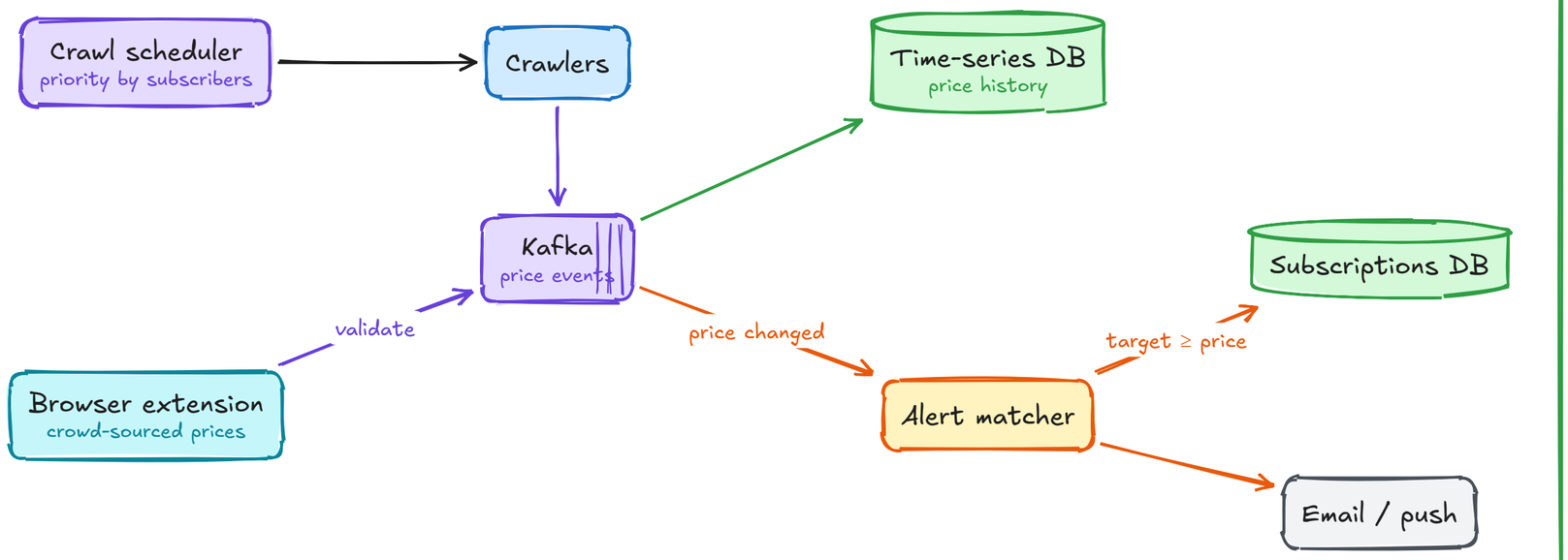
500M products crawled daily = ~6k pages/s, which runs into rate limits.

Subscriptions are concentrated on a small fraction of products, so prioritize those.

API + DATA

```
POST /subscriptions {product_id, target_price}
GET /products/{id}/history?range
price_history(product_id, ts, price) time-series
subscriptions(product_id, user_id, target_price)
indexed (product_id, target_price)
```

FINAL DESIGN



V1, AND WHY IT BREAKS

A scheduler decides what to crawl: products with many subscribers or volatile prices are checked more often; the long tail rarely.

A browser extension reports prices users actually see, which adds coverage for free. Validate it (N agreeing reports or a crawler re-check) before trusting it.

DEEP DIVES

Crawl budget?
Prioritize by subscribers and volatility; adaptive frequency; respect robots and rate limits.

Bad extension data?
Require agreement or confirm with a crawl before alerting.

Matching efficiently?
Index on (product_id, target_price); react to events, never scan all subscriptions.

Charts?
Daily aggregates, cached.

HOW IT EVOLVES

1. A browser extension reports prices users actually see, which adds coverage for free.
2. Every price change becomes an event; history goes to a time-series table (downsampled for charts).
3. Alerts are event-driven: on a drop, query subscriptions WHERE product_id=? AND target_price ≥ new_price and notify, deduped per user and product.

AVOID

- × Re-scanning all subscriptions on a schedule instead of reacting to price changes.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Add crowd-sourced prices from a browser extension, validated before they trigger alerts."

DDIA: Ch 11 event-driven · Ch 3 time-series storage · Similar: Top-K videos · Ad click aggregator · Web crawler · Metrics monitoring

System Design, Step by Step · CC BY-SA 4.0

RATE LIMITER

Every public API needs one, and the obvious implementation has a race condition.

Medium | Scaling reads | Dealing with contention

TL;DR

1. Counting per server gives each client N times its limit; the counter must be shared, usually in Redis.
2. Read-then-write on a shared counter races. Do the refill, check and decrement atomically in one Lua script.
3. Decide what happens when Redis is down: allow traffic (fail-open) or block it (fail-closed), per endpoint.

REQUIREMENTS

- ✓ Limit requests per client (user, IP, API key) by configurable rules
- ✓ Return 429 with Retry-After and rate-limit headers

Non-functional

- Adds < 10ms latency
- Accurate across many API servers
- Highly available (choose fail-open vs fail-closed)
- 1M requests/sec total

NUMBERS

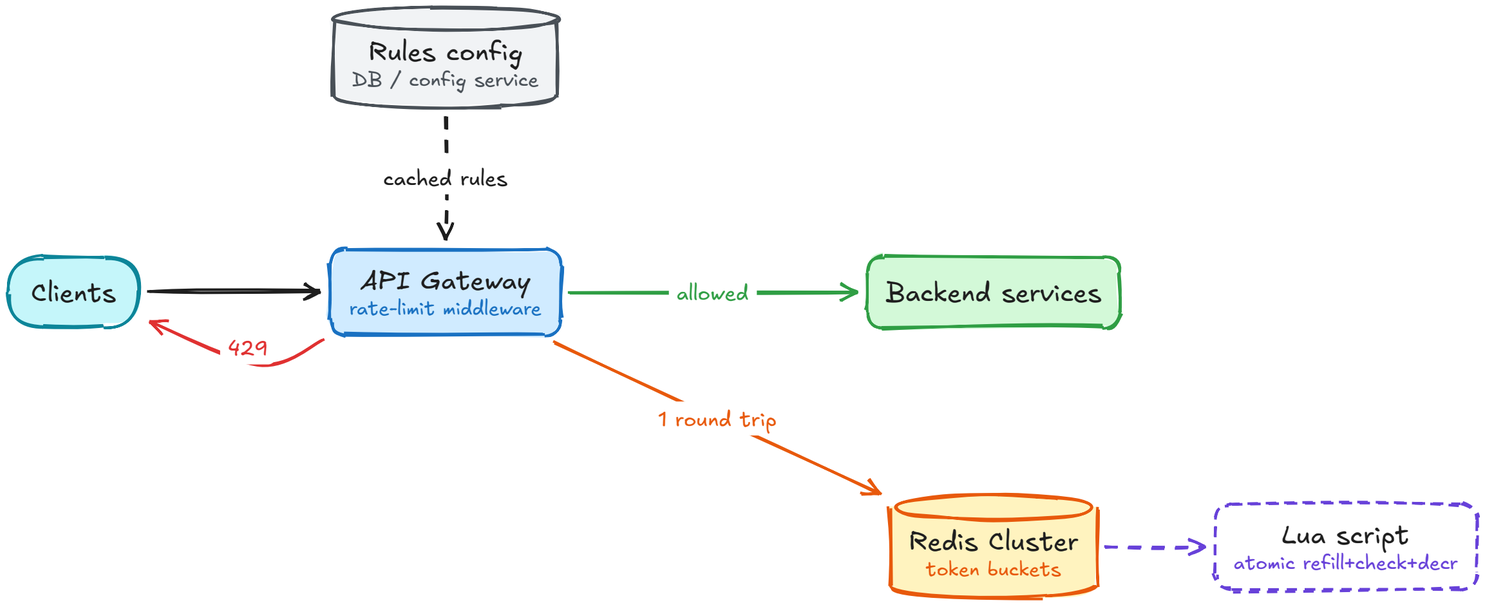
1M req/s = 1M Redis ops/s; one Redis node does ~100k ops/s, so a ~10–20 shard Redis Cluster keyed by client.

10M active keys × 100B ≈ 1GB of memory.

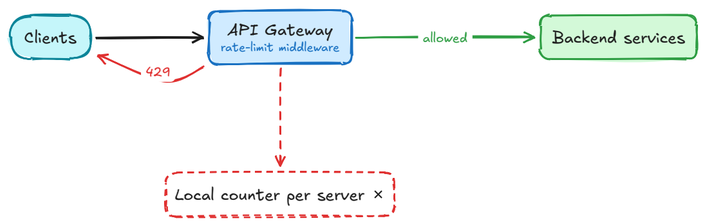
API + DATA

```
isAllowed(client_key, rule) → {allowed, remaining, reset_at}
Response headers: X-RateLimit-Limit / Remaining / Reset
rules(endpoint, limit, window)
bucket:{client}:{rule} → {tokens, last_refill}
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: each API server counts in local memory. With 50 servers, a client gets 50× the limit.

HOW IT EVOLVES

1. Centralize counters in Redis. But GET then SET from two servers races: both see 99/100 and both allow.
2. A Lua script runs refill + check + decrement atomically inside single-threaded Redis, in one round trip, with no race.
3. Rules come from a config store cached in each gateway. Decide failure mode: fail-open for normal APIs, fail-closed for login and payments.

DEEP DIVES

Which algorithm?
Token bucket (capacity = burst, refill = sustained rate) is the industry default.

Race between servers?
Read-modify-write is a lost update (DDIA Ch 7). Atomic Lua or INCR + EXPIRE inside Redis fixes it.

Redis is down?
Fall back to local limits (limit ÷ N servers). Fail-open vs fail-closed is a product decision per endpoint.

Very hot client?
One key lands on one shard. Give each server a local allowance and sync periodically, trading accuracy for load.

AVOID

- × Counters in Postgres: 1M writes/sec of throwaway data.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Read-then-write on a shared counter races. Do the refill, check and decrement atomically in one Lua script."

DDIA: Ch 7 lost update · Ch 6 shard by key · Similar: URL shortener · Business discovery · News feed · News aggregator

System Design, Step by Step · CC BY-SA 4.0

RIDE SHARING (UBER)

Four problems in one: a flood of location updates, matching, locking drivers, and a multi-step trip lifecycle.

Hard | Real-time updates

TL;DR

1. Driver locations are ephemeral and frequent, so keep them in Redis's geospatial index, not a database.
2. Offer a ride to one driver at a time and lock that driver with a short TTL so they can't get two offers.
3. Run ride requests through a durable, per-region queue so a crashed matcher loses nothing.

REQUIREMENTS

- ✓ Rider requests a ride (pickup, destination) and sees a fare estimate
- ✓ Match to a nearby available driver; driver accepts or declines
- ✓ Live location of driver during the trip

Non-functional

- Match within ~1 minute
- A driver is never assigned two rides (consistency)
- High availability for location updates
- Handle surge peaks per city

NUMBERS

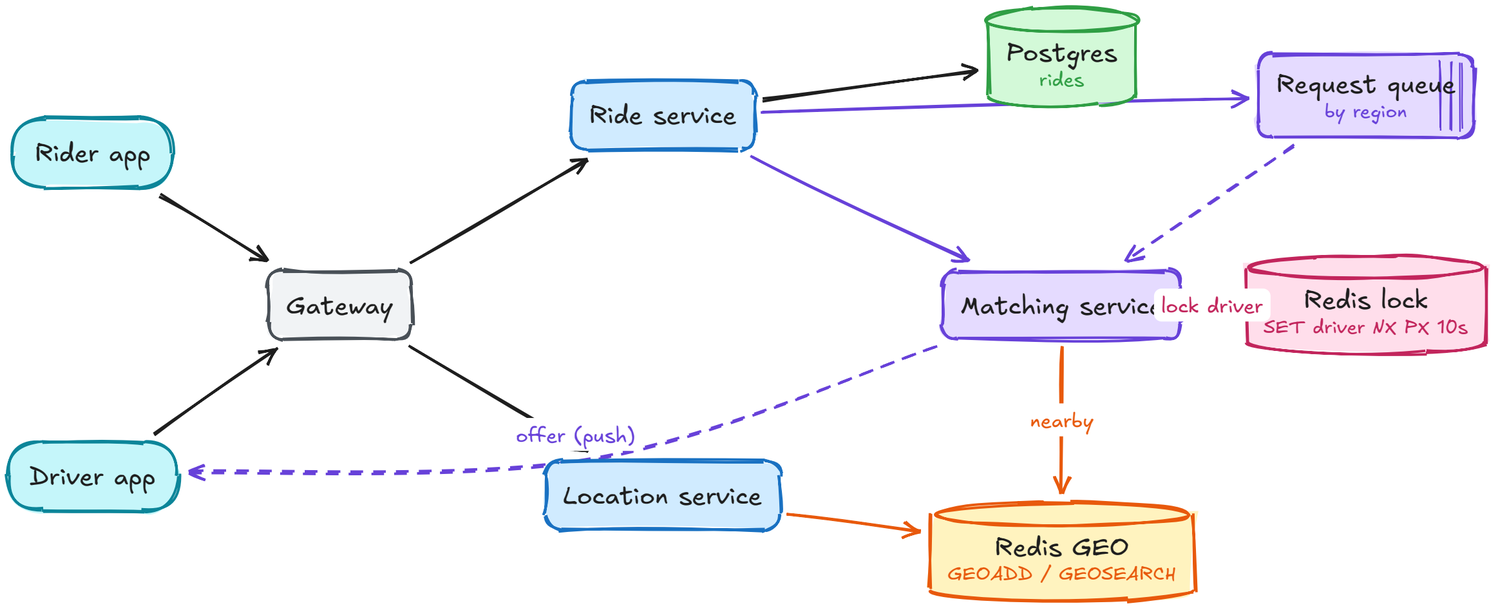
1M active drivers × 1 update/4s = 250k location writes/s, ephemeral (only the latest matters).

Ride requests ~100/s avg.

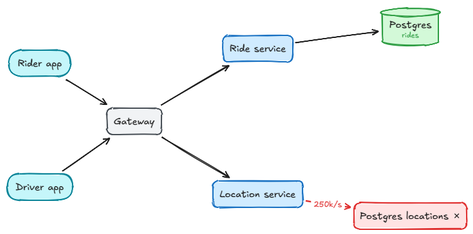
API + DATA

```
POST /rides/estimate
POST /rides {estimate_id}
POST /drivers/location {lat, lng}
PATCH /rides/{id} {accept|decline}
rides(id, rider, driver, status, pickup, dest, fare)
driver_location in Redis GEO (TTL)
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: drivers write locations to Postgres. 250k writes/sec of throwaway data breaks it.

DEEP DIVES

Driver double-assigned?
Lock per driver with TTL = accept window, plus conditional assignment in Postgres. Lock expiry = automatic decline.

Reduce location load?
Adaptive update frequency (less when stationary), client batching.

Why not query Postgres + PostGIS?
Fine for reads, but 250k updates/sec of ephemeral data belongs in memory.

Matcher crash mid-match?
Durable workflow/queue; the request is re-processed and idempotent by ride_id.

HOW IT EVOLVES

1. Locations go to Redis GEO: in memory, latest point only, with TTL. GEOSEARCH finds drivers within radius.
2. Matching offers the ride to one driver at a time and takes a lock on that driver (SET NX with a 10s TTL), so they can't be offered two rides.
3. Requests go through a durable queue partitioned by region, so a crashed matcher doesn't lose rides and surges scale per city.

AVOID

- × Offering a ride to many drivers at once without locks, so two drivers accept the same ride.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Offer a ride to one driver at a time and lock that driver with a short TTL so they can't get two offers."

DDIA: Ch 8 leases · Ch 3 geo index · Ch 6 partition by region · Similar: Live comments · Collaborative editor · Stock trading · Online auction

System Design, Step by Step · CC BY-SA 4.0

STOCK TRADING (ROBINHOOD)

Two separate problems: pushing live prices to millions, and never losing or duplicating an order.

Hard | Real-time updates

TL;DR

1. Prices: one consumer of the exchange feed publishes per symbol; push servers subscribe and stream to clients.
2. Orders: persist the order as pending with your own ID before sending it to the exchange.
3. Treat exchange calls as having unknown outcomes; reconcile by your order ID instead of resubmitting.

REQUIREMENTS

- ✓ See live stock prices for watched symbols
- ✓ Place market and limit orders; cancel orders
- ✓ See order status and fills

Non-functional

- Price updates < 200ms
- Orders never lost or duplicated
- Strong consistency on order state
- 20M users; market-open spikes

NUMBERS

Exchange feed ~ thousands of updates/s for ~10k symbols.

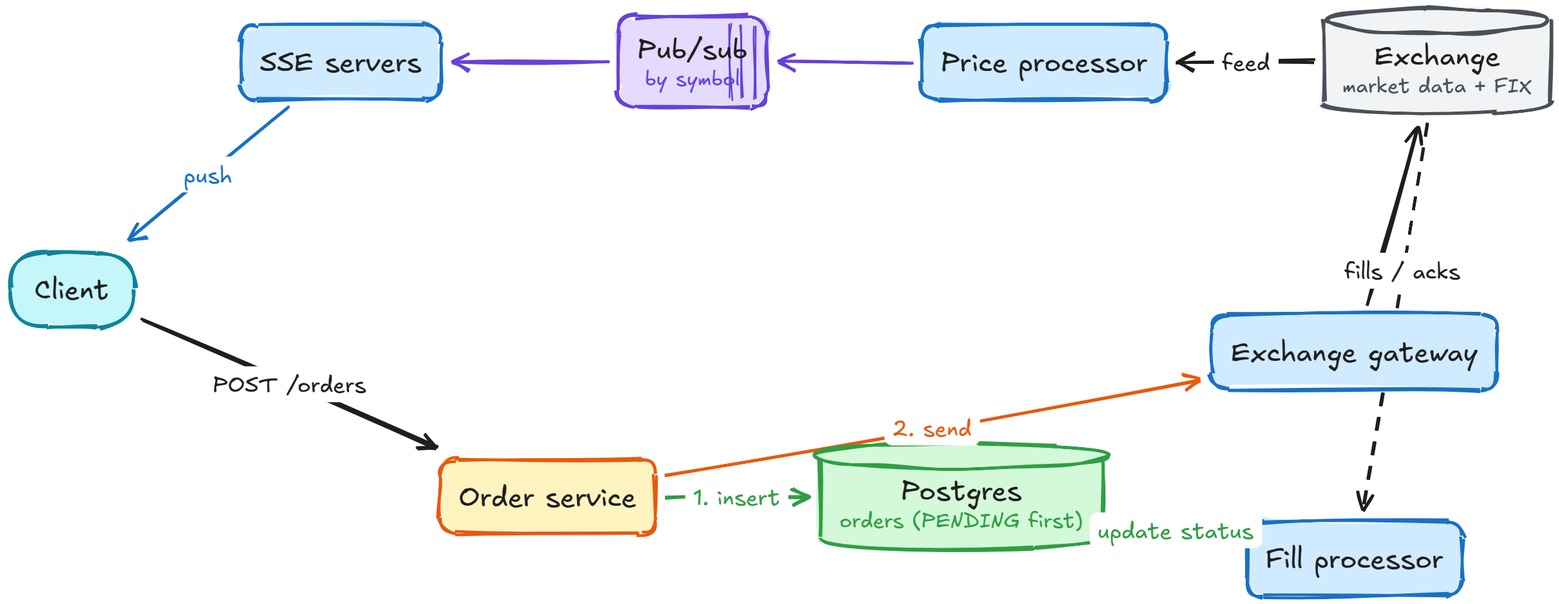
20M users polling = millions of req/s, so push instead.

Orders ~ thousands/s at peak.

API + DATA

```
GET /prices/stream?symbols (SSE)
POST /orders {symbol, side, qty, type, limit?} +
Idempotency-Key
DELETE /orders/{id}
orders(id, user_id, symbol, status
PENDING|SUBMITTED|FILLED|CANCELLED,
exchange_order_id) partitioned by user
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Order and price paths are separate problems: one is real-time fan-out, the other is correctness.

Prices: one consumer reads the exchange feed and publishes per symbol; SSE servers subscribe to the symbols their clients watch. Throttle to ~1 update/s per symbol for the app.

DEEP DIVES

Write DB before or after the exchange call?
Before. Otherwise a crash after sending leaves a live order you don't know about.

Duplicate submits?
Client Idempotency-Key with a unique index; the exchange also dedupes on clientOrderId.

Cancel races with a fill?
Cancel is a request; status changes only on the exchange ack. Handle partial fills.

Market-open spike?
Autoscale SSE servers; per-user rate limits on orders.

HOW IT EVOLVES

1. Prices: one consumer reads the exchange feed and publishes per symbol; SSE servers subscribe to the symbols their clients watch.
2. Orders: persist PENDING first with our own order ID, then send to the exchange using that ID as clientOrderId.
3. Fills and acks come back asynchronously and update order status; a reconciliation job compares against exchange reports daily.

AVOID

- × Treating the exchange call as a local function call, so timeouts create ghost or duplicate orders.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Orders: persist the order as pending with your own ID before sending it to the exchange."

DDIA: Ch 8 unknown outcomes · Ch 12 idempotency · Ch 11 pub/sub · Similar: Ride sharing · Live comments · Collaborative editor · Online auction

System Design, Step by Step · CC BY-SA 4.0

TICKETMASTER

Millions of people arrive for one on-sale, and no seat may ever be sold twice.

Hard | Dealing with contention | Scaling reads

TL;DR

1. Browsing is huge, read-only traffic that can be slightly stale: cache it hard.
2. Hold a seat with one atomic conditional update so two buyers can never both win.
3. For huge on-sales, a virtual waiting room admits users at the rate the system can handle.

REQUIREMENTS

- ✓ View events and seat maps
- ✓ Search events
- ✓ Book tickets: pick seats, hold, pay, confirm

Non-functional

- Never double-book a seat (strong consistency)
- Browse and search highly available, low latency
- Extreme spikes: ~10M users for one on-sale
- Seat hold ~10 minutes during checkout

NUMBERS

Hot event: 10M users, 100k seats.

Reads are massive; successful bookings are capped by seat count (100k).

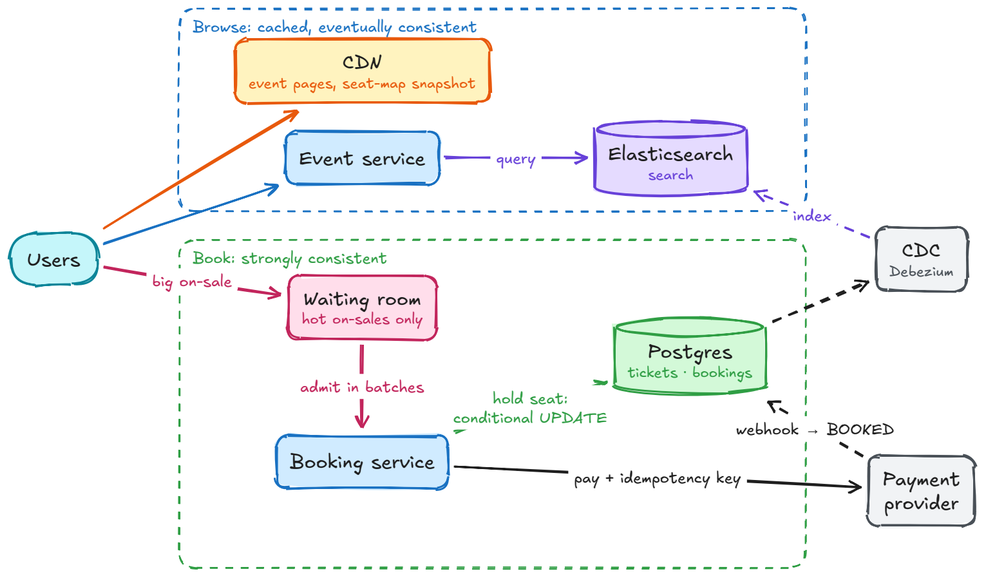
Protect the booking path from the read storm.

API + DATA

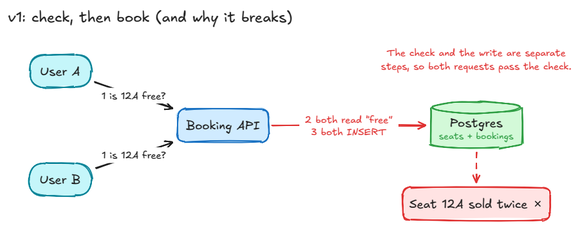
```
GET /events/{id} · GET /search?q&date&location
POST /bookings/hold {event_id, seat_ids}
POST /bookings/{id}/confirm (after payment)
events
tickets(id, event_id, seat, status
AVAILABLE|HELD|BOOKED, hold_expires_at, user_id)
bookings
```

FINAL DESIGN

Final design



V1, AND WHY IT BREAKS



Naive: check the seat is free, then insert a booking. Two users read "free" at the same time and both book: write skew.

DEEP DIVES

Two users click the same seat?
A conditional update on the ticket row (the seat row materializes the conflict, DDIA Ch 7). Exactly one gets 1 row updated.

Release abandoned holds?
The expiry is part of the condition, or a Redis TTL. No background job is needed for correctness.

Taylor Swift spike?
Waiting room queue (Redis sorted set by arrival), push "your turn" via SSE, admit at the rate booking can handle.

Stale seat map?
Cache for a few seconds; the booking path is the source of truth.

HOW IT EVOLVES

1. Hold atomically: UPDATE tickets SET status=HELD, user=?, hold_expires=now()+10m WHERE id=? AND (status=AVAILABLE OR hold_expires < now()).
2. The user pays with Stripe; the idempotent webhook flips HELD → BOOKED. An expired hold frees itself, with no cron job.
3. For hot on-sales, a virtual waiting room admits users in batches; the CDN serves pages and Elasticsearch handles search, keeping the booking DB small and safe.

AVOID

- × Holding a DB transaction or row lock open while the user pays for 10 minutes.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Hold a seat with one atomic conditional update so two buyers can never both win."

DDIA: Ch 7 write skew · Ch 8 leases / TTL

System Design, Step by Step · CC BY-SA 4.0

TOP-K VIDEOS (YOUTUBE)

The design falls straight out of one number: hundreds of thousands of views per second.

Hard | Scaling writes

TL;DR

1. A counter row per video in a database can't take ~800k updates per second.
2. Stream views through Kafka partitioned by video ID; each consumer counts its videos in memory and keeps a local top-K.
3. Merge the per-partition top-K lists; the result is exact because each video lives in exactly one partition.

REQUIREMENTS

✓ Top K most-viewed videos for the last hour, day, month, and all time

Non-functional

- Query latency < 100ms
- Freshness ~1 minute
- Accurate counts (billing-grade not required, but no big drift)
- Massive view volume

NUMBERS

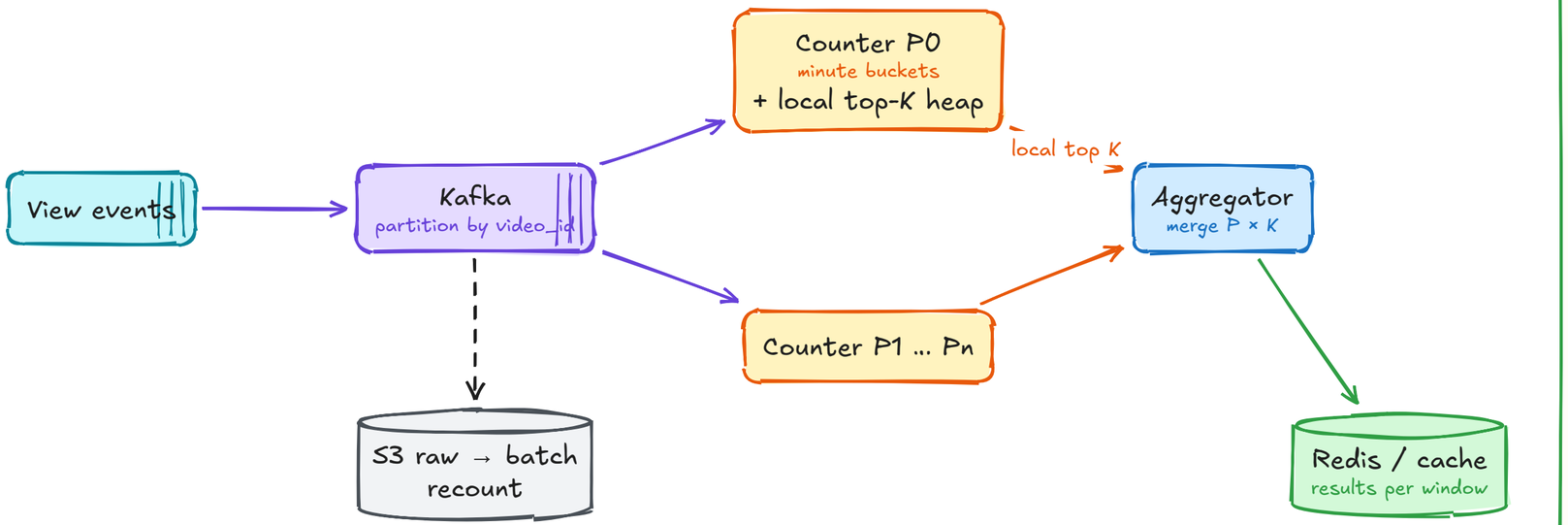
70B views/day ≈ ~800k views/s.

4B videos × 8B counters ≈ 32GB of state, spread over partitions.

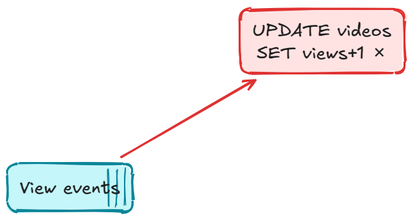
API + DATA

```
GET /topk?window=hour|day|month|all&k=100
view events (video_id, ts) → per-minute counts per video → per-window top-K heap
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: increment a counter row per view and ORDER BY views. That is 800k UPDATEs/sec on hot rows, then a full sort.

DEEP DIVES

Why is the merge exact?
A video's full count lives in one partition, so its global rank can only come from its local rank. The global top K ⊆ union of local top Ks.

Sliding windows?
A ring buffer of minute counts per video; for months use hourly buckets. Rolling sums avoid rescanning.

Very hot video?
Pre-aggregate on producers (send counts per second, not individual views).

Approximate is OK?
Count-Min Sketch + heap uses fixed memory and is good for trending; exact needs partitioned counting.

HOW IT EVOLVES

1. Stream views into Kafka partitioned by video_id, so every view of a video lands on the same consumer.
2. Each consumer counts in memory in per-minute buckets and keeps a local top-K heap.
3. Windows: hour = sum of the last 60 minute-buckets (subtract the expiring one). Results are cached; raw events go to S3 for batch correction.

AVOID

× A counter row per video in the DB: 800k writes/sec on hot rows.
× Adding parts before a number shows they are needed.
× Ignoring retries, timeouts and crashes.

"Stream views through Kafka partitioned by video ID; each consumer counts its videos in memory and keeps a local top-K."

DDIA: Ch 11 windows, partitioned streams · Ch 6 partition by key · Similar: Ad click aggregator · Web crawler · Metrics monitoring · Fitness tracking

System Design, Step by Step · CC BY-SA 4.0

URL SHORTENER (BITLY)

It sounds like a toy, but it is the cleanest example of a read-heavy system: almost every request is a redirect.

Easy | Scaling reads

TL;DR

1. Redirects outnumber new links about 100 to 1, so the design is mostly about serving reads fast.
2. Generate short codes from a counter encoded in base62: unique by construction, no collision checks.
3. Put a cache in front of the database; a small share of links gets most clicks, so the hit rate is high.

REQUIREMENTS

- ✓ Create a short URL for a long URL (optional custom alias, expiry)
- ✓ Redirect short URL → original URL

Non-functional

- Redirect p99 < 100ms
- 99.99% available for redirects
- Short codes globally unique
- Read:write ≈ 100:1

NUMBERS

100M DAU × 10 clicks = 1B redirects/day ≈ 12k/s avg, ~100k/s peak.

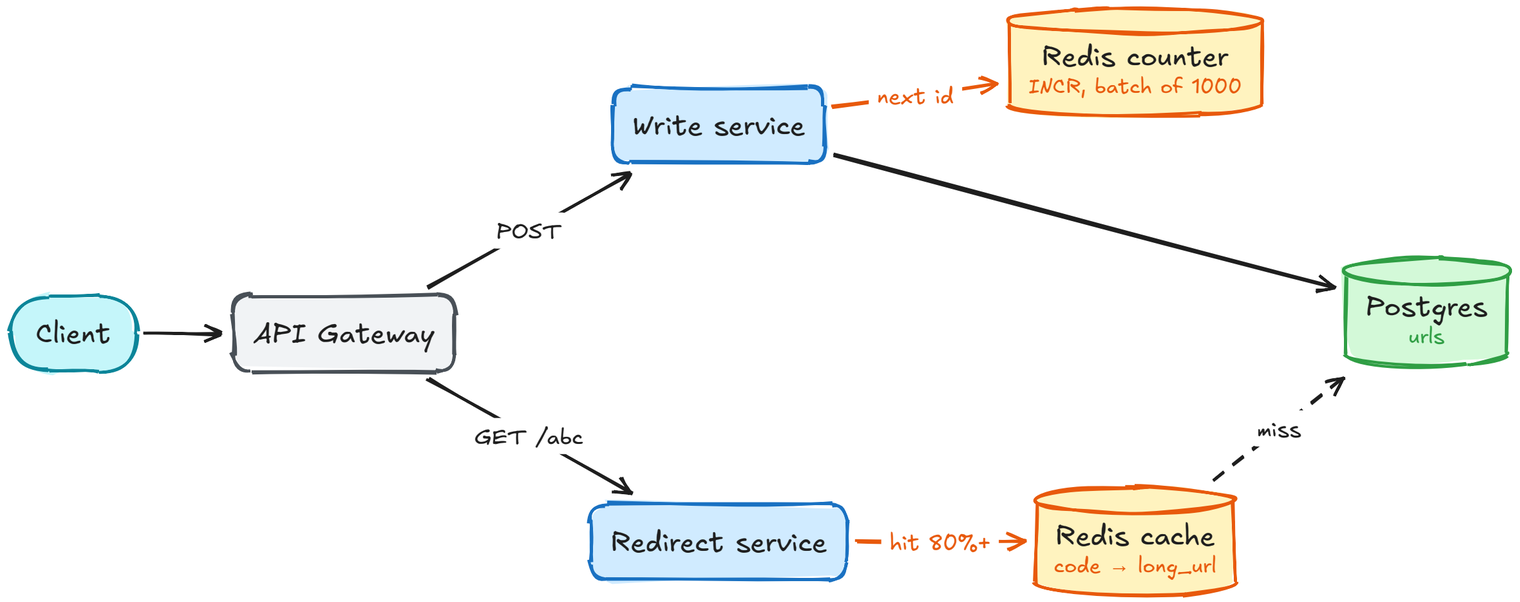
Writes ~1M/day ≈ 12/s.

1B URLs × 500B = 500GB, which fits one Postgres.

API + DATA

```
POST /urls {long_url, alias?, expires_at?} → short_url
GET /{code} → 302 Location: long_url
urls(code PK, long_url, user_id, created_at, expires_at)
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Start simple: a write service creates a code and stores it in Postgres. ~12 writes/sec is nothing.

Redirects are 100× more frequent. 100k reads/sec straight to Postgres is what breaks first.

DEEP DIVES

How do you guarantee unique codes?
Counter + base62 is unique by construction. Hashing (MD5 prefix) needs a unique constraint plus retry on collision.

301 or 302?
302: every click reaches you (analytics, expiry, editing). 301 lets browsers cache it, which cuts load but loses analytics.

What if the counter node dies?
Redis with AOF + replica, or servers lease ranges from a Postgres sequence. A crash wastes a range; gaps are harmless.

Custom aliases?
Same table with a unique constraint; the insert fails if taken. Keep namespaces apart (e.g., counter codes are 7 chars, aliases ≥ 8).

HOW IT EVOLVES

1. Redirects are 100× more frequent. 100k reads/sec straight to Postgres is what breaks first.
2. Add a Redis cache (code → URL). Clicks follow Pareto, so 80%+ hit the cache in under 1ms. Postgres only sees misses; add replicas if needed.
3. Generate codes from a global counter: Redis INCR + base62. 7 chars = 3.5 trillion codes, no collisions. Servers grab IDs in batches of 1,000.

AVOID

- × Sharding 500GB at 12 writes/sec. It fits one machine; scale reads with cache and replicas.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Generate short codes from a counter encoded in base62: unique by construction, no collision checks."

DDIA: Ch 5 replicas · Ch 6 hash sharding · Similar: Rate limiter · Business discovery · News feed · News aggregator

System Design, Step by Step · CC BY-SA 4.0

VIDEO PLATFORM (YOUTUBE)

Two hard problems: getting gigabyte uploads in reliably, and streaming smoothly on any connection.

Hard | Handling large blobs | Scaling reads

TL;DR

1. Upload directly to object storage with presigned, multipart, resumable URLs; bytes never touch your servers.
2. Split videos into segments and transcode them in parallel into several resolutions.
3. Serve segments from a CDN and let the player switch quality per segment (adaptive bitrate).

REQUIREMENTS

- ✓ Upload videos (up to many GB)
- ✓ Watch videos with smooth playback on any network

Non-functional

- Resumable uploads
- Fast start, adaptive quality
- ~1M uploads/day; billions of views
- Metadata availability over consistency

NUMBERS

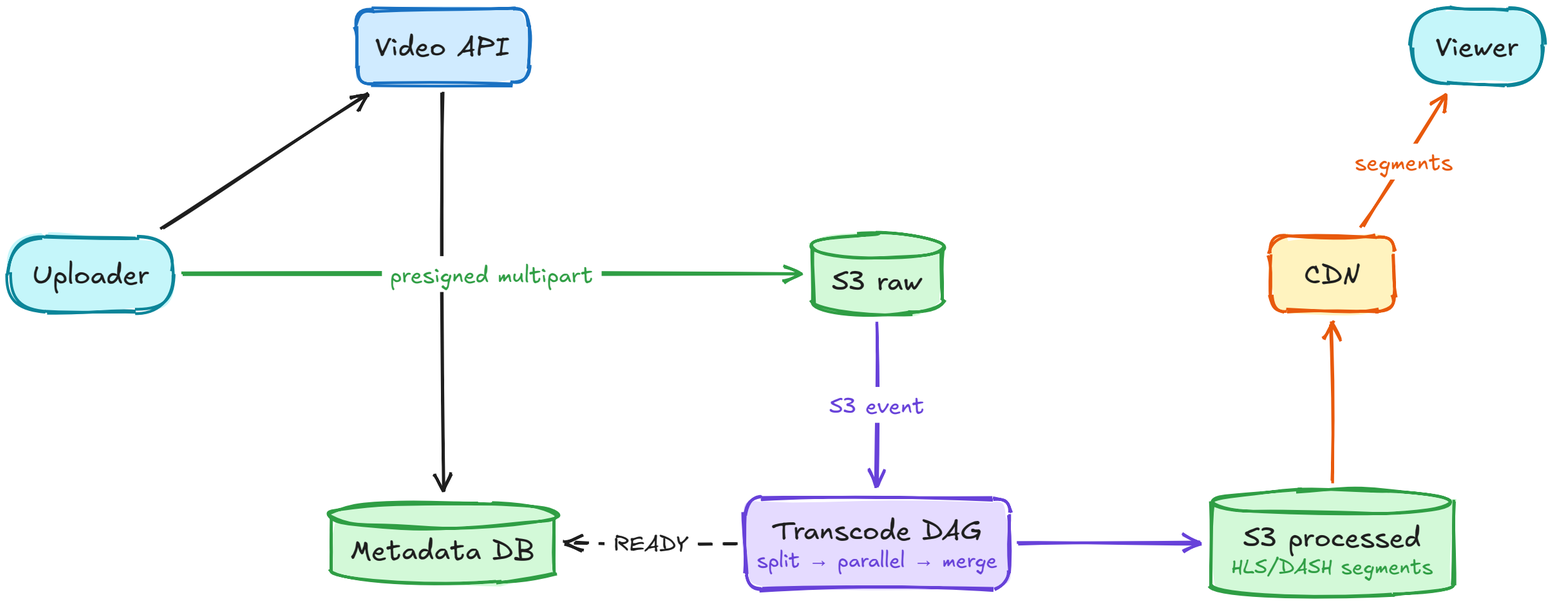
1M uploads/day × 1GB raw ≈ 1PB/day raw, plus several renditions.

Views are mostly served from the CDN, not origin.

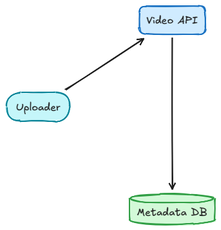
API + DATA

```
POST /videos → {video_id, presigned multipart URLs}
GET /videos/{id} → manifest URL
PUT parts directly to S3
videos(id, uploader, title, status
UPLOADING|PROCESSING|READY, manifest_url)
S3 raw/ + processed/{rendition}/{segment}
```

FINAL DESIGN



V1, AND WHY IT BREAKS



Naive: upload gigabytes through your API servers. Slow, fragile, and expensive.

DEEP DIVES

Why presigned URLs?
Bytes never touch your servers; S3 handles scale; multipart gives parallel, resumable parts.

Fast transcoding?
Segment-level parallelism across many workers; the DAG orchestrator tracks completion.

Adaptive bitrate?
The manifest lists renditions; the player switches per segment.

Cost?
Transcode popular formats eagerly; long-tail renditions lazily.

HOW IT EVOLVES

1. The API returns presigned multipart URLs; the client uploads parts directly to S3 in parallel and resumes failed parts.
2. An S3 event starts a transcoding DAG: split into short segments, transcode each to several resolutions in parallel, write HLS/DASH manifests, and mark the video READY.
3. Viewers fetch the manifest and segments from the CDN; the player picks the quality per segment from measured bandwidth (adaptive bitrate).

AVOID

- × Serving or uploading whole files through app servers.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Split videos into segments and transcode them in parallel into several resolutions."

DDIA: Ch 10 batch DAGs · Ch 1 bandwidth math · Similar: File storage and sync · Photo sharing

System Design, Step by Step · CC BY-SA 4.0

WEB CRAWLER

Ten billion pages in five days is a pipeline problem, constrained by politeness and network bandwidth.

Hard | Scaling writes

TL;DR

1. About 23k pages per second; size the fleet by network bandwidth, not CPU.
2. Split fetching and parsing into separate stages with queues and stored raw pages, so each scales and retries on its own.
3. Respect per-domain rate limits and robots.txt, and skip already-seen URLs and duplicate content.

REQUIREMENTS

- ✓ Crawl from seed URLs, fetch pages, extract text, follow links
- ✓ Store text for LLM training: 10B pages in 5 days

Non-functional

- Politeness: robots.txt, per-domain rate limits
- Fault tolerant: resume without losing progress
- No duplicate URLs or content
- Efficient (network-bound)

NUMBERS

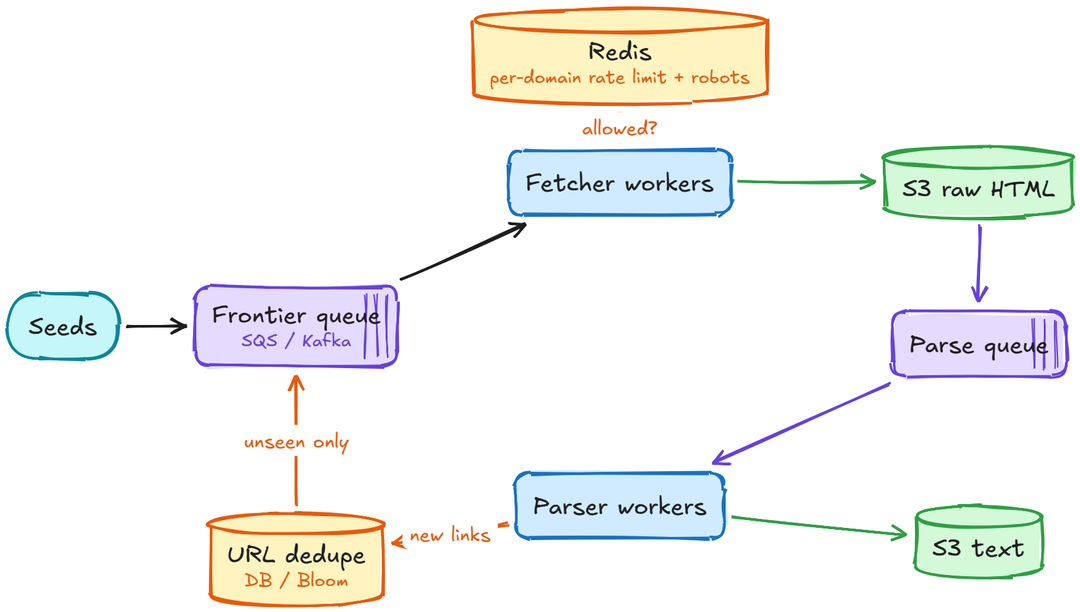
10B ÷ 5 days ≈ 23k pages/s. × ~2MB ≈ 46GB/s, so size the fleet by network bandwidth, not CPU.

Raw storage ~20PB → S3.

API + DATA

```
internal: frontier queue messages {url, depth}
S3: raw/{url_hash}, text/{url_hash}
urls(url_hash PK, url, status, last_crawled)
domains(domain, robots, crawl_delay,
next_allowed_at)
```

FINAL DESIGN



V1, AND WHY IT BREAKS

Start with a frontier queue of URLs and fetcher workers pulling from it.

Split fetch (network-bound) from parse (CPU-bound) with S3 + a second queue. Each scales independently, and parsing can be re-run from stored HTML.

DEEP DIVES

Why separate stages?
Different bottlenecks, independent scaling and retries, and you can re-parse 20PB without re-crawling.

Crash mid-crawl?
A message is acked only after the raw page is in S3: at-least-once, idempotent by URL hash.

Crawler traps?
Max depth per domain, URL normalization, content-hash dedupe.

DNS bottleneck?
Cache DNS per worker; multiple resolvers.

HOW IT EVOLVES

1. Split fetch (network-bound) from parse (CPU-bound) with S3 + a second queue. Each scales independently, and parsing can be re-run from stored HTML.
2. Politeness: per-domain token buckets and cached robots.txt in Redis. Not allowed yet? Re-queue with a delay (visibility timeout).
3. Extracted links pass a dedupe check (URL hash in DB or Bloom filter) before entering the frontier. Content hashes catch mirrored pages.

AVOID

- × One monolithic fetch→parse loop: a slow parser stalls fetching and a crash loses in-flight work.
- × Adding parts before a number shows they are needed.
- × Ignoring retries, timeouts and crashes.

"Split fetching and parsing into separate stages with queues and stored raw pages, so each scales and retries on its own."

DDIA: Ch 10 batch pipelines · Ch 3 Bloom filters · Ch 11 queues · Similar: Top-K videos · Ad click aggregator · Metrics monitoring · Fitness tracking

System Design, Step by Step · CC BY-SA 4.0

DEALING WITH CONTENTION

Many users want the same thing at the same moment. Exactly one should win, every time.

Pattern

✓ WHEN TO REACH FOR IT

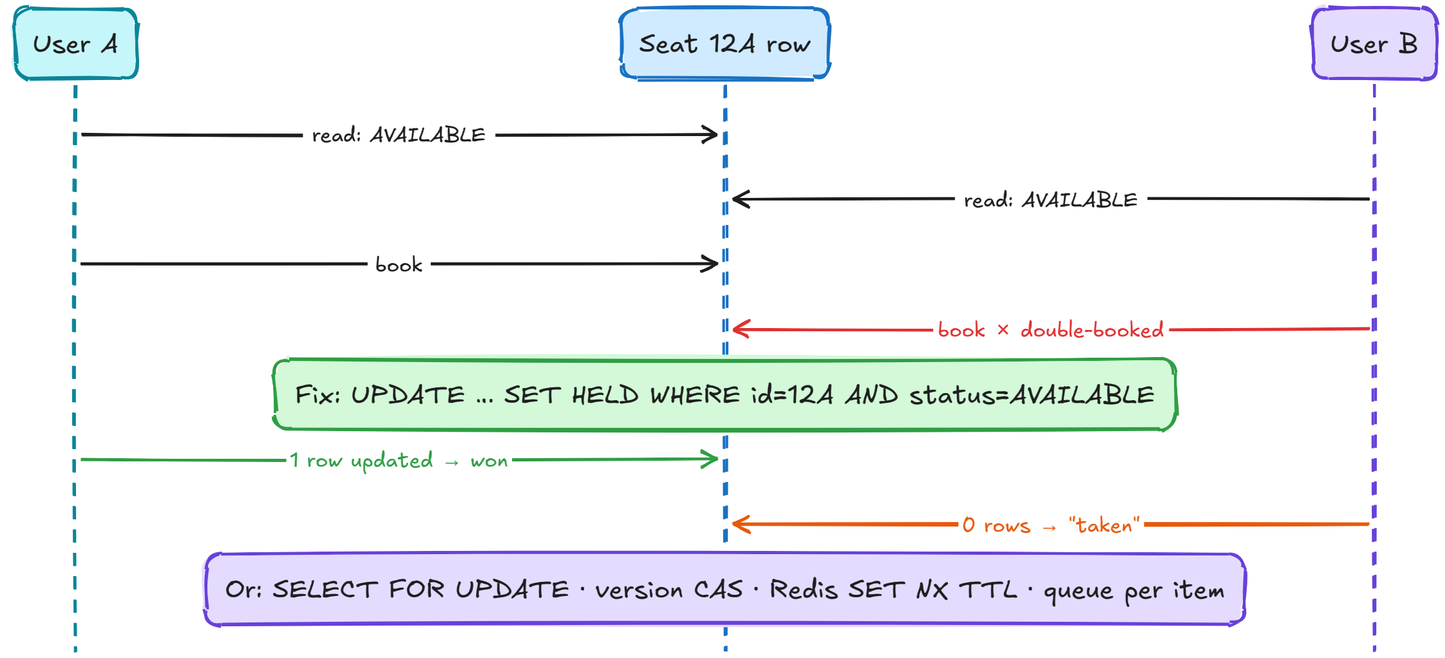
- Limited resources: seats, stock, a username, a driver, the highest bid.
- Any "check, then act" sequence that two requests can run at the same time.

⚡ STEP BY STEP

1. Check-then-act from two users: both see AVAILABLE, both book. A race condition (DDIA Ch 7 write skew).
2. Make the check and the write one atomic statement. The database guarantees only one succeeds.
3. Other tools: pessimistic lock (high contention), optimistic version check (low contention), distributed lock with TTL (holds across services), or a per-item queue (extreme contention).

Tool	Use when
Atomic conditional UPDATE	Single row, single DB: the default
SELECT … FOR UPDATE	Multi-step logic on few rows, high contention
Optimistic (version column)	Low contention; retry on conflict
Redis lock SET NX PX	Temporary holds across services (seat hold, driver lock)
Queue per resource	Extreme hot spots (auction last seconds)

◎ THE IDEA



✓ DECISION CHECKLIST

1. Can the check and the write be one statement? Use a conditional UPDATE (… WHERE status = AVAILABLE). This is the default.
2. Multi-step logic on a few rows with high contention? SELECT … FOR UPDATE, locking rows in a fixed order.
3. Low contention? Optimistic concurrency with a version column; retry on conflict.
4. A temporary hold that spans services? A lock with a TTL (Redis SET NX EX), plus a fencing token if correctness depends on it.
5. Extremely hot single item? Serialize requests through a queue partitioned by item.

💬 SAY THIS

"The check and the write must be atomic, so the hold is a single conditional UPDATE. One row updated means you got the seat; zero means someone else did."

⚠ AVOID · USED IN

- × Check-then-act in application code.
- × Holding database locks while waiting on a human (payment, form).
- × Trusting a distributed lock without fencing when a paused process could wake up later.

Used in: Rate limiter, Online auction, Ticketmaster

Theory: DDIA ch 7 Transactions · DDIA ch 8 Trouble with distributed systems

System Design, Step by Step · CC BY-SA 4.0

HANDLING LARGE BLOBS

Files, photos and videos belong in object storage, and their bytes should never pass through your application servers.

Pattern

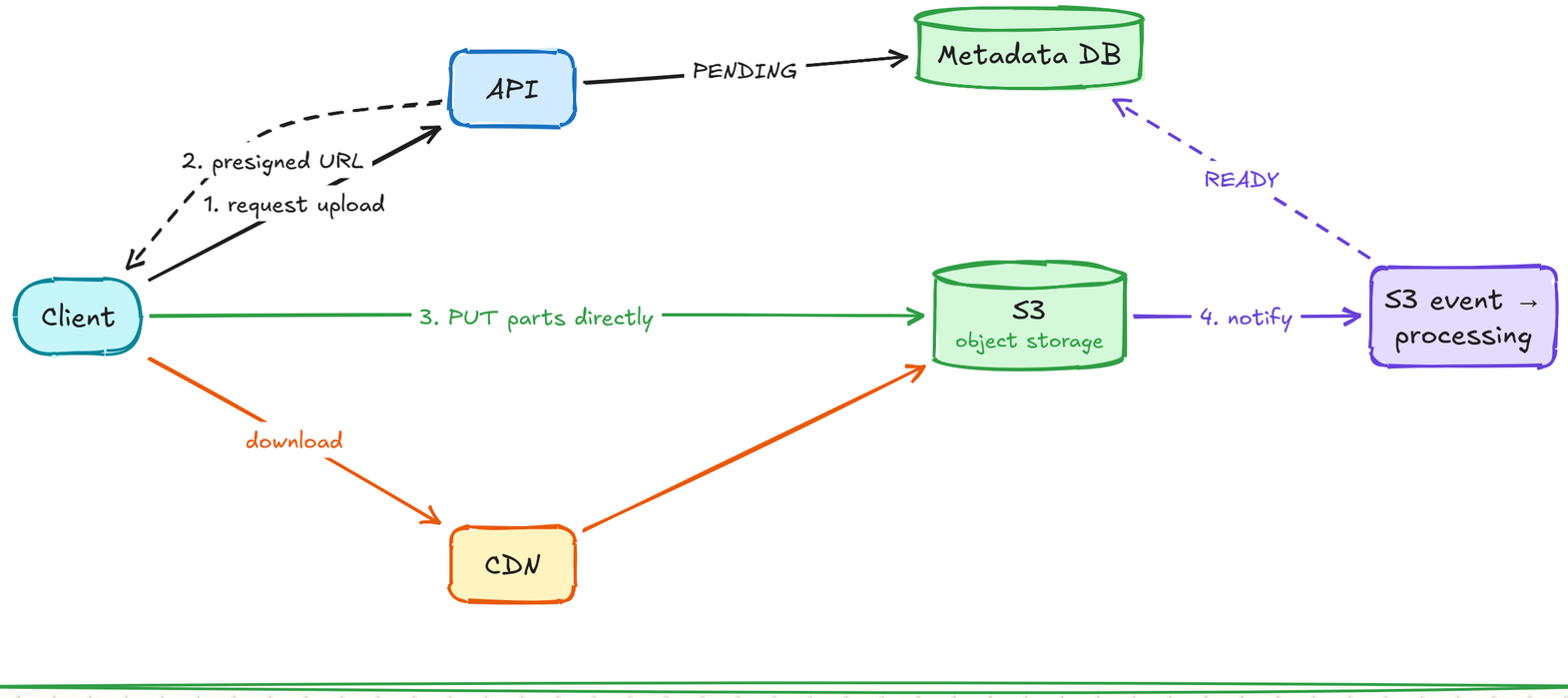
✓ WHEN TO REACH FOR IT

- Users upload or download files larger than a few megabytes.
- Media needs processing after upload (thumbnails, transcoding).

⚡ STEP BY STEP

1. Pushing gigabytes through your API servers wastes their bandwidth and memory and fails on long uploads.
2. The API hands out a presigned URL; the client uploads directly to S3 (multipart: parallel, resumable chunks). Metadata is recorded separately.
3. S3 event notifications trigger processing (thumbnails, transcoding) and flip status to READY. Don't trust the client to say it finished.
4. Downloads come from a CDN (or presigned GET URLs), never through the app servers.

◎ THE IDEA



✓ DECISION CHECKLIST

1. Upload with presigned URLs directly to object storage.
2. Use multipart upload for anything big: parallel, resumable chunks.
3. Store metadata (owner, status, size, checksum) in a database, separate from the bytes.
4. React to storage events to start processing; mark the item ready only when done.
5. Serve downloads through a CDN or presigned GET URLs.

💬 SAY THIS

"The API only hands out a presigned multipart URL; the client uploads straight to S3. An S3 event triggers processing, and the file shows as ready when processing finishes."

⚠ AVOID · USED IN

- × Streaming file bytes through API servers.
- × Storing blobs in the database.
- × Trusting the client to report that an upload completed.

Used in: Video platform (YouTube), File storage and sync (Dropbox), Photo sharing (Instagram)

System Design, Step by Step · CC BY-SA 4.0

LONG-RUNNING TASKS

If work takes longer than a request should, accept it, queue it, and let the client check back.

Pattern

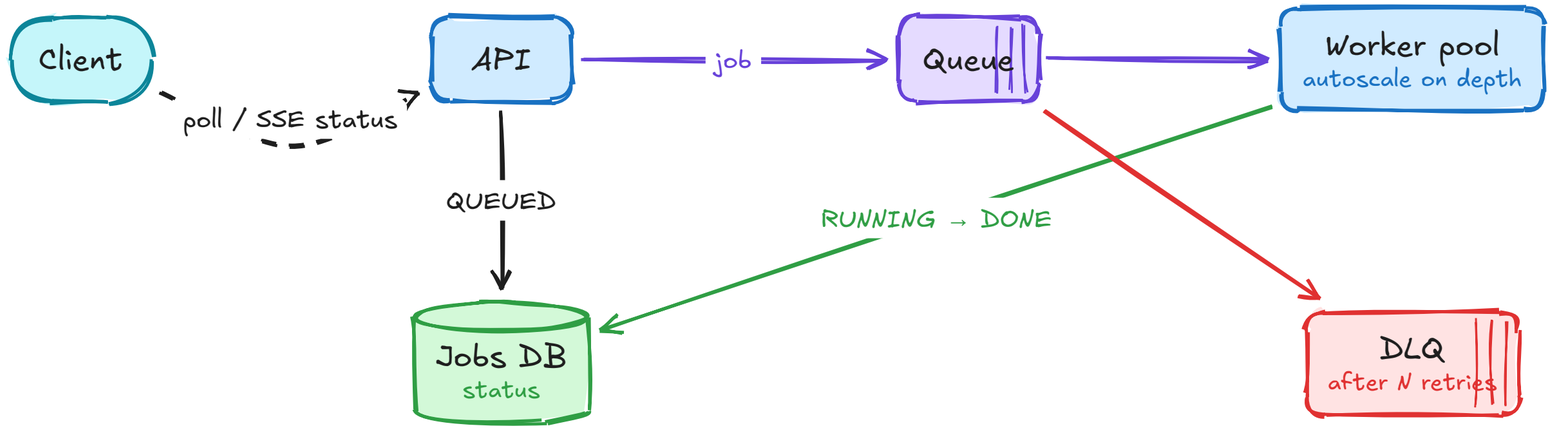
✓ WHEN TO REACH FOR IT

- Work takes more than a second or two: transcoding, report generation, code execution, model inference.
- Work is bursty and expensive, so it needs backpressure.

STEP BY STEP

1. Doing heavy work inside the request: timeouts, retried requests duplicate work, and no backpressure.
2. Return a job ID immediately; enqueue the work. Workers scale independently of the API.
3. Workers update the job status; the client polls or subscribes (SSE/WebSocket) for progress.
4. Failures retry with backoff (visibility timeout); poison jobs go to a dead-letter queue. Jobs must be idempotent.

THE IDEA



✓ DECISION CHECKLIST

1. Return a job ID immediately (HTTP 202) and enqueue the work.
2. Workers scale on queue depth, independently of the API.
3. Store status (queued, running, done, failed); clients poll or subscribe.
4. Retry with backoff using a visibility timeout; send poison jobs to a dead-letter queue.
5. Make jobs idempotent: a retried job must not duplicate side effects.

SAY THIS

"Submission returns a job ID right away. The job goes on a queue; autoscaled workers run it and update its status, and the client polls every second."

AVOID · USED IN

- × Doing heavy work inside the HTTP request.
- × No dead-letter queue, so one bad job blocks or loops forever.
- × Non-idempotent jobs combined with at-least-once delivery.

Used in: Distributed cache, Online judge (LeetCode), Job scheduler

Theory: DDIA ch 11 Stream processing · DDIA ch 8 Trouble with distributed systems

System Design, Step by Step · CC BY-SA 4.0

MULTI-STEP PROCESSES

A business action spans several services and any step can fail halfway. Make progress durable and failures reversible.

Pattern

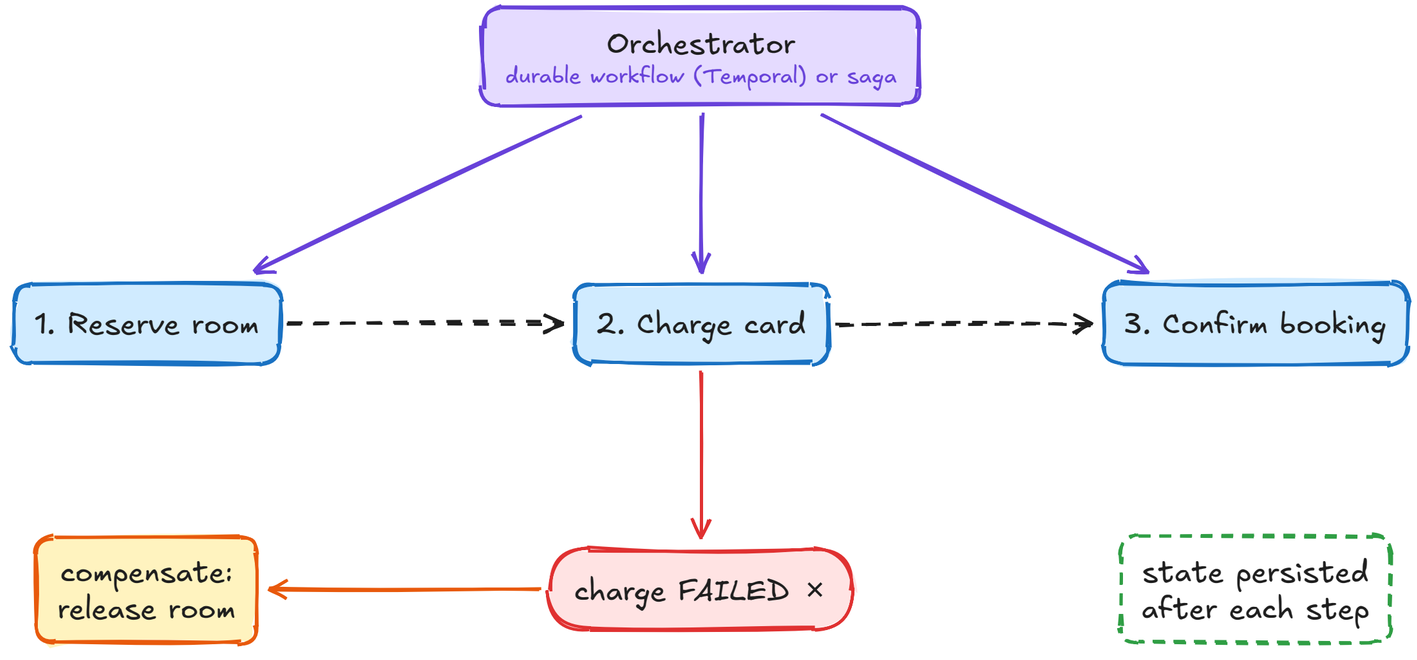
WHEN TO REACH FOR IT

- One user action touches several services or external APIs: booking + payment + confirmation, ride lifecycle, order fulfilment.
- Steps can take seconds to days, or depend on outside callbacks.

STEP BY STEP

1. A business action spans several services, and no distributed ACID transaction is available.
2. Any step can fail after earlier steps succeeded.
3. A saga runs a compensating action for each completed step (refund, release, cancel), in reverse order.
4. A durable orchestrator records progress after each step, so a crash resumes where it stopped instead of starting over or forgetting.

THE IDEA



DECISION CHECKLIST

1. Is there a cross-service transaction? Replace it with a saga: a compensating action for each step.
2. Who drives the steps? An orchestrator (Temporal, Step Functions) is easier to reason about than event choreography for complex flows.
3. Are all steps idempotent so retries are safe?
4. Do events leave the database reliably? Use the outbox pattern.
5. What happens on timeouts with unknown outcomes? Query status; don't blindly retry side effects.

SAY THIS

"There is no distributed transaction across booking and payment, so I'll run a saga: hold the room, charge, confirm, and if the charge fails, release the hold. A workflow engine persists progress so a crash resumes instead of restarting."

AVOID · USED IN

- × Proposing two-phase commit across microservices.
- × Non-idempotent steps that double-charge on retry.
- × Dual writes to a database and a queue without an outbox.

Used in: Payment system (Stripe), Online judge (LeetCode), Job scheduler

Theory: DDIA ch 9 Consistency and consensus · DDIA ch 11 Stream processing · DDIA ch 12 The future of data systems

System Design, Step by Step · CC BY-SA 4.0

REAL-TIME UPDATES

Push changes to clients the moment they happen, even when the sender and receiver are connected to different servers.

Pattern

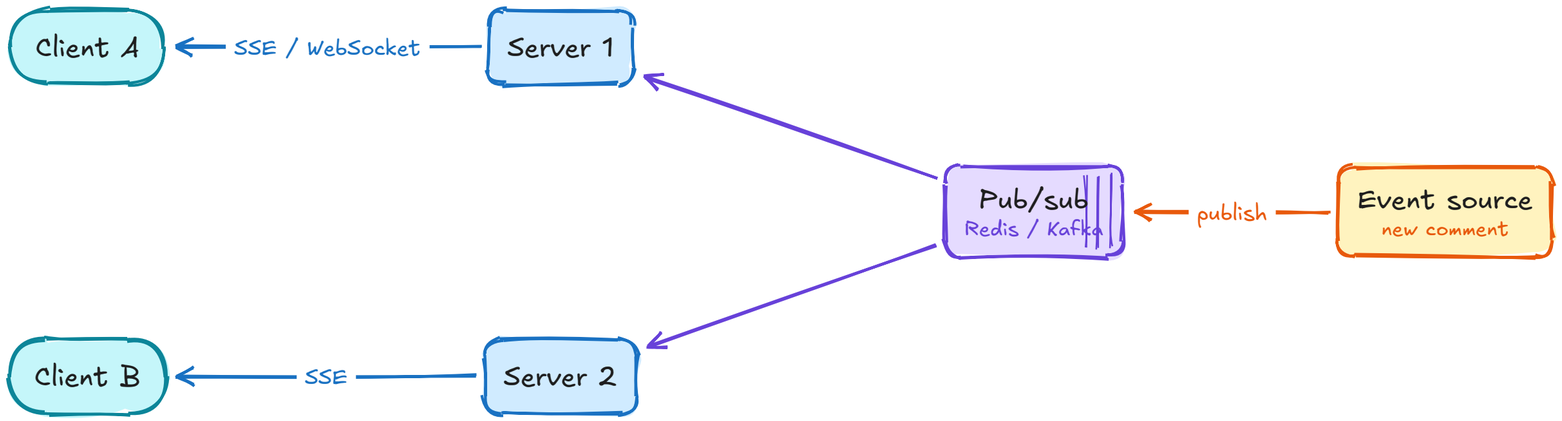
✓ WHEN TO REACH FOR IT

- Users expect to see changes within about a second: chat, live comments, prices, collaboration, ride tracking.
- Polling would be mostly empty responses or far too frequent.

STEP BY STEP

1. Polling: simple, but latency = interval and most requests return nothing. Fine for slow updates (every ~30s).
2. Server push: SSE (server→client, plain HTTP, auto-reconnect) or WebSocket (bidirectional). Long-polling is the fallback.
3. With many servers, the receiver's connection lives on another server than the one that got the event.
4. Pub/sub routes events to whichever servers hold the interested connections: channel per video, per user, or per symbol.

THE IDEA



✓ DECISION CHECKLIST

1. How fresh must it be? Every 30 s or slower: just poll.
2. Server-to-client only? Use Server-Sent Events (plain HTTP, auto-reconnect).
3. Both directions, low latency? Use WebSockets.
4. Many servers? Route events with pub/sub (Redis, Kafka) to whichever server holds the connection.
5. Can events be lost? If not, persist first and let clients resume from an offset after reconnecting.

SAY THIS

"Viewers only receive, so SSE is enough. Each comment is published once to a channel per video; every server holding viewers of that video is subscribed and pushes it to them."

⚠ AVOID · USED IN

- × Polling at large scale.
- × Assuming sender and receiver share a server.
- × No plan for reconnects and missed messages.

Used in: Ride sharing (Uber), Live comments (Facebook Live), Collaborative editor (Google Docs), Stock trading (Robinhood), Online auction, LLM chat (ChatGPT), Chat app (WhatsApp)

Theory: DDIA ch 11 Stream processing

System Design, Step by Step · CC BY-SA 4.0

SCALING READS

Most systems read far more than they write. Make the common read cheap, and the database only sees what the cache can't answer.

Pattern

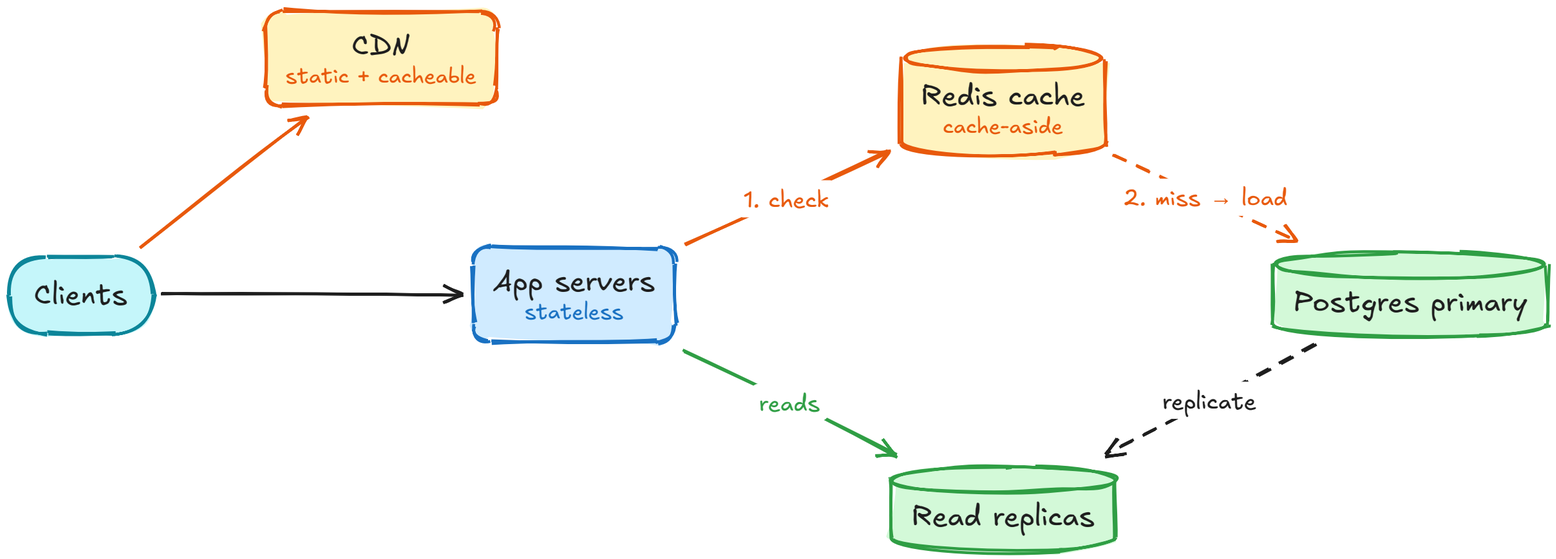
✓ WHEN TO REACH FOR IT

- Read:write ratio is 10:1 or higher.
- The same items are read over and over (popularity is skewed).
- A read path misses its latency target, or the calculated read QPS exceeds what one database serves (~10–50k/s).

STEP BY STEP

1. One database serves every read. At ~10k+ reads/sec it becomes the bottleneck.
2. Cache-aside: check Redis; on a miss, read the DB and populate the cache with a TTL. A hit takes <1ms vs ~5–50ms from the DB.
3. Read replicas spread the remaining reads (expect replication lag, DDIA Ch 5).
4. CDN serves static and shared responses from the edge, so origin never sees them.

THE IDEA



✓ DECISION CHECKLIST

1. Is the response the same for many users? Put a CDN in front.
2. Is it per-user but read often? Cache-aside in Redis with a TTL.
3. Still too much load on the primary? Add read replicas and accept replication lag.
4. Is one key extremely hot? Replicate the key or add a short-lived local cache.
5. Which reads must never be stale? Send those to the primary.

SAY THIS

"Reads outnumber writes about 100 to 1 and popularity is skewed, so a cache-aside Redis layer should absorb most reads. The rest go to replicas; only checkout reads the primary."

⚠ AVOID · USED IN

- × Sharding the database before trying a cache and replicas.
- × Caching data that must be exact (stock at checkout, balances).
- × Forgetting cache stampedes on hot keys.

Used in: URL shortener (Bitly), Rate limiter, Business discovery (Yelp), News feed (Facebook), News aggregator (Google News), Local delivery (Gopuff), Dating app (Tinder), Distributed cache, Post search (Facebook), Price tracker (CamelCamelCamel), Ticketmaster, Video platform (YouTube), Photo sharing (Instagram)

Theory: DDIA ch 5 Replication · DDIA ch 6 Partitioning

System Design, Step by Step · CC BY-SA 4.0

SCALING WRITES

When writes outgrow one machine, absorb the spikes, write in batches, split the data, and write less.

Pattern

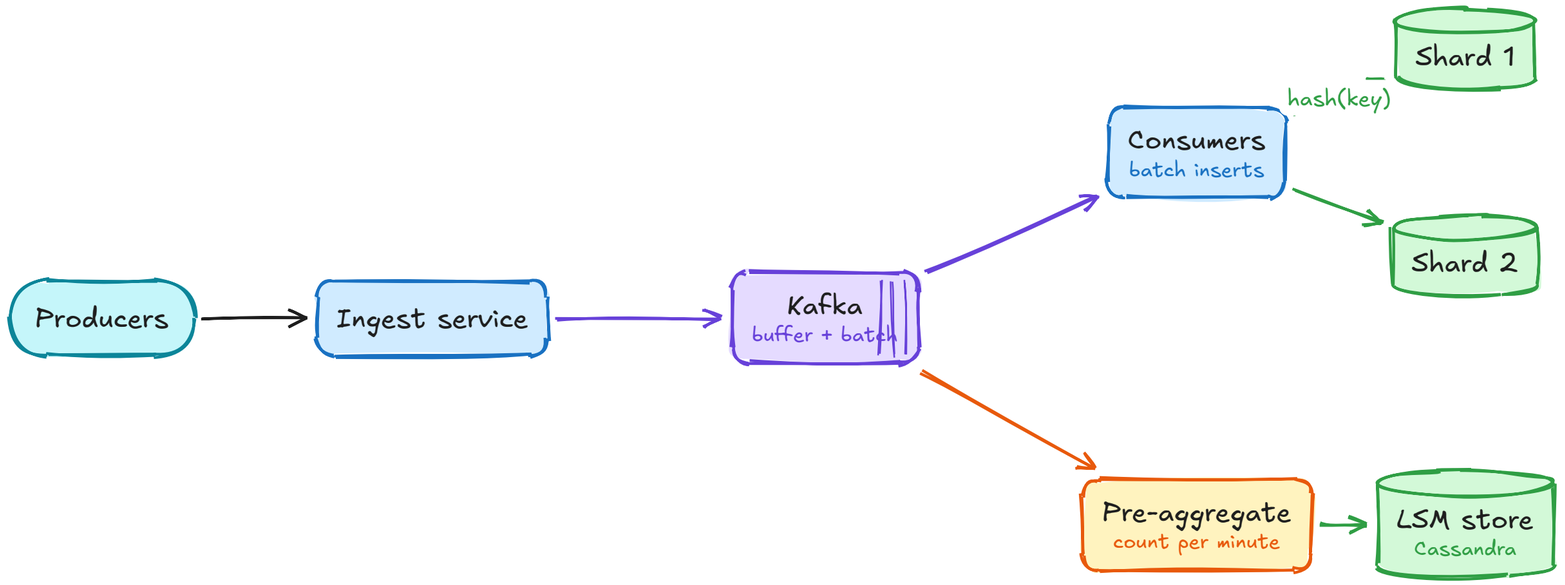
WHEN TO REACH FOR IT

- Sustained writes exceed what one primary handles (roughly 10k+ per second for simple rows).
- Write traffic is spiky: events, clicks, telemetry.
- Data is append-mostly and read in aggregate (counts, time series).

STEP BY STEP

1. Every event is an individual synchronous DB write. Spikes overwhelm the database.
2. A queue/log (Kafka) absorbs spikes and decouples producers from the DB. Consumers write in batches.
3. Shard by a key that spreads load evenly (hash of id); each shard takes a fraction of the writes.
4. Reduce the writes: pre-aggregate (10k clicks → 1 row per minute), and use write-optimized LSM storage.

THE IDEA



DECISION CHECKLIST

1. Can the client get a quick ack while the work happens later? Put a durable log (Kafka) in front.
2. Can writes be grouped? Batch inserts in consumers.
3. Can you store less? Pre-aggregate (one row per key per minute instead of one per event).
4. Still too much for one node? Shard by a key that spreads load evenly.
5. Write-heavy and simple key access? Use an LSM-based store such as Cassandra.

SAY THIS

"Clicks arrive at 10k/s with spikes, so they go to Kafka first. A stream job aggregates per ad per minute, which cuts the database writes by orders of magnitude."

AVOID · USED IN

- × A hot partition: one key still lands on one shard.
- × Losing data by acknowledging before anything is durable.
- × Using a B-tree OLTP database for a firehose of append-only events.

Used in: Top-K videos (YouTube), Ad click aggregator, Web crawler, Metrics monitoring (Datadog), Fitness tracking (Strava), Post search (Facebook), Price tracker (CamelCamelCamel)

Theory: DDIA ch 3 Storage and retrieval · DDIA ch 6 Partitioning · DDIA ch 11 Stream processing

System Design, Step by Step · CC BY-SA 4.0

API DESIGN

Spend five minutes, get the important details right, and move on to the design.

Core concept

HOW IT WORKS

Rule	Example
Resources = core entities	GET /events/{id}, POST /bookings
Cursor pagination, not offset	GET /feed?cursor=(ts,id)&limit=20: stable when new items arrive
Idempotency on mutations	Idempotency-Key header + unique index → retries are safe
Auth from token, not body	user_id comes from the JWT; never trust {user_id} in the request
Async for long work	POST returns 202 + job_id; GET /jobs/{id}
Version	/v1/... or header; add optional fields only

"REST with resources from our entities: POST /bookings with an idempotency key, cursor-paginated GET /bookings. User identity comes from the auth token."

Spending 15 minutes on endpoint shapes is a classic way to run out of time for the deep dives.

CHECKLIST

1. Name resources after your core entities.
2. Use cursor pagination, not offsets.
3. Accept an Idempotency-Key on every mutating call.
4. Take the user's identity from the auth token, never from the body.
5. Return 202 and a job ID for long work.

SAY THIS

"Three endpoints cover the requirements; writes take an idempotency key, lists use cursor pagination, and identity comes from the token."

System Design, Step by Step · CC BY-SA 4.0

CACHING

Keep hot data in memory so most reads never reach the database.

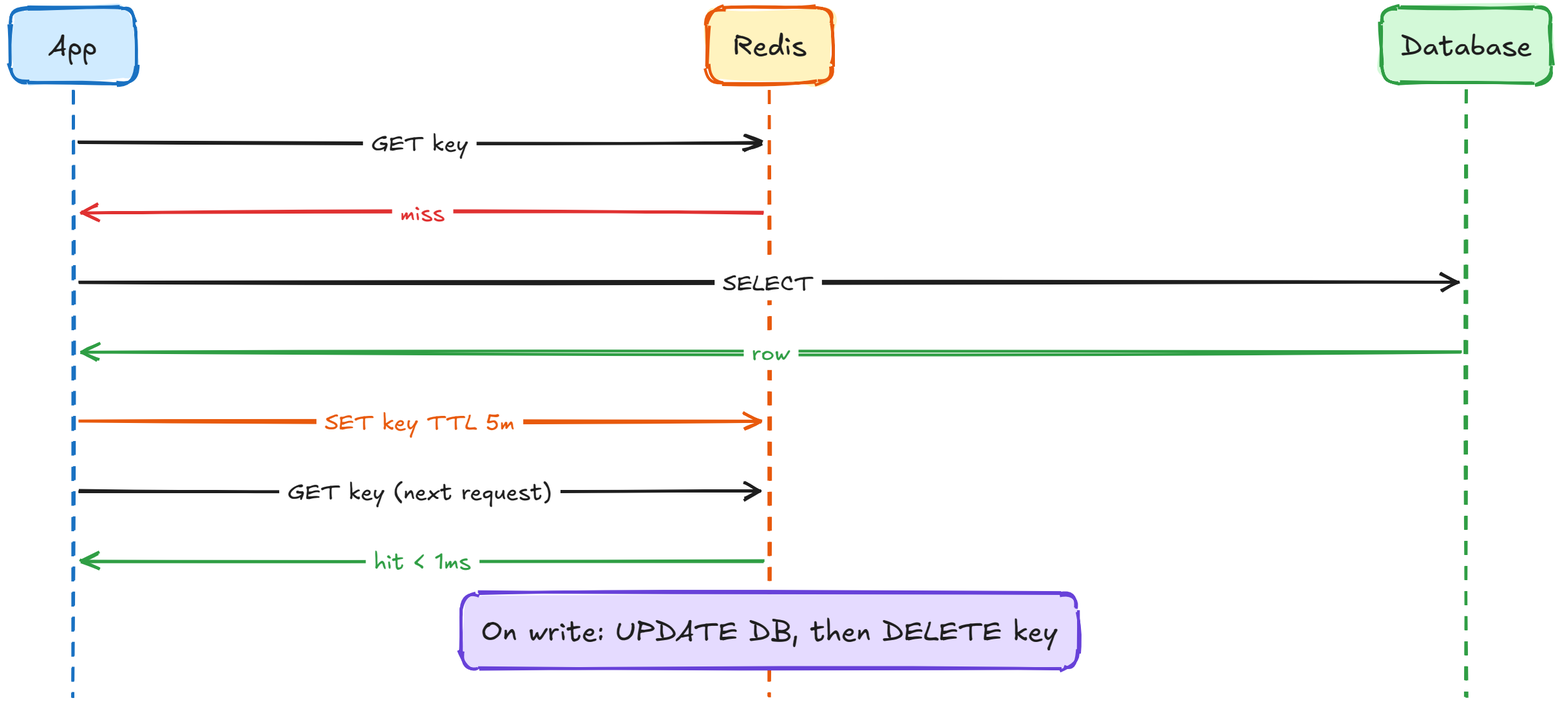
Core concept

HOW IT WORKS

1. Cache-aside: check the cache first.
2. On a miss, read the database.
3. Populate the cache with a TTL.
4. The next request is served from memory.
5. On writes, update the DB and delete (not update) the cached key, so the next read refreshes it.

Strategy	Trade-off
Cache-aside	Default; app controls; possible brief staleness
Write-through	Write cache + DB together; fresher, slower writes
Write-behind	Write cache, flush later; fast, risk of loss
CDN	Edge cache for static or shared responses

THE PICTURE



CHECKLIST

1. Add a cache when numbers show the DB can't keep up, or a read path misses its latency target.
2. Default: cache-aside. Read cache → on miss read DB → set with TTL.
3. On write: update the DB, then DELETE the key (safe under concurrent writers).
4. Always set a TTL so invalidation bugs heal themselves.
5. Guard hot keys: single-flight rebuilds, jittered TTLs, local L1 cache.
6. Never cache what must be exact at checkout.

SAY THIS

"Reads outnumber writes 100 to 1 and popularity is skewed, so cache-aside Redis should absorb 80–90% of reads."

System Design, Step by Step · CC BY-SA 4.0

CAP THEOREM AND PACELC

When the network splits, choose between refusing requests and answering with possibly stale data, separately for each feature.

Core concept

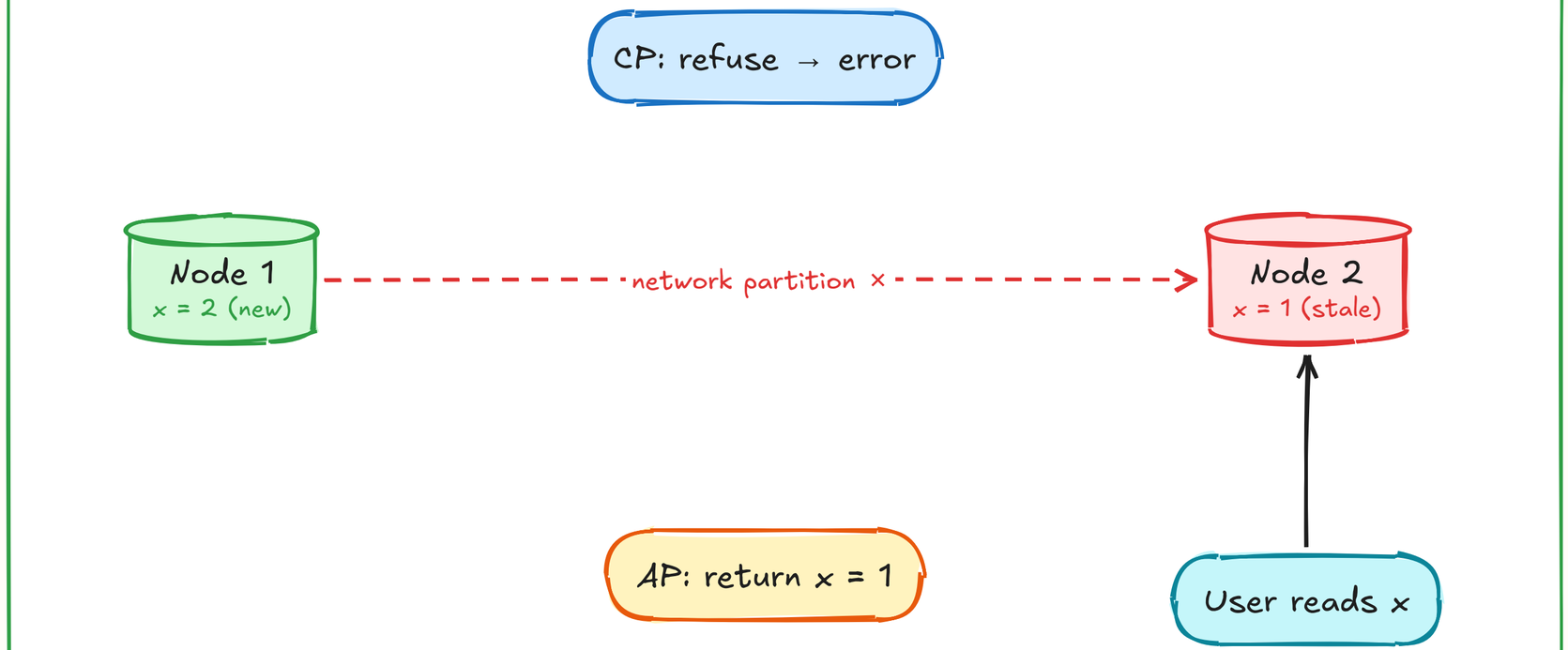
HOW IT WORKS

1. The network between two replicas breaks. Partitions will happen, so P is not optional.
2. A user asks the node that can't see the latest write.
3. Either refuse (consistent, unavailable) or answer with stale data (available, inconsistent).

Feature	Choice
Booking, payments, inventory	CP: never oversell or double-charge
Search, feeds, reviews, likes	AP: stale for seconds is fine
Usernames / unique constraints	CP (linearizable)

"CAP is a per-feature choice: the booking path is CP, browsing is AP."

THE PICTURE



CHECKLIST

1. Partitions will happen, so the real choice is C or A during one.
2. Choose per feature, not per system.
3. Money, inventory and uniqueness: consistency (CP).
4. Feeds, search, reviews, counters: availability (AP).
5. PACELC: even without partitions, stronger consistency costs latency.

SAY THIS

"Booking is CP: I'd rather reject than double-sell. Browsing is AP: a few seconds stale is fine."

System Design, Step by Step · CC BY-SA 4.0

CONSISTENT HASHING

Add or remove a node and move only a small share of the keys.

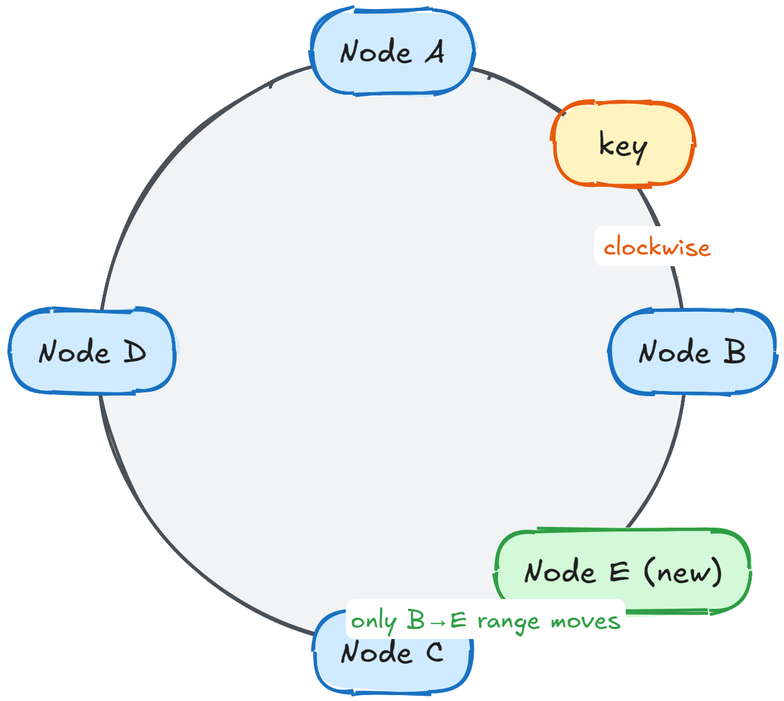
Core concept

HOW IT WORKS

1. Nodes and keys hash onto a ring.
2. A key belongs to the next node clockwise.
3. Adding a node moves only the keys in one arc (~1/N), unlike hash % N which moves almost everything. Virtual nodes even out the load.

Distributed caches, Cassandra, DynamoDB, L7 routing of video/doc IDs to servers.

THE PICTURE



CHECKLIST

1. Place nodes and keys on the same hash ring.
2. A key belongs to the first node clockwise from it.
3. Use virtual nodes (many points per server) to even out load.
4. Adding a node moves roughly 1/N of keys; hash % N moves almost all of them.

SAY THIS

"Cache nodes sit on a consistent-hash ring with virtual nodes, so adding a node remaps about 1/N of keys instead of emptying the whole cache."

System Design, Step by Step · CC BY-SA 4.0

DATA MODELING

Start from the queries the product runs, then choose keys and indexes that make each one cheap.

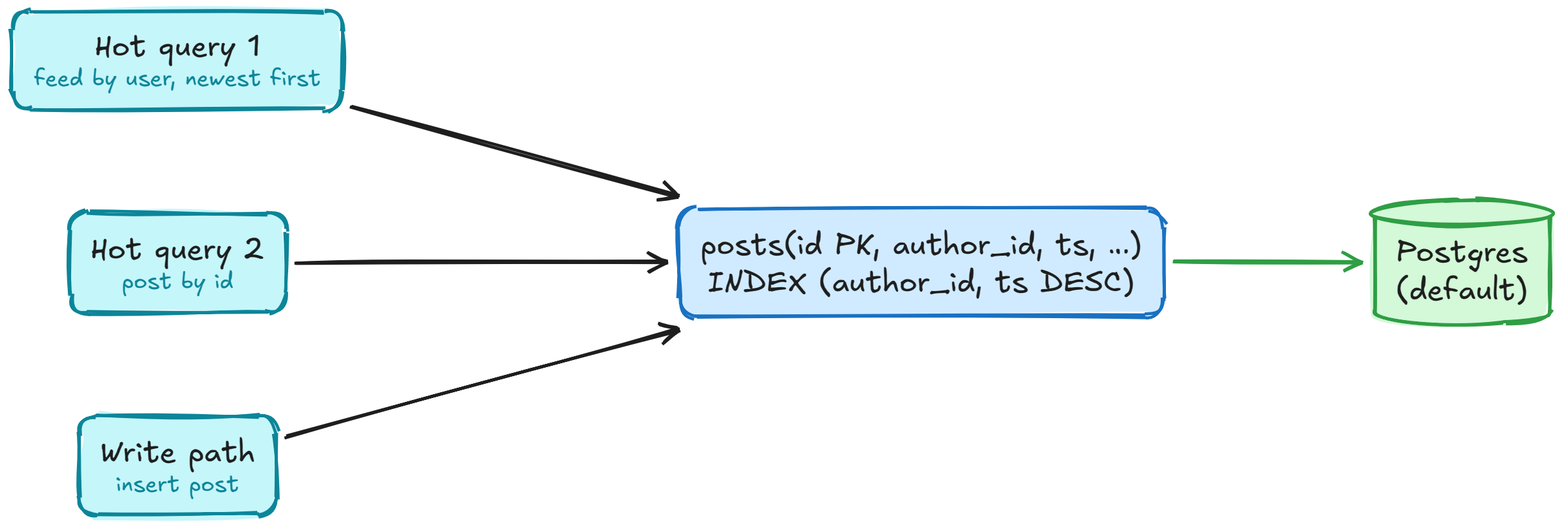
Core concept

HOW IT WORKS

1. Start from the queries the product actually runs, not from the nouns.
2. Choose keys and indexes that make each hot query a single index lookup or range scan.
3. Default to Postgres; deviate only with a reason (write volume → Cassandra, flexible docs → DynamoDB/Mongo, search → Elasticsearch).

Normalize by default (one place to update). Denormalize deliberately for a hot read path, and say how the copy stays in sync (CDC, async job).

THE PICTURE



CHECKLIST

1. List the two or three hottest queries first.
2. Make each one a key lookup or a single index range scan.
3. Default to Postgres; switch only with a reason you can state.
4. Normalize by default; denormalize deliberately for a hot read and say how the copy stays in sync.
5. Choose the partition key from the hottest query if you will shard.

SAY THIS

"The hot query is a user's posts, newest first, so the posts table gets an index on (author_id, created_at DESC)."

System Design, Step by Step · CC BY-SA 4.0

DATABASE INDEXING

An index turns a scan of every row into a short walk down a tree. Pick the right kind for the query.

Core concept

HOW IT WORKS

Index	Use for	Example
B-tree	Equality + range + ORDER BY	created_at > X; composite (user_id, ts)
Hash	Equality only	session lookup
Geospatial (R-tree, geohash)	Nearby queries	Yelp, Uber, Gopuff
Inverted	Full-text search	Post search, Elasticsearch
Composite	Multi-column filters (leftmost prefix rule)	WHERE user_id=? ORDER BY ts
Covering	Index-only scans	INCLUDE (title)

"A composite index on (user_id, created_at DESC) makes the feed query a single range scan."

Every index slows writes; index only the hot queries.

CHECKLIST

1. Equality and ranges: B-tree (the default).
2. Several filters: a composite index, ordered by the leftmost-prefix rule.
3. Distance queries: a geospatial index (R-tree, geohash).
4. Full-text search: an inverted index.
5. Every index slows writes, so index only the queries you need.

SAY THIS

"A composite index on (user_id, created_at DESC) turns the feed query into one index range scan."

System Design, Step by Step · CC BY-SA 4.0

NETWORKING ESSENTIALS

How a request travels from a user to your service, and which protocol to pick for which job.

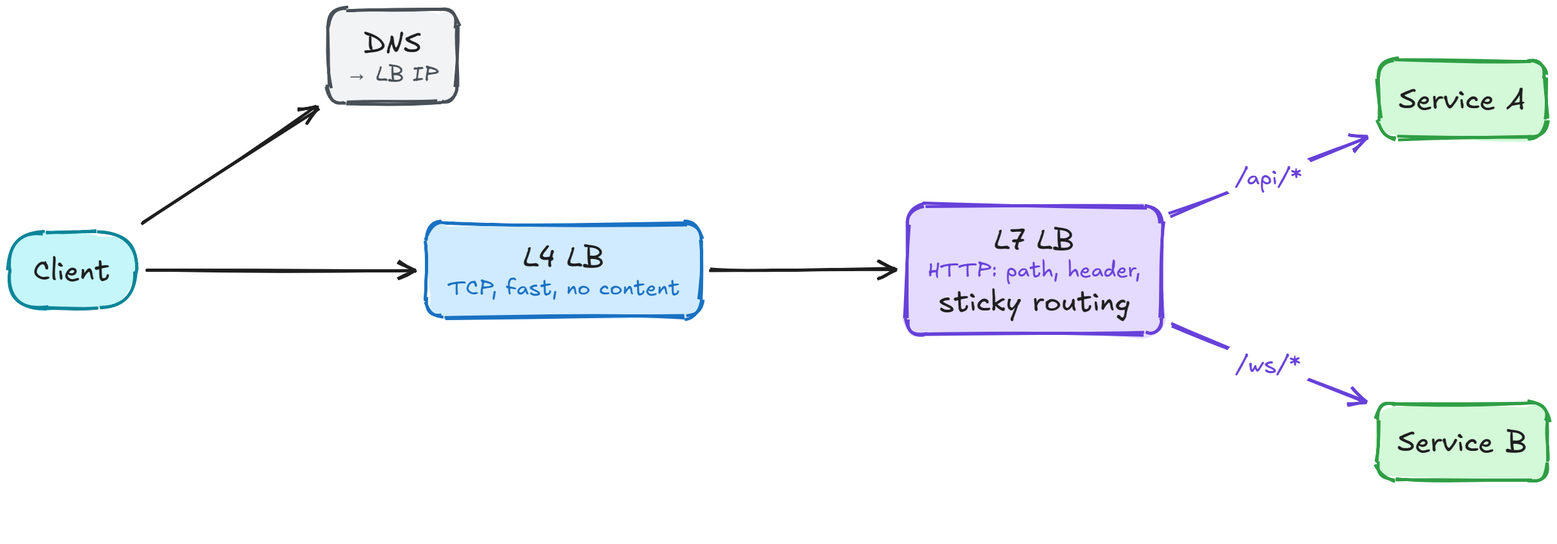
Core concept

HOW IT WORKS

1. DNS resolves the name; an L4 load balancer forwards TCP connections without reading them. Very fast, good for WebSockets and raw TCP.
2. An L7 load balancer reads HTTP and routes by path, header, or cookie; it terminates TLS and can do sticky sessions.

Protocol	Use
REST over HTTP	Default public API
gRPC (HTTP/2 + Protobuf)	Internal service-to-service, high QPS
SSE	Server→client stream (feeds, tokens, comments)
WebSocket	Bidirectional real-time (chat, docs, games)
Long polling	Fallback push when nothing else works

THE PICTURE



CHECKLIST

1. Public API used by browsers and partners: REST over HTTPS.
2. High-volume calls between internal services: gRPC.
3. Server pushes a stream to the client: SSE.
4. Two-way, low-latency messages: WebSocket.
5. Routing by URL path, header or cookie: L7 load balancer. Raw TCP or very high throughput: L4.

SAY THIS

"Clients talk REST through an L7 load balancer; internal services use gRPC. Live updates go over SSE, which reconnects automatically."

System Design, Step by Step · CC BY-SA 4.0

SHARDING

Split data across machines when one can't hold it or keep up with the writes, but only then.

Core concept

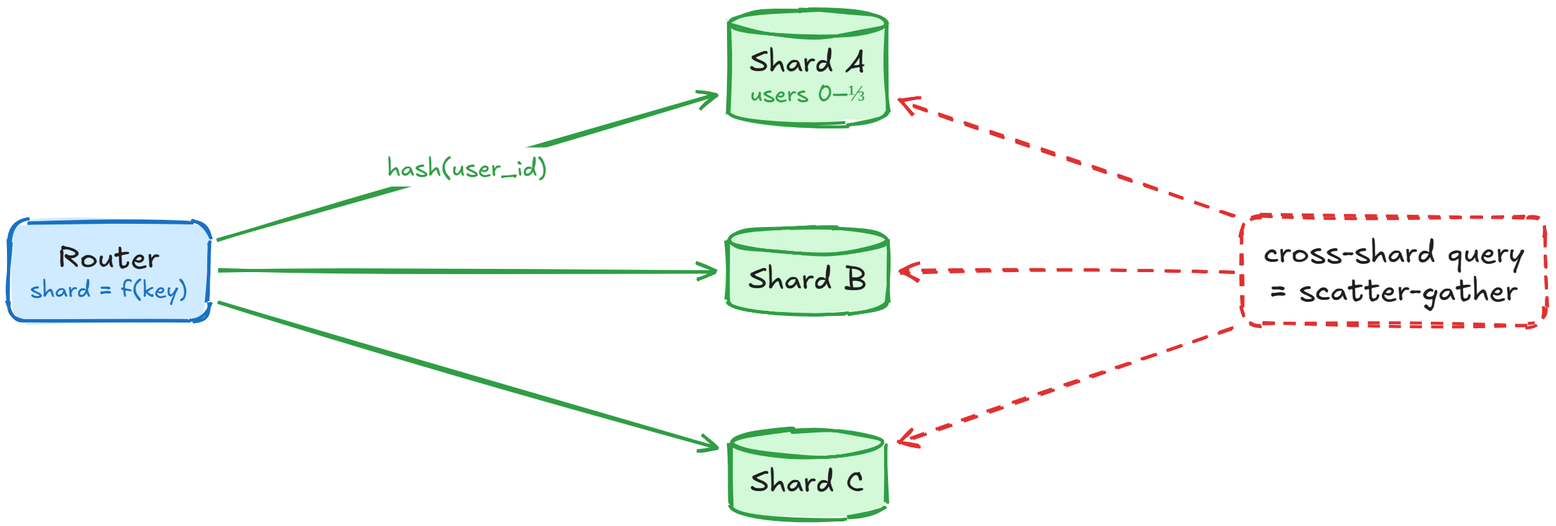
HOW IT WORKS

1. Pick a shard key from the hottest access pattern so most queries hit one shard. Hash for even spread; range for range scans.
2. Queries that don't include the shard key hit every shard. That is the cost you accept.

One Postgres handles ~10–50k TPS and multiple TB. Shard when the calculation says so.

Even with hashing, one hot key overloads one shard: key splitting, caching, or buffering.

THE PICTURE



CHECKLIST

1. Has the calculation shown one primary isn't enough after caching and replicas?
2. Pick the shard key from the hottest query, so most requests hit one shard.
3. Hash for even spread; ranges only if you need range scans on the key.
4. Plan for hot keys and for rebalancing (many logical partitions, or consistent hashing).
5. Accept that queries without the shard key fan out to every shard.

SAY THIS

"Writes outgrow one primary, so I'll shard by hash of user_id: every hot query includes it. Cross-user analytics can scatter-gather or run on a warehouse."

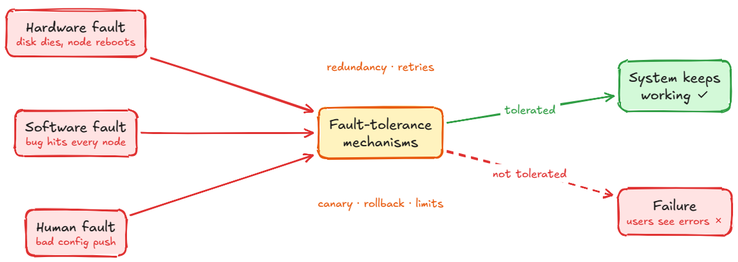
System Design, Step by Step · CC BY-SA 4.0

DDIA 1 · RELIABLE, SCALABLE, MAINTAINABLE

The three words every NFR discussion comes back to.

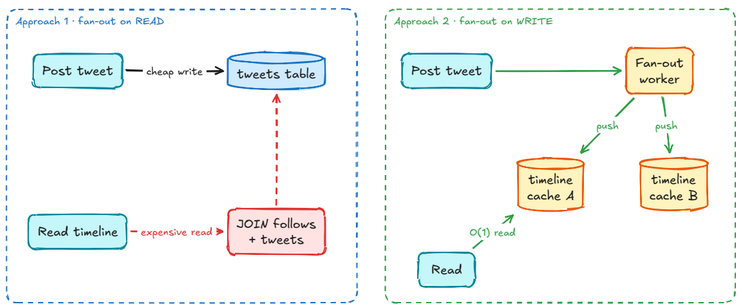
faults vs failures | load parameters | p99 latency | tail amplification

FAULT ≠ FAILURE



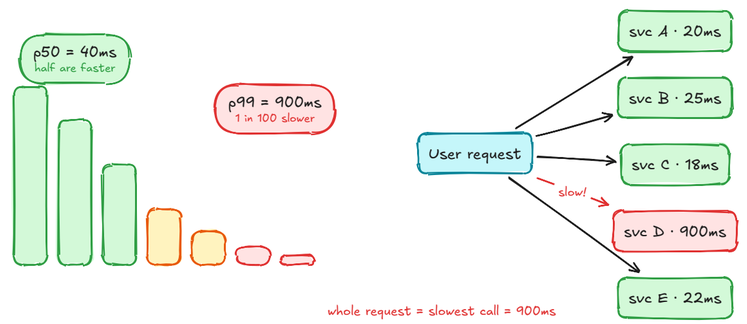
A failure is when the whole system stops serving users. You design so faults don't become failures.

DESCRIBING LOAD



Reads become one cache lookup. The cost moved to write time.

DESCRIBING PERFORMANCE



The user waits for the slowest backend. With 100 backends at 1% slow each, 63% of user requests hit at least one slow call.

SCALING UP VS SCALING OUT, AND

Choice	What it means	When it wins
Scale up (vertical)	Bigger machine: more cores, RAM, NVMe	Simple, no distribution bugs. A single Postgres box handles 10–50k TPS. Start here.
Scale out (horizontal)	More machines, shared-nothing	When one box is not enough or you need fault tolerance.
Elastic	Add or remove capacity automatically	Unpredictable load. Manual scaling is simpler and has fewer surprises.
Operability	Easy for ops to keep running	Monitoring, good defaults, predictable behaviour, runbooks.
Simplicity	Remove accidental complexity	Good abstractions, for example SQL hiding storage and concurrency.
Evolvability	Easy to change later	Loose coupling, schema evolution (Ch 4), tests.

There is no "magic scaling sauce". An architecture for 100k req/sec of 1KB reads looks nothing like one for 3 req/min of 2GB uploads.

IN THE INTERVIEW

"How will this scale?"
Name the load parameter (reads/sec, fan-out, data size), calculate it, then say which component breaks first.

"What's your latency target?"
p99 < 200ms for reads, not averages. Mention tail amplification if requests fan out.

"What if a node dies?"
Separate fault from failure: "A node dying is a fault. Replicas plus health checks mean it never becomes a failure."

"Before choosing components, let me pin down the load parameters: DAU, read:write ratio, and peak QPS, because they decide whether we need any distribution at all."

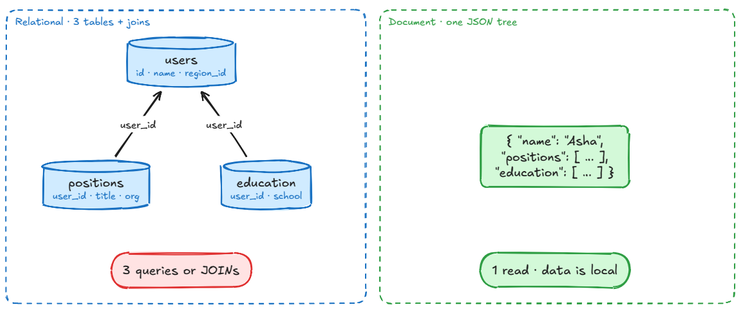
System Design, Step by Step · CC BY-SA 4.0

DDIA 2 · DATA MODELS & QUERY LANGUAGES

The shape of your data decides which database fits. Trees of data that are read together (one-to-many) suit documents.

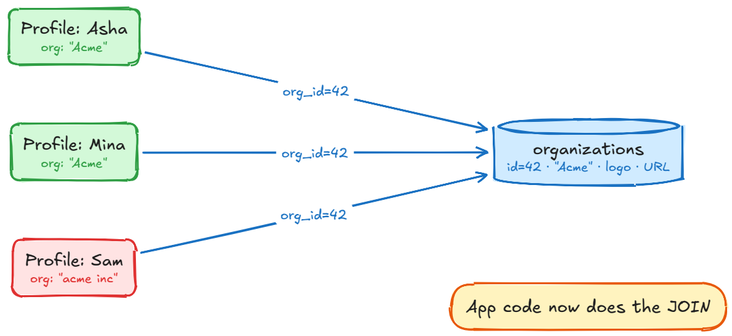
relational | document | graph | normalization

ONE LINKEDIN PROFILE, TWO MODELS



Document: a profile is a self-contained tree. One read fetches it all because the data has locality. This works when the data is one-to-many and read as a whole.

WHERE DOCUMENTS BREAK



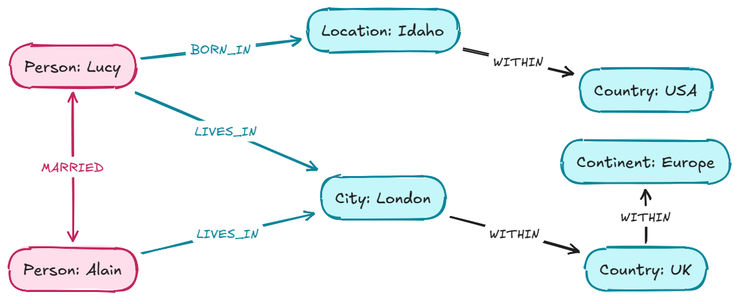
Document databases have weak join support, so the join moves into application code. As data gains references, the relational model wins again.

SCHEMA-ON-WRITE VS SCHEMA-ON-READ

	Schema-on-write (relational)	Schema-on-read (document)
Analogy	Static typing: checked at compile time	Dynamic typing: checked at run time
Change a field	ALTER TABLE + migration (fast in Postgres; MySQL used to copy the whole table)	Just write new docs; code handles both old and new shapes
Good for	Uniform records with stable structure	Heterogeneous items: different shapes per type, or external data you don't control
Locality	Rows are spread across tables (but see Postgres JSONB, Spanner interleaving)	Whole doc stored together; rewrites the whole doc on update, so keep docs small
Joins	Native, optimized by query planner	Weak or none; done in application

There is always a schema, it's just implicit in the reading code. You still pay for migrations, just later and in more places.

GRAPH MODEL



A recursive query ("WITHIN, any number of hops") is one line in Cypher and painful in SQL (recursive CTE).

IN THE INTERVIEW

Data shape	Pick	Interview example
Lots of cross-references, need transactions	Relational (Postgres)	Bookings, payments, inventory, users
Self-contained tree, read whole	Document (Mongo, DynamoDB item, JSONB)	Hotel content, product catalog page, chat message
Relationships are the query	Graph (Neo4j) or adjacency list	Social graph, recommendations, fraud
Huge write volume, simple key access	Wide-column (Cassandra)	Chat history, metrics, activity logs

"I start from access patterns. The two hot queries are X and Y, and both are key lookups with one join, so Postgres with the right indexes covers it. I'd only move to Cassandra if write volume passes about 50k/sec."

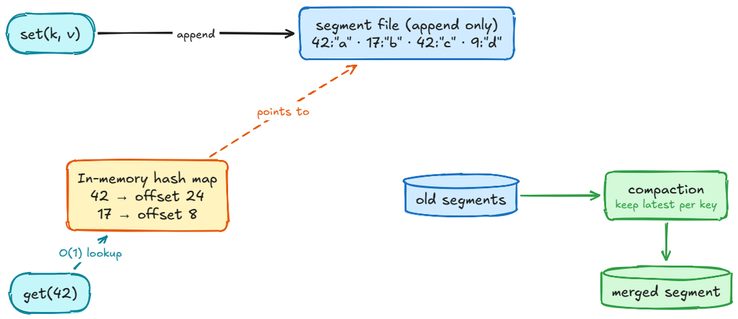
System Design, Step by Step · CC BY-SA 4.0

DDIA 3 · STORAGE & RETRIEVAL

How a database actually stores bytes.

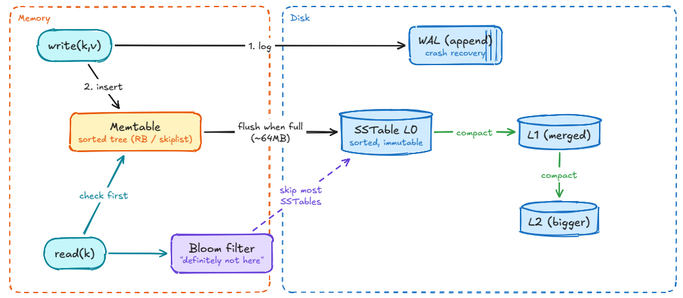
append-only log | hash index | SSTable | LSM-tree

THE SIMPLEST DATABASE



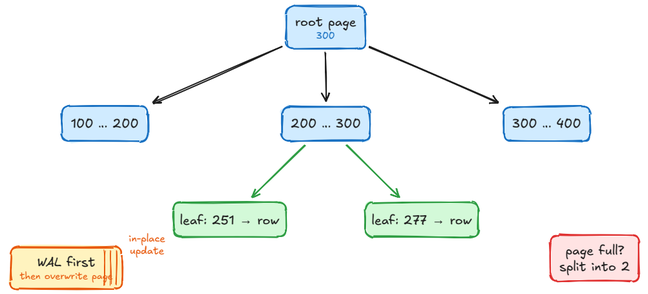
Old values pile up, so background compaction merges segments and keeps only the latest value per key.

LSM-TREE



A read checks the memtable, then SSTables from newest to oldest. Bloom filters skip files that can't contain the key, so reads cost more than writes.

B-TREE



Updates overwrite the page in place, after writing to the WAL first so a crash mid-write can be repaired. A full page splits in two.

LSM VS B-TREE

	LSM-tree (Cassandra, RocksDB, HBase)	B-tree (Postgres, MySQL InnoDB)
Writes	Faster: sequential appends, less write amplification	Slower: random page writes + WAL (each write ≥ 2 writes)
Reads	Slower: may check several SSTables (Bloom filters help)	Faster, predictable: one path root → leaf
Space	Compresses well, little fragmentation	Pages partly empty after splits
Weak spot	Compaction competes with live traffic, so p99 spikes	Write-heavy workloads hit I/O limits
Transactions	Key may exist in many files, so locking is harder	Key in one place: easy range locks
Pick when	Write-heavy: metrics, logs, chat, IoT, events	Read-heavy, transactional: bookings, payments, users

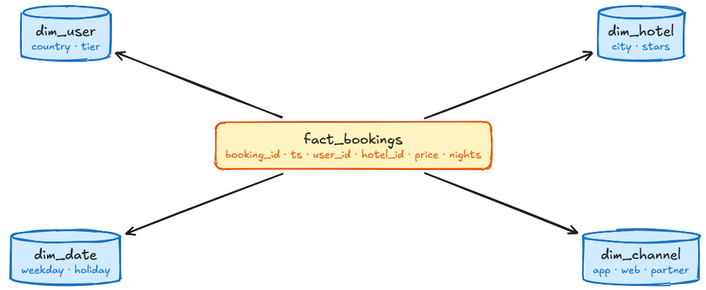
"Ad clicks are write-heavy and append-only, around 10k/sec, so an LSM-based store like Cassandra fits.

INDEX TYPES YOU SHOULD NAME IN

Index	How	Use for
B-tree (default)	Sorted tree on column(s)	Equality + range: WHERE created_at > X, ORDER BY
Composite	Sorted by (a, b): leftmost prefix rule	WHERE user_id=? ORDER BY ts DESC: one index scan
Covering / INCLUDE	Index stores extra columns	Index-only scans: no heap lookup
Clustered	Rows stored in index order	InnoDB primary key; range scans on PK are cheap
Hash	Hash map	Equality only; no ranges
Geospatial	R-tree, geohash, quadtree (PostGIS)	"Hotels within 5km": Yelp, Uber, Gopuff
Inverted	term → list of doc IDs	Full-text search: Elasticsearch, Postgres GIN
Bloom filter	Probabilistic set membership	"Seen this URL?" in a web crawler, LSM reads

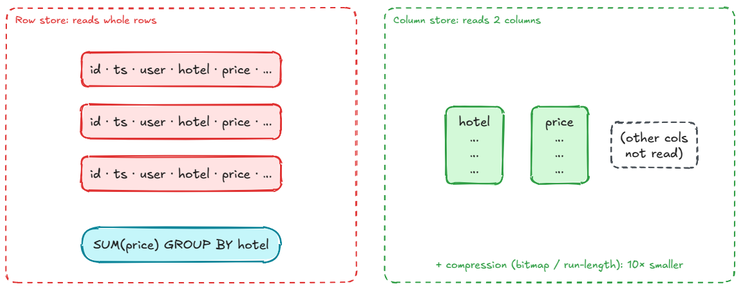
Each index slows down every write and uses disk. Index for the hot read queries you named, not for every column.

OLTP VS OLAP, AND THE STAR SCHEMA



Dimension tables hold the who, what, where, and when. Queries filter on dimensions and aggregate facts: "revenue by city by weekday".

COLUMN-ORIENTED STORAGE



Values in one column look alike, so they compress very well. Vectorized CPU processing on top makes these scans 10–100× faster.

IN THE INTERVIEW

"Why Cassandra for this?"
"LSM-tree storage: writes are sequential appends to memtable + commit log, so write throughput is very high.

"How do you make this query fast?"
Name the exact index: "a composite index on (user_id, created_at DESC) turns it into a single index range scan."

"Where do analytics queries run?"
Never on the OLTP primary. Use CDC or Kafka into a column store (ClickHouse, BigQuery, Druid).

"What does a WAL do?"
Log the change before applying it. After a crash, replay the log. The same idea powers replication (Ch 5) and CDC (Ch 11).

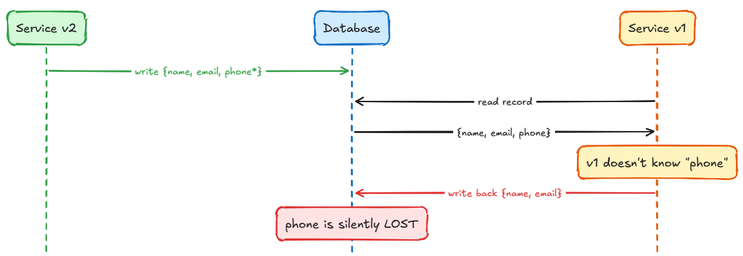
System Design, Step by Step · CC BY-SA 4.0

DDIA 4 · ENCODING & EVOLUTION

Data outlives code. During a rolling deploy, old and new versions of your service run at the same time, reading each other's data.

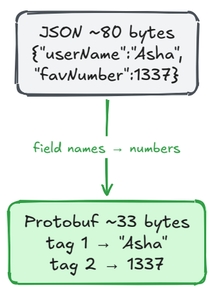
backward compat | forward compat | Protobuf field tags | Avro schemas

ROLLING UPGRADE



v1 updates the name and writes the whole object back, and phone is gone. Old code must preserve unknown fields.

PROTOBUF / THRIFT



✓ Add field: new tag 3
old code skips unknown tag

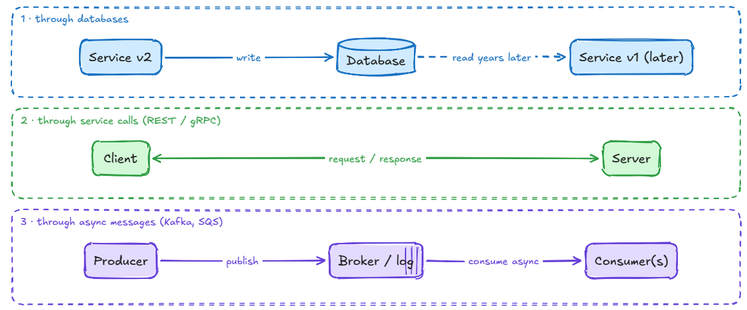
✓ Remove field: never reuse tag
mark reserved

× Make new field required
old data lacks it → crash

× Change a tag number
data decodes to wrong field

Breaking changes: required fields, renumbering tags, or changing a type in an incompatible way.

THREE WAYS DATA FLOWS BETWEEN



Messages: the sender doesn't wait. A broker buffers, retries, and fans out. Producer and consumer evolve independently, so a schema registry keeps them compatible.

IN THE INTERVIEW

"How do you deploy schema changes with zero downtime?"
Expand → migrate → contract: add the nullable column, deploy code that writes both, backfill, switch reads, and only then drop the old column.

"How do Kafka producers and consumers evolve?"
Avro or Protobuf with a schema registry enforcing backward-compatible changes. Consumers upgrade on their own schedule.

"REST or gRPC between services?"
gRPC (Protobuf, HTTP/2) for internal high-QPS calls; REST/JSON for public APIs and browsers.

"Every message carries a schema version, and we only add optional fields. That lets the booking service deploy without coordinating with its consumers."

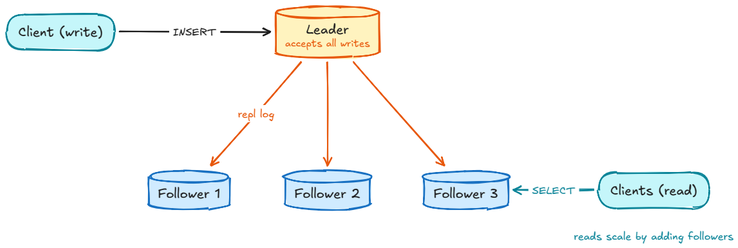
System Design, Step by Step · CC BY-SA 4.0

DDIA 5 · REPLICATION

Keep a copy of the same data on several machines: for fault tolerance (a node dies, a copy survives), read scaling (spread reads), and latency (a copy near the user).

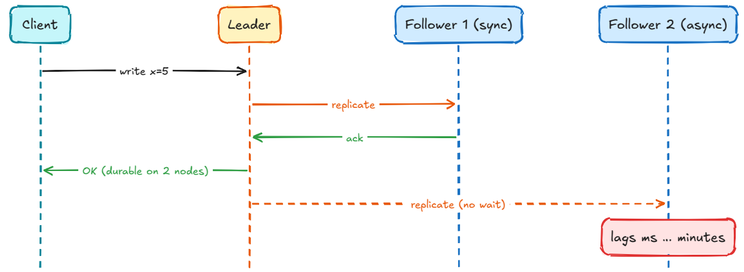
leader / follower | sync vs async | failover | replication lag

SINGLE-LEADER REPLICATION



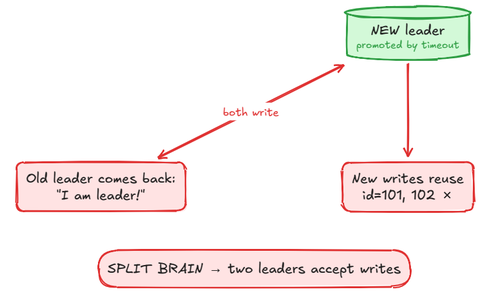
Reads can go to any follower. This is read scaling, but followers may be a little behind: replication lag.

SYNCHRONOUS VS ASYNCHRONOUS FOLLOWERS



The async follower is updated in the background. It is fast, but if the leader dies now, writes the async follower never got are lost.

FAILOVER

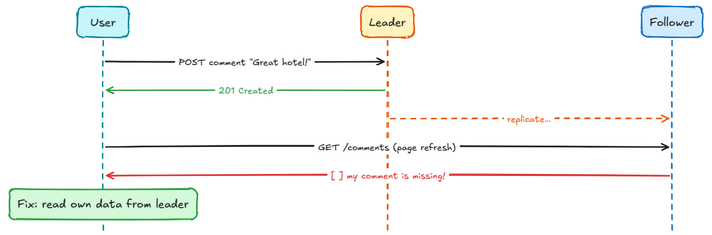


If the old leader returns and still thinks it is leader, there are two leaders: split brain. A fix is fencing: shut it down (STONITH) or reject its epoch number (Ch 8).

HOW THE REPLICATION LOG IS IMPLEMENTED

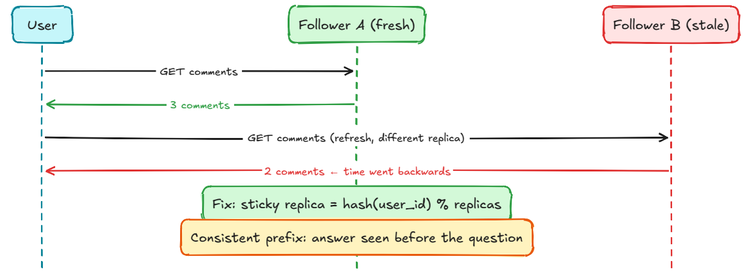
Method	What is sent	Problem / benefit
Statement-based	Raw SQL: UPDATE … SET ts=NOW()	NOW(), RAND(), auto-increment differ per replica, so data diverges. Mostly abandoned.
WAL shipping	Raw storage-engine bytes (Postgres physical)	Exact copy, but tied to the storage format, so replicas must run the same DB version (no zero-downtime upgrade).
Logical (row-based)	"row id=7: col price 100→120"	Decoupled from storage, so versions can differ.
Trigger-based	App-level trigger writes a change table	Flexible (replicate a subset, transform) but slow and fragile.

REPLICATION LAG ANOMALY 1



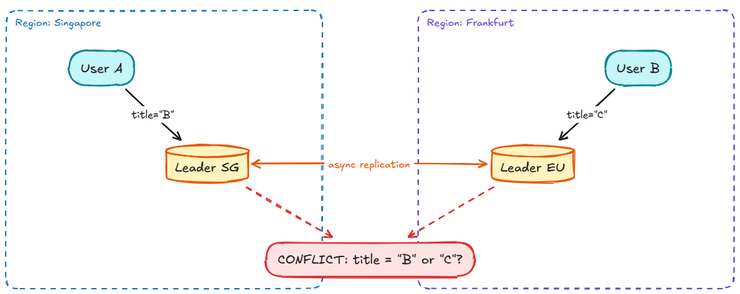
The fixes give read-after-write consistency: read the user's own profile from the leader; for about 1 minute after a write, route that user to the leader; or have the client remember its last write LSN and read only from a replica that has caught up to it.

LAG ANOMALIES 2 AND 3



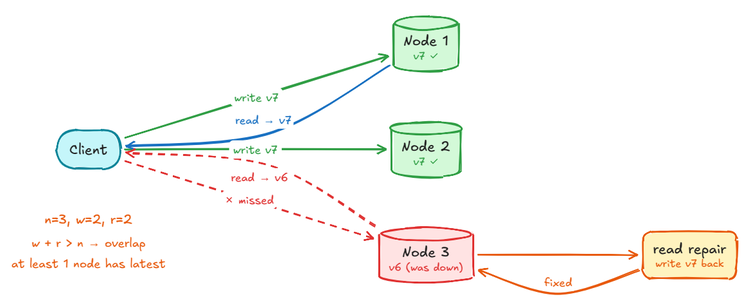
Consistent prefix reads: across partitions a reader can see a reply before the message it replies to.

MULTI-LEADER REPLICATION



When the changes meet, there is a conflict. Resolution options: last-write-wins (silently loses data), merge values, CRDTs, or ask the user.

LEADERLESS REPLICATION AND QUORUMS



Read repair writes the latest value back to the stale node. A background anti-entropy process (Merkle trees) fixes values that are never read.

IN THE INTERVIEW

"Your DB primary dies. What happens?"
Automated failover promotes a replica (Patroni, RDS Multi-AZ, ~30s).

"User updates a price and refreshes but sees the old price."
Replication lag. Read-your-writes: route that user to the primary for N seconds, or track LSN.

"Multi-region active-active?"
Multi-leader with conflict risk. Prefer a home region per user or hotel, and CRDTs for things like counters.

"Why Cassandra over Postgres replicas?"
Leaderless: no failover pause, tunable consistency, write availability during node loss. The cost is conflicts and LWW.

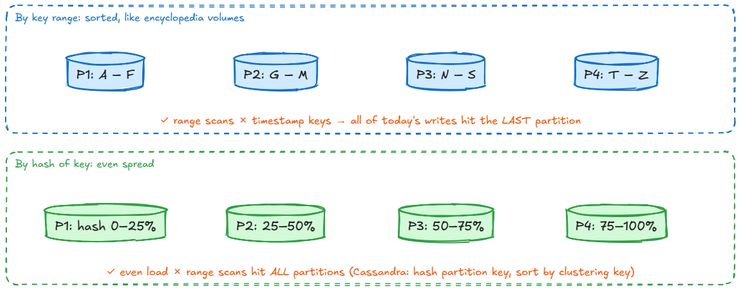
System Design, Step by Step · CC BY-SA 4.0

DDIA 6 · PARTITIONING (SHARDING)

When data or write load is too big for one machine, split it. Each record belongs to exactly one partition.

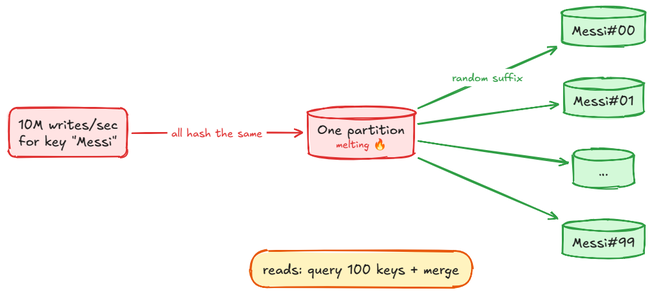
key-range | hash partitioning | hot spots | secondary indexes

KEY-RANGE VS HASH PARTITIONING



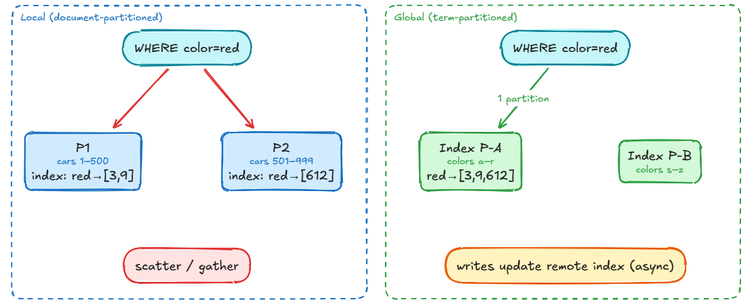
Hash: hash(key) spreads keys evenly, so load is uniform. You lose range queries on the key.

SKEW AND HOT SPOTS



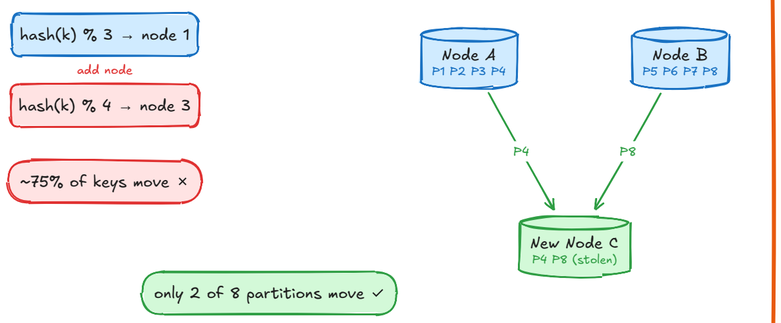
The cost moves to reads, which must fetch all 100 sub-keys and merge. Only do this for keys you know are hot, and track which ones those are.

SECONDARY INDEXES: LOCAL VS GLOBAL



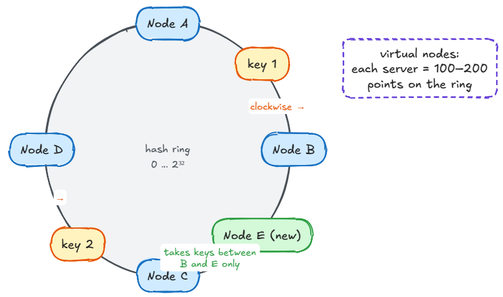
Global: the index itself is partitioned by term.

REBALANCING: NEVER USE HASH MOD N



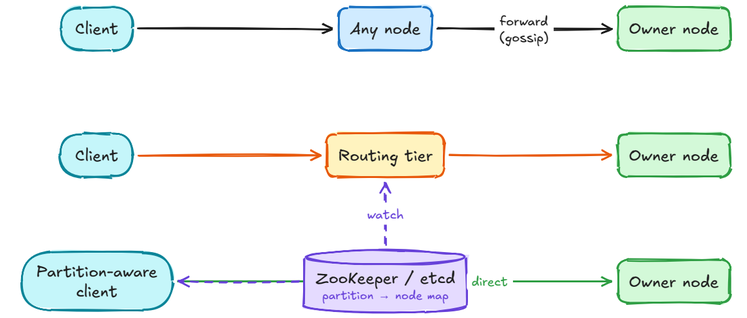
A new node steals a few whole partitions from each existing node. Used by Elasticsearch, Kafka, Riak, and Couchbase.

CONSISTENT HASHING: THE RING



With few nodes, arcs are uneven. Virtual nodes place each server at many points so load evens out, and a bigger server gets more points.

REQUEST ROUTING



The map itself lives in a coordination service (ZooKeeper, etcd, KRaft), and routers or clients watch it for changes.

IN THE INTERVIEW

"How would you shard this?"
Name the shard key from the hottest query: "hash(user_id), because every hot query filters by user". Then name the cost: cross-user queries scatter.

"What about the celebrity or hot hotel?"
Key splitting with a suffix, write buffering in Redis or Kafka, and a cache in front for reads.

"Adding shards later?"
Pre-split into many logical partitions (e.g., 1024) mapped to few physical nodes, or consistent hashing. Never mod N.

A single Postgres handles ~10–50k TPS and multiple TB. Shard only when the calculation says one primary can't take the write load.

System Design, Step by Step · CC BY-SA 4.0

DDIA 7 · TRANSACTIONS

A transaction groups reads and writes into one unit: all of it happens or none of it does, and concurrent transactions don't see each other's half-finished work.

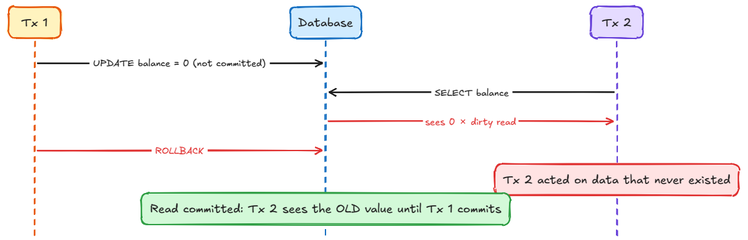
ACID | dirty reads | read skew | MVCC

WHAT ACID REALLY MEANS

Letter	Common belief	What it actually guarantees
A · Atomicity	"Atomic like threads"	Abortability: on any error mid-transaction, all writes are rolled back. Safe to retry.
C · Consistency	"DB keeps data consistent"	Your application's invariants (debits = credits).
I · Isolation	"Transactions are serial"	Concurrent transactions don't interfere.
D · Durability	"Never lost"	Committed data survives crashes (WAL + fsync, or replication). Nothing is 100%.

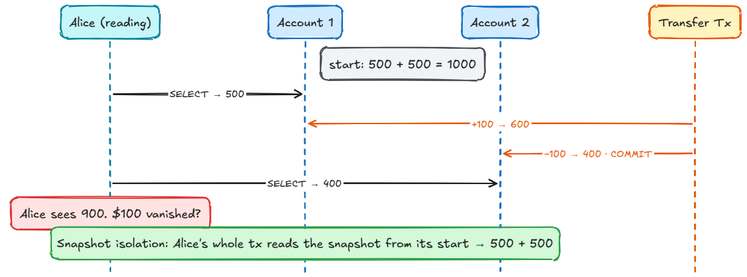
Postgres defaults to Read Committed. MySQL InnoDB defaults to Repeatable Read.

READ COMMITTED



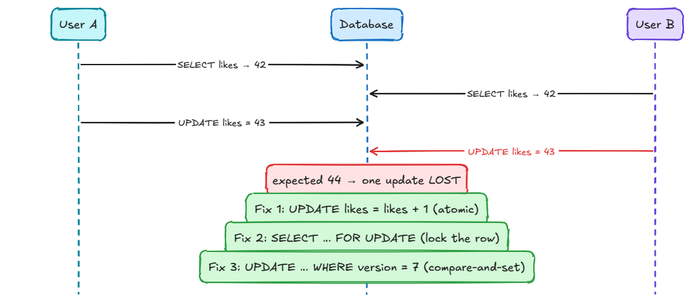
Read committed prevents dirty reads (readers see the last committed value) and dirty writes (row locks until commit). It is the Postgres default.

READ SKEW → SNAPSHOT ISOLATION (MVCC)



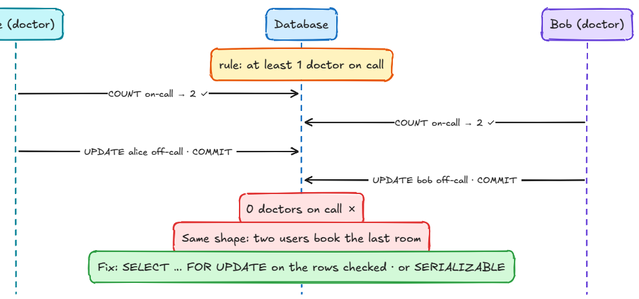
Snapshot isolation: each transaction sees a consistent snapshot as of its start.

LOST UPDATE



Optimistic (CAS): a version column; if 0 rows are updated, retry. Good under low contention. Postgres Repeatable Read also auto-detects lost updates and aborts.

WRITE SKEW: THE DOUBLE-BOOKING BUG

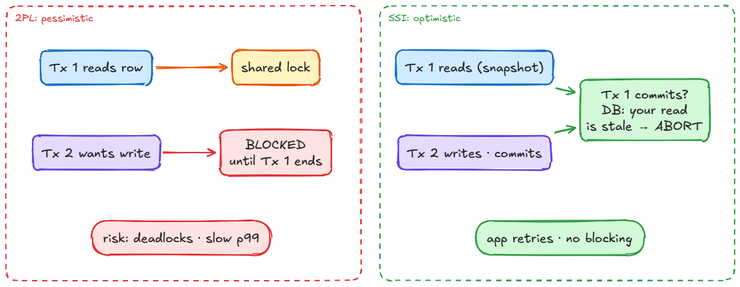


Fixes: lock the rows you checked; use SERIALIZABLE; add a unique constraint; or materialize the conflict by creating a row per seat or room-night so there is something to lock.

ISOLATION LEVELS

Anomaly	Read Committed	Snapshot / Repeatable Read	Serializable
Dirty read / write	✓ prevented	✓	✓
Read skew (non-repeatable read)	×	✓ prevented	✓
Lost update	× (use atomic op / lock)	✓ Postgres detects; MySQL ×	✓
Write skew	×	×	✓ prevented
Phantoms	×	× (mostly)	✓ prevented
Default in	Postgres, Oracle, SQL Server	MySQL InnoDB	CockroachDB, FoundationDB

TWO-PHASE LOCKING VS SERIALIZABLE



SSI: everyone runs on snapshots without blocking. At commit, the DB checks whether anything you read has since changed and, if so, aborts you. Fast at low contention.

IN THE INTERVIEW

"Two users book the last room at the same time."
Write skew. Use a row per room-night + conditional UPDATE, or SELECT FOR UPDATE, or a unique constraint on (room, date).

"Like counter at 50k/sec?"
Not row locks. Use atomic INCR in Redis, sharded counters, or batch into the DB asynchronously.

"Optimistic or pessimistic?"
Low contention: optimistic version check. High contention on few rows (hot seat map): pessimistic lock or a queue to serialize.

"Transaction across 2 services?"
No distributed ACID. Use a saga with compensations plus the outbox pattern (Ch 9 and 12).

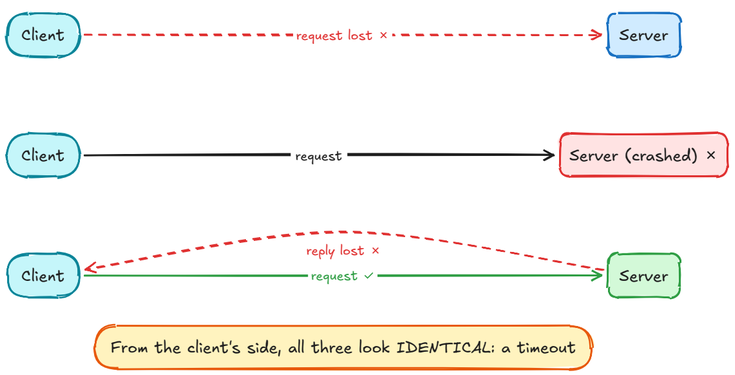
System Design, Step by Step · CC BY-SA 4.0

DDIA 8 · THE TROUBLE WITH DISTRIBUTED SYSTEMS

On one machine, things either work or crash. Across machines you get partial failure: some parts work, some don't, and you often can't tell which.

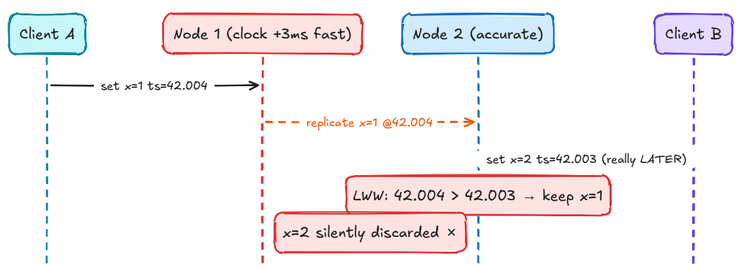
partial failure | timeouts | clock skew | LWW danger

THE NETWORK



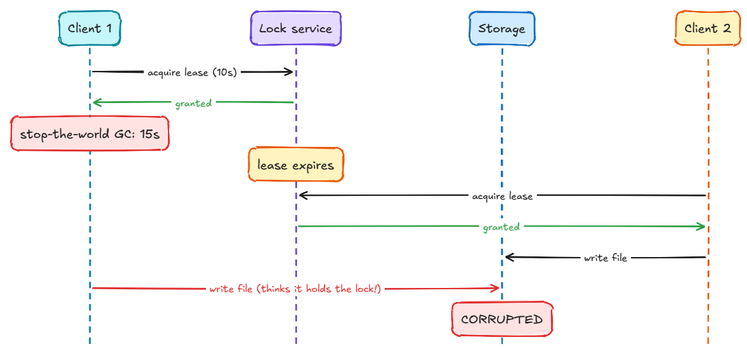
The client only knows "no reply before the timeout". That is why every retried operation must be idempotent.

CLOCKS



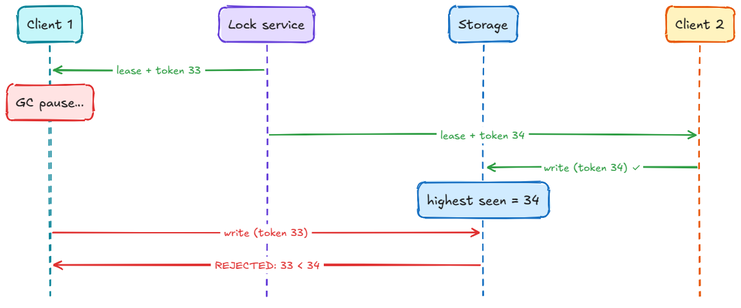
Last-write-wins compares timestamps, and the causally later write loses. No error is raised. NTP accuracy is only tens of ms at best.

PROCESS PAUSES



Client 1 wakes up, checks nothing, and writes. Two writers think they own the lock.

FENCING TOKENS: THE FIX



A late writer with an old token is rejected. The protection is enforced by the resource, not trusted to the client.

SYSTEM MODELS AND BYZANTINE FAULTS

Model	Assumes	Reality
Timing: synchronous	Bounded network delay, pauses, clock drift	Not realistic
Timing: partially synchronous	Usually bounded, sometimes not	The realistic model; algorithms must stay safe when bounds break
Timing: asynchronous	No timing assumptions at all	Very restrictive; no timeouts
Node: crash-stop	Crashed nodes never return	Simplifies proofs
Node: crash-recovery	Nodes crash and return; disk survives, RAM doesn't	Realistic for datacenters
Node: Byzantine	Nodes may lie or act maliciously	Blockchains, aerospace. Rarely needed inside one company.

IN THE INTERVIEW

"What if the payment provider times out?"
Outcome unknown. Retry with the same idempotency key, or reconcile with a status query. Never blindly charge again.

"Distributed lock with Redis?"
Fine for efficiency (avoid duplicate work). For correctness add fencing tokens, or use the DB's own conditional writes.

"Order events by timestamp?"
Wall clocks skew. Use a sequence from one leader, a Kafka partition offset, or logical clocks (Ch 9).

"How do you detect a dead node?"
Heartbeats + timeouts tuned to p99.9, and accept false positives. Leader election by majority, with epochs to fence the old leader.

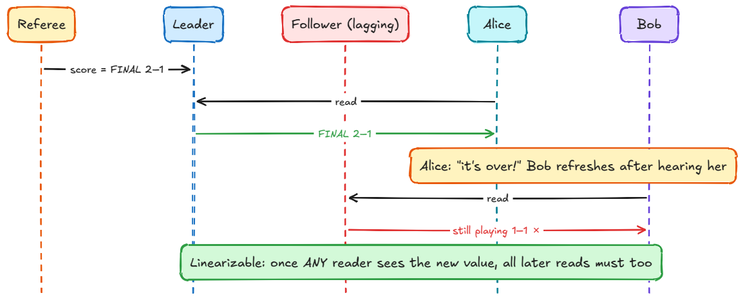
System Design, Step by Step · CC BY-SA 4.0

DDIA 9 · CONSISTENCY & CONSENSUS

Given everything in Ch 8, how do nodes agree?

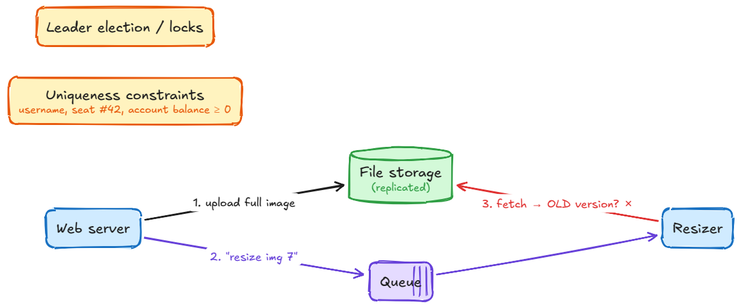
linearizability | CAP | causality | Lamport clocks

LINEARIZABILITY



Linearizability means every operation appears to take effect atomically at one point in time, consistent with real-time order.

WHEN YOU ACTUALLY NEED LINEARIZABILITY



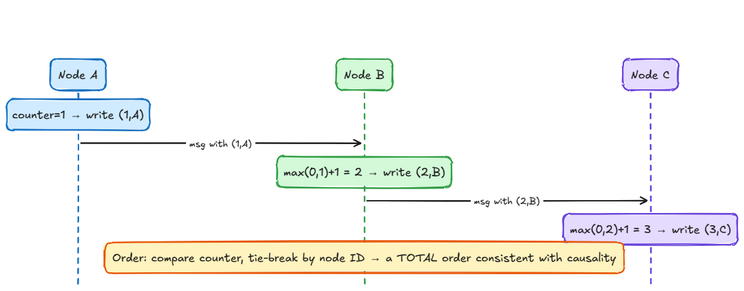
If the resizer reads a lagging storage replica, it resizes the old image. Two channels give a race.

WHICH SYSTEMS ARE LINEARIZABLE, AND

Replication	Linearizable?	Why
Single-leader (reads from leader)	Potentially yes	If reads hit the leader and the leader is really the leader (fencing)
Consensus (Raft / ZooKeeper / etcd)	Yes	Designed for it
Multi-leader	No	Concurrent writes on different leaders
Leaderless (Dynamo / Cassandra)	Probably not	LWW + clock skew, sloppy quorums, timing edge cases

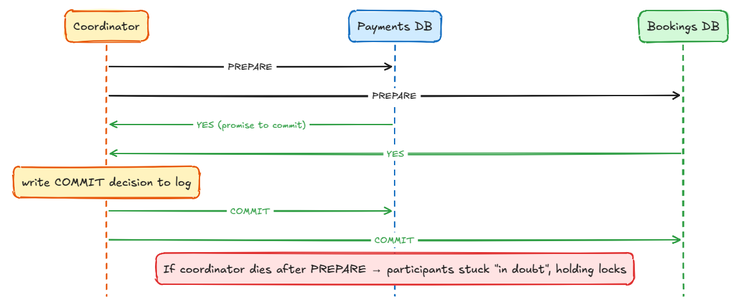
Else, when there is no partition, you trade Latency vs Consistency.

ORDERING: LAMPORT TIMESTAMPS



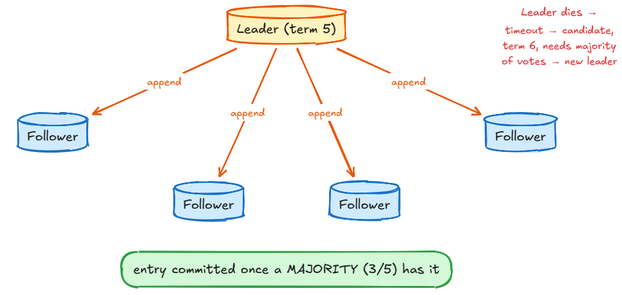
The limit: the order is only known after the fact.

TWO-PHASE COMMIT (2PC)



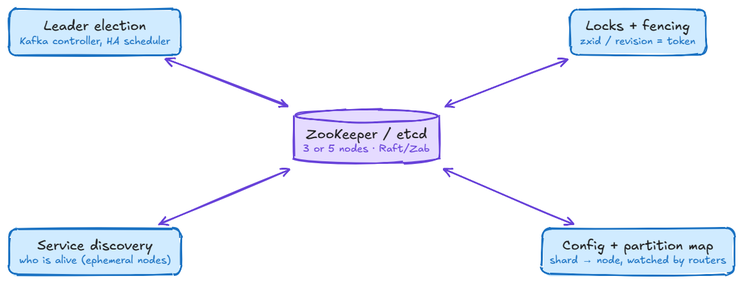
The weakness is that 2PC blocks. If the coordinator dies, participants hold locks until it recovers. This is why microservices use sagas instead.

CONSENSUS: RAFT IN ONE PICTURE



If the leader goes silent, a follower times out, increments the term, and asks for votes. A majority makes it leader. A higher term fences the old leader.

ZOOKEEPER / ETCD



Membership via ephemeral nodes that vanish when a session dies; config and shard maps that clients watch for changes.

IN THE INTERVIEW

"Is this CP or AP?"
Answer per feature: booking and payment are CP, search and feeds are AP. Mention PACELC latency cost.

"Transaction spanning booking + payment services?"
Not 2PC (blocking, coupling). Use a saga + outbox + idempotent consumers.

"Exactly one scheduler must run."
Leader election via ZooKeeper or etcd (consensus), lease + fencing token.

"Unique usernames across regions?"
Needs linearizable compare-and-set: one strongly consistent store (Spanner, CockroachDB, or a single-leader region for usernames).

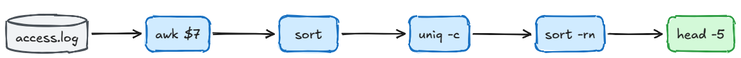
System Design, Step by Step · CC BY-SA 4.0

DDIA 10 · BATCH PROCESSING

Take a big, bounded, immutable input, run a pure function over it, and produce a derived output.

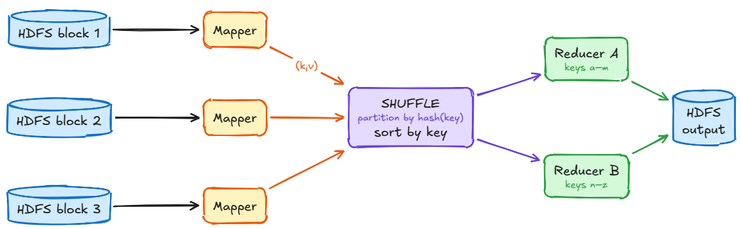
Unix philosophy | MapReduce | HDFS | shuffle

THE UNIX PHILOSOPHY



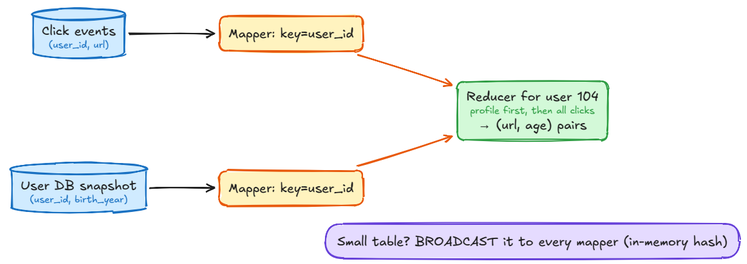
Sort by count and take the top 5. Six tiny programs, composed through one uniform interface: a stream of lines.

MAPREDUCE: MAP, SHUFFLE, REDUCE



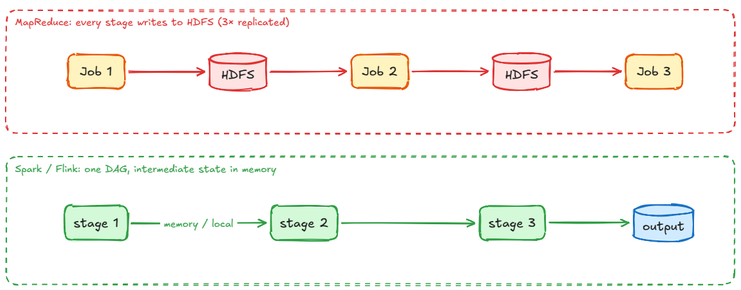
Each reducer sees all values for its keys together, aggregates them, and writes output files. If a task fails, just rerun that task, because inputs are immutable.

JOINS IN BATCH



Broadcast hash join: if one side is small (a hotel dimension table), load it into each mapper's memory. No shuffle is needed.

BEYOND MAPREDUCE



Dataflow engines run the whole workflow as one DAG and pipeline data between operators.

IN THE INTERVIEW

"Web crawler for 10B pages?"
The frontier queue feeds fetchers. Text extraction and dedup run as batch (Spark) over S3. Re-runnable because raw pages are stored immutably.

"Daily top-K or reconciliation reports?"
Nightly batch over the event log in S3 or the warehouse. The streaming path gives fast approximate results and batch gives exact ones (Ch 12, Lambda).

"How do you rebuild the search index?"
Batch-build a new index from the source of truth, then atomically switch the alias. Old index kept for rollback.

"Raw events land immutably in S3, partitioned by date. Any derived table can be recomputed from them, which is our safety net for bugs."

System Design, Step by Step · CC BY-SA 4.0

DDIA 11 · STREAM PROCESSING

Batch with the input never ending. Events are processed shortly after they happen.

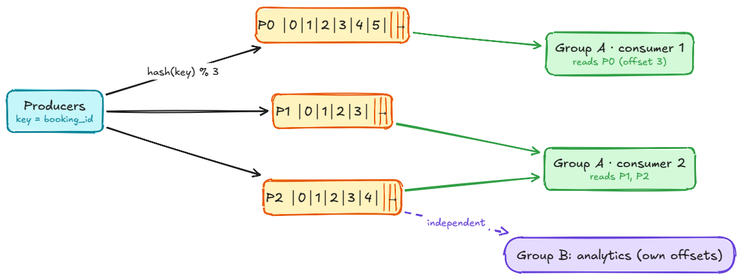
message brokers | Kafka partitions | consumer groups | CDC

TWO KINDS OF MESSAGE BROKER

	Queue (RabbitMQ, SQS, JMS)	Log (Kafka, Kinesis, Pulsar)
After consume	Message deleted once acked	Message stays; consumer just advances an offset
Ordering	Lost under parallel consumers + redelivery	Strict order within a partition
Parallelism	Any consumer takes any message (load balancing)	One consumer per partition per group
Replay	No: gone after ack	Yes: reset offset, re-read days of history
Slow message	Others continue (per-message ack)	Blocks its partition (head-of-line)
Best for	Task queues: send email, resize image, each job independent	Event streams: ordering matters, multiple consumers, replay, CDC

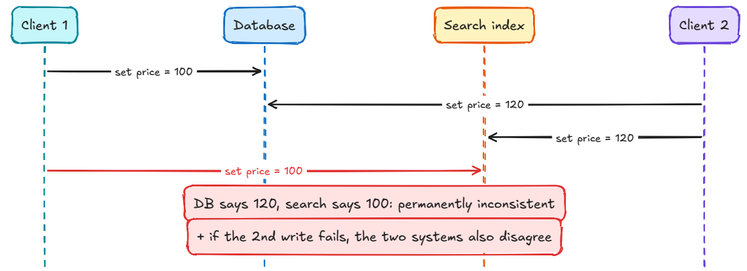
"Image resize jobs go on SQS, since they are independent with per-message retry.

KAFKA



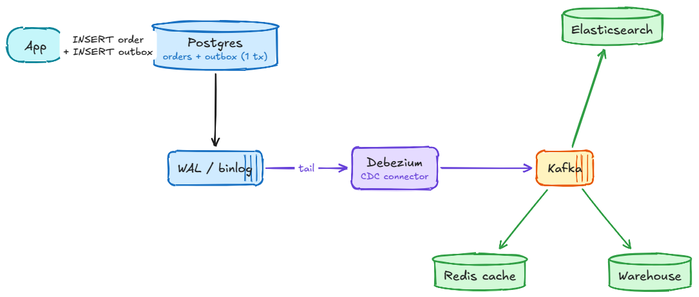
Another group reads the same data independently with its own offsets. Adding consumers doesn't cost the producer anything.

THE DUAL-WRITE PROBLEM



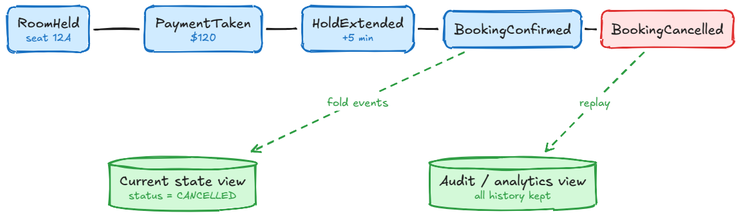
And if the app crashes between the two writes, one system has the change and the other doesn't. Writing to two systems is never atomic.

CHANGE DATA CAPTURE (CDC) AND THE



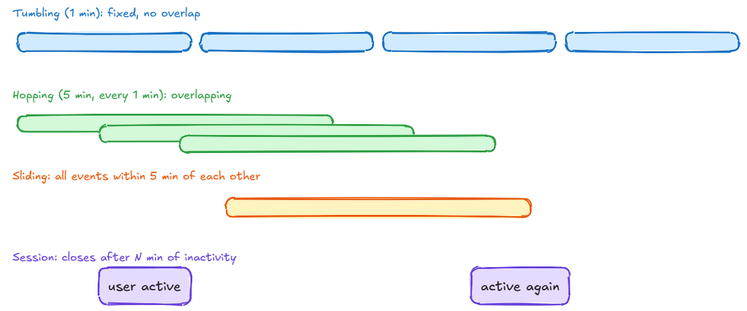
Every derived system (search, cache, warehouse) consumes the log and updates itself. Each can be rebuilt by replaying from offset 0, or from a snapshot plus the log.

EVENT SOURCING



State is a derived view, a fold over the events. You can build new views later by replaying, and debug by replaying to any point in time.

TIME AND WINDOWS

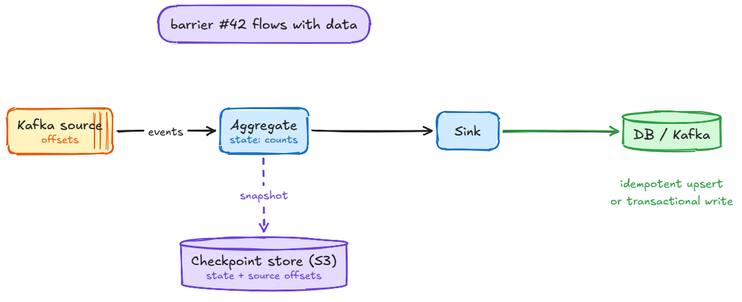


Session: group a user's activity until a gap, as in web analytics and Strava activities.

STREAM JOINS

Join	Example	How
Stream–stream (window join)	Search event + click event within 1 hour → click-through rate	Buffer both streams in state, indexed by session ID, for the window length
Stream–table (enrichment)	Click event + user profile → add country and age	Keep a local copy of the table in the processor, updated by CDC.
Table–table (materialized view)	Tweets ⋈ follows → each user's timeline cache	Both are changelogs; maintain the joined view incrementally (fan-out on write from Ch 1)

EXACTLY-ONCE REALLY MEANS



Replay means some outputs are sent twice.

IN THE INTERVIEW

"Ad click aggregation at 10k/sec?"
Kafka partitioned by ad_id → Flink 1-min tumbling windows on event time → OLAP store. Raw clicks to S3 for batch reconciliation.

"Keep search, cache, and DB in sync?"
CDC (Debezium) or outbox → Kafka → idempotent consumers. Never dual-write.

"Kafka or SQS?"
Ordering, replay, or many consumers → Kafka. Independent tasks with per-message retry and DLQ → SQS.

"Exactly-once?"
"At-least-once delivery plus idempotent processing equals effectively-once. I make the consumer idempotent with an event ID."

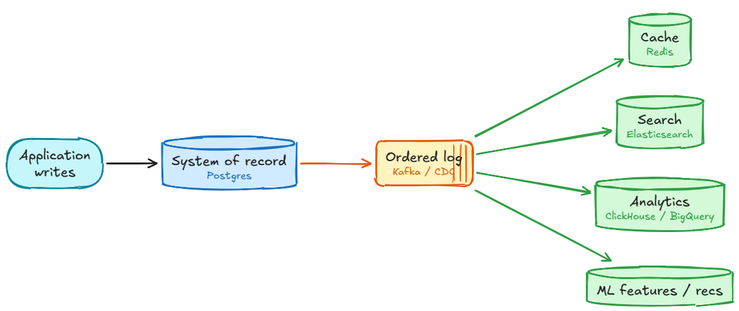
System Design, Step by Step · CC BY-SA 4.0

DDIA 12 · THE FUTURE OF DATA SYSTEMS

The book's conclusion: there is no one database that does everything. Build systems as a source of truth plus derived views connected by logs.

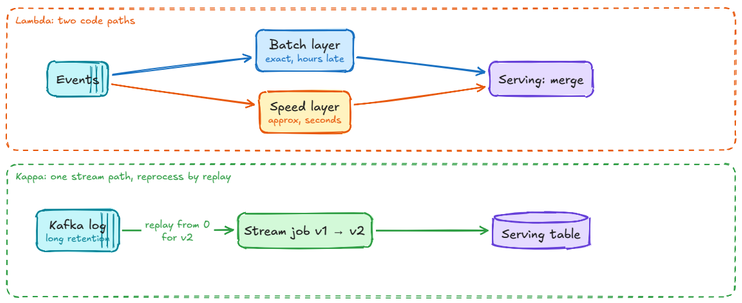
derived data | unbundling the DB | Lambda vs Kappa | end-to-end argument

UNBUNDLING THE DATABASE



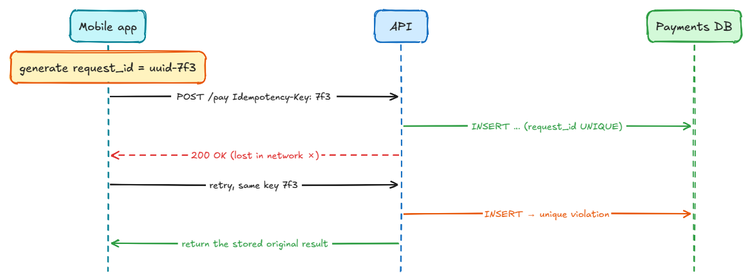
Every other store is a derived view, like an index or materialized view, except in separate specialized systems. Any view can be dropped and rebuilt by replaying the log.

LAMBDA VS KAPPA ARCHITECTURE



Kappa: one streaming codebase. To fix a bug or change logic, start v2 reading the log from the beginning, let it catch up, and switch reads over.

THE END-TO-END ARGUMENT



The duplicate hits the unique constraint, and the server returns the original result. Only an end-to-end ID from the origin catches every duplicate.

TIMELINESS VS INTEGRITY

	Timeliness	Integrity
Means	Users see an up-to-date state	No corruption: no lost or double money, no contradictions
If violated	Temporary and self-healing: "wait and refresh"	Permanent: needs manual repair
Example	Booking shows in "My trips" 2s late	Room sold twice; card charged twice
Mechanism	Read-your-writes, sync replication	Idempotency, unique constraints, atomic log append, reconciliation
Can relax?	Often yes (eventual consistency)	Almost never

IN THE INTERVIEW

"How do you guarantee no double charge?"
Client idempotency key + unique constraint + the payment provider's idempotency + a daily reconciliation job against provider settlement files.

"Search shows a stale price."
"Search is a derived view. Timeliness can lag seconds; the checkout price is read from the source of truth (integrity)."

"How would you change the aggregation logic?"
Kappa: deploy v2 consumer from offset 0 to a new table, compare, switch.

"I separate integrity from timeliness: money and inventory are strongly consistent and audited; everything the user browses is a derived, eventually consistent view."

System Design, Step by Step · CC BY-SA 4.0